



Neon.B0 User Guide

Table of Contents

1	Get Started	3
1.1	Introduction	3
1.2	What You Need	3
1.2.1	Hardware	3
1.2.2	Software	4
1.3	Installation and Setup for Linux	4
1.3.1	IDE	5
1.3.2	Manual Installation	5
1.3.3	Build Your First Project	12
1.3.4	Uninstall GTS-IDF	13
1.4	Installation and Setup for Window	13
1.4.1	IDE	13
1.4.2	Installation Steps	13
1.4.3	Build Applications	17
1.4.4	Debug Setup	18
2	API Reference	21
2.1	Networking APIs	21
2.1.1	Wi-Fi	21
2.1.2	Application Protocols	79
2.1.2.1	MQTT	79
2.1.2.2	SNTP	108
2.1.2.3	HTTP Client	116
2.2	Peripherals API	141
2.2.1	UART	141
2.2.2	GPIO	148
3	Disclaimers	153
	Index	155

List of Figures

Neon.B0 Build and Flash Workflow	4
Board Connection for Flashing	7

Chapter 1

Get Started

1.1 Introduction

Neon.B0 is a system on a chip that integrates the following features:

- Ultra-Low-Power (ULP) & High-Performance Wi-Fi optimized for robust throughput in noisy environments.
- Single 2.4 GHz
- Integrated Bluetooth LE 6.0 & Bluetooth Classic to enable localization, proximity awareness & music streaming use cases.
- Support for Thread & ZigBee mesh networks.
- Integrated high-performance Cortex-M33 processor & comprehensive peripheral suite to build all-in-one IoT devices.
- Processing power to enable Edge AI applications.
- PSA L2/L3 certification to address cybersecurity threats.
- Ready-to-use solutions with SDKs, hardware kits & tools reducing time-to-market.
- Global regulatory certifications & market specific interoperability certifications – Matter, AWS, Azure, Aliro, etc.

Neon.B0 offers excellent power efficiency, RF performance, security, and reliability, making it suitable for a wide range of application scenarios and power consumption requirements.

IIoT provides basic hardware and software resources to help application developers realize their ideas using the Neon.B0 hardware. The software development framework by IIoT is intended for the development of Internet-of-Things (IoT) applications with Wi-Fi, Bluetooth, power management, and several other system features.

1.2 What You Need

1.2.1 Hardware

- Neon.B0 board
- Type-C USB cable
- INP 3000 5.0 (SWD) Programmer Board with USB cable - USB A to micro-USB B
- Computer running Linux

1.2.2 Software

To start using GTS-IDF on Neon.B0, install the following software:

- **Toolchain** to compile code for Neon.B0
- **Build tools** - CMake and Ninja to build a full **Application** for Neon.B0
- **GTS-IDF** that essentially contains API (software libraries and source code) for Neon.B0 and scripts to operate the **Toolchain**

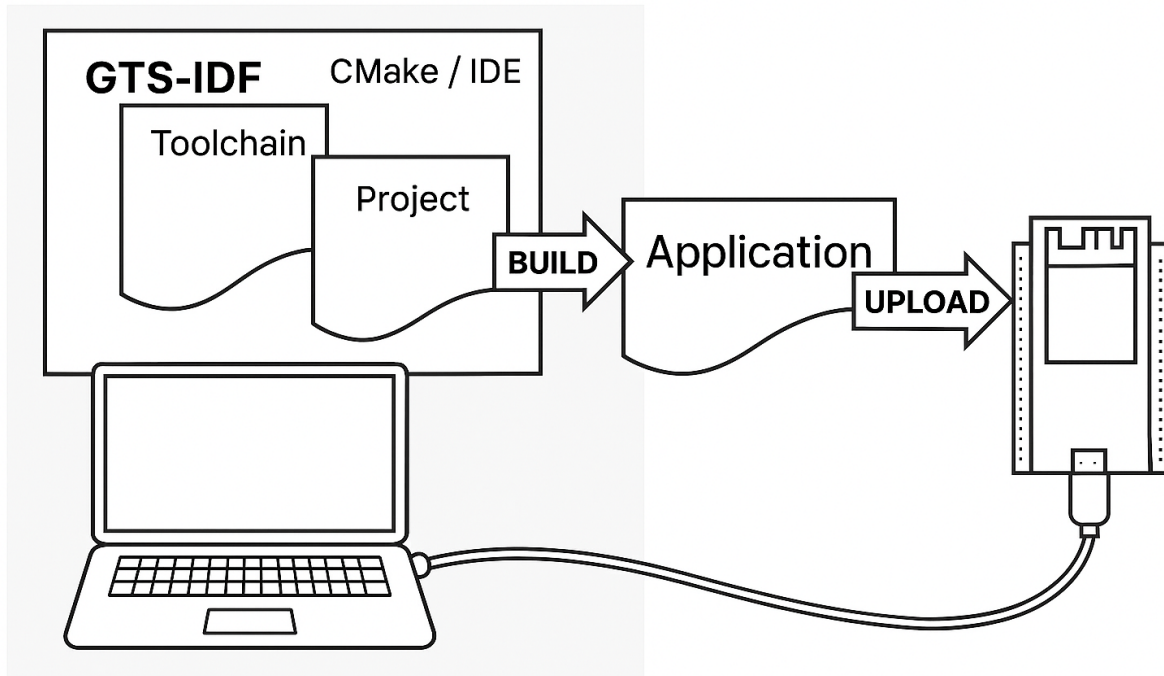


Fig. 1: Neon.B0 Build and Flash Workflow

1.3 Installation and Setup for Linux

To install all the required software, we offer some different ways to facilitate this task. Choose the following available option:

1.3.1 IDE

- IIoT VSCode Extension

1.3.2 Manual Installation

For the manual procedure, please select according to your operating system.

Standard Toolchain Setup for Linux

Installation Step by Step This is a detailed roadmap to walk you through the installation process.

These are the steps for setting up the GTS-IDF for your Neon.B0.

- Step 1. Install Prerequisites
- Step 2. Get GTS-IDF
- Step 3. Set up the Tools
- Step 4. Set up the Environment Variables
- Step 5. First Steps on GTS-IDF

Step 1. Install Prerequisites

To use GTS-IDF with Neon.B0, you need to install some software packages based on your Operating System. This setup guide helps you with getting everything installed on Linux based systems.

For Linux Users

To compile using GTS-IDF, you need to get the following packages. The command to run depends on which distribution of Linux you are using:

- Ubuntu and Debian:

```
sudo apt-get install git wget flex bison gperf python3 python3-pip python3-venv \
→cmake ninja-build ccache libffi-dev libssl-dev dfu-util libusb-1.0-0
```

Note: CMake version 3.22 or newer is required for use with GTS-IDF. Run “tools/idf_tools.py install cmake” to install a suitable version if your OS versions does not have one.

If you do not see your Linux distribution in the above list, then please check its documentation to find out which command to use for package installation.

Step 2. Get GTS-IDF

To build applications for Neon.B0, you need the software libraries provided by IIoT.

To get GTS-IDF, navigate to your installation directory and clone the repository with ‘git clone’, following instructions below specific to your operating system.

Open Terminal, and run the following commands:

```
mkdir -p ~/iiot/apu
cd ~/iiot/apu
git clone git@git.innophaseiot.com:skywalker/software/apu_uc.git .
git submodule update --recursive --remote --merge
```


Step 3. Set up the Tools

Aside from the GTS-IDF, you also need to install the tools used by GTS-IDF, such as the compiler, debugger, Python packages, etc., for projects supporting Neon.B0.

```
cd ~/iiot/apu
./install.sh
```

Customizing the Tools Installation Path

The scripts introduced in this step install compilation tools required by GTS-IDF inside the user home directory: `$HOME/.espressif` on Linux. If you wish to install the tools into a different directory, **export the environment variable `IDF_TOOLS_PATH` before running the installation scripts**. Make sure that your user account has sufficient permissions to read and write this path.

Note: Using `IDF_TOOLS_PATH` in variable assignment, e.g., `IDF_TOOLS_PATH="$HOME/required_idf_tools_path" ./install.sh`, without prior exporting, will not work in most shells because the variable assignment will not affect the current execution environment, even if it's exported/changed in the sourced script.

Step 4. Set up the Environment Variables

The installed tools are not yet added to the `PATH` environment variable. To make the tools usable from the command line, some environment variables must be set. GTS-IDF provides another script which does that.

In the terminal where you are going to use GTS-IDF, run:

```
source export.sh
```

If you plan to use `esp-idf` frequently, you can create an alias for executing `export.sh`:

Copy and paste the following command to your shell's profile (`.profile`, `.bashrc`, `.zprofile`, etc.)

```
alias get_idf='. $HOME/iiot/apu/export.sh'
```

Refresh the configuration by restarting the terminal session or by running `source [path to profile]`, for example, `source ~/.bashrc`.

Now you can run `get_idf` to set up or refresh the `esp-idf` environment in any terminal session.

Technically, you can add `export.sh` to your shell's profile directly; however, it is not recommended. Doing so activates IDF virtual environment in every terminal session (including those where IDF is not needed), defeating the purpose of the virtual environment and likely affecting other software.

Step 5. First Steps on GTS-IDF

Now since all requirements are met, the next topic will guide you on how to start your first project.

This guide helps you with the first steps using GTS-IDF. Follow this guide to start a new project on the neon.B0 and build, flash, and monitor the device output.

Note: If you have not yet installed GTS-IDF, please go to Installation and follow the instruction in order to get all the software needed to use this guide.

Start a Project

Now you are ready to prepare your application for neon.B0. You can start with `wifi/iperf3` project from examples directory in GTS-IDF.

Important: The GTS-IDF build system does not support spaces in the paths to either GTS-IDF or to projects.

Navigate to the ~/iiot/apu directory:

```
cd ~/iiot/apu/examples/wifi/iperf3
```

Note: There is a range of example projects in the **examples** directory in GTS-IDF. You can copy any project in the same way as presented above and run it. It is also possible to build examples in-place without copying them first.

Connect Your Device

OpenOCD Installation

OpenOCD can be installed using the following command:

```
sudo apt install openocd -y
```

You can verify the installation with:

```
which openocd
```

If OpenOCD is installed successfully, its installation path should be displayed.

Board Connection for Flashing

PC connects to the INP3000 over USB, and the INP3000 interfaces with the target board through the SWD debug connection as below

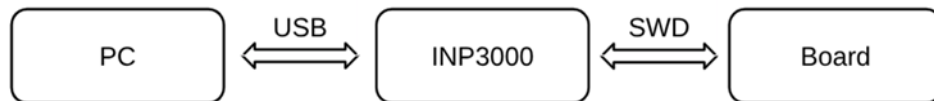


Fig. 2: Board Connection for Flashing

Configure Your Project

Navigate to your wifi/iperf3 directory, set **neon-b0** as the target, and run the project configuration utility **menuconfig**.

```
cd ~/iiot/apu/examples/wifi/iperf3
idf.py set-target neon-b0
idf.py menuconfig
```

After opening a new project, you should first set the target with **idf.py set-target neon-b0**. Note that existing builds and configurations in the project, if any, are cleared and initialized in this process. The target may be saved in the environment variable to skip this step at all. See Select the Target Chip: set-target for additional information.

If the previous steps have been done correctly, the following menu appears:

```

(Top)
Innophase IoT Development Framework Configuration
Build type ---->
Example Configuration ---->
Compiler options ---->
Component config ---->
[ ] Make experimental features visible (default value)

[Space/Enter] Toggle/enter  [ESC] Leave menu          [S] Save
[O] Load                   [?] Symbol info            [/] Jump to symbol
[F] Toggle show-help mode  [C] Toggle show-name mode  [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced) [R] Reset to default value

```

You are using this menu to set up project specific variables, e.g., Wi-Fi network name and password, the processor speed, etc.

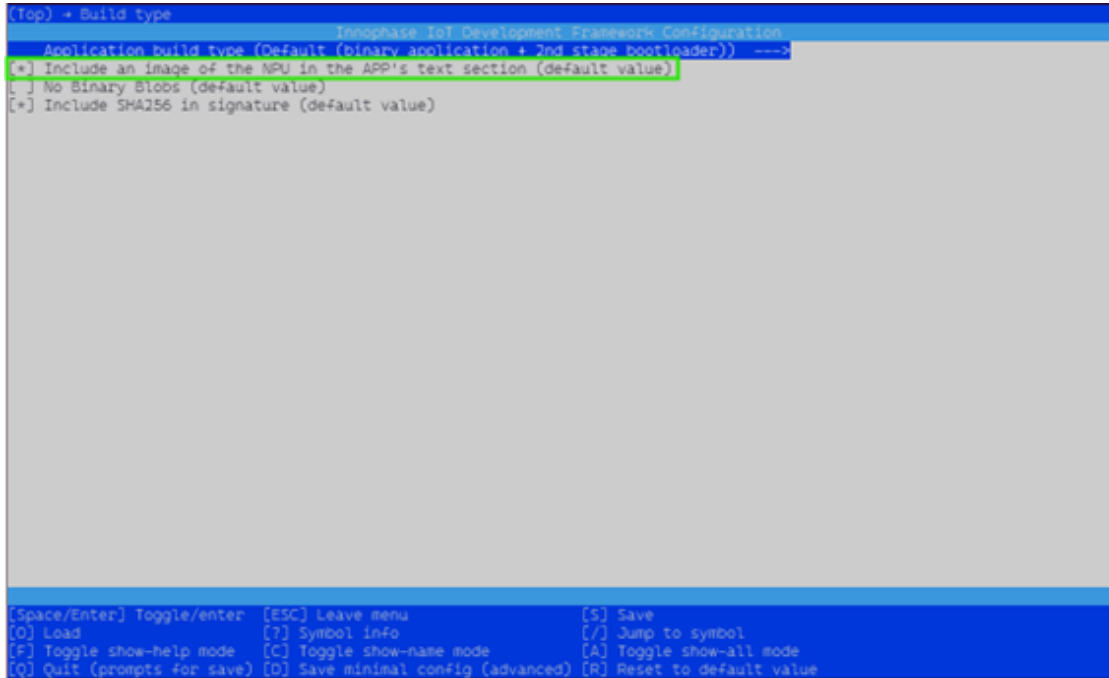
```

(Top) -> Example Configuration -> STA Configuration
Innophase IoT Development Framework Configuration
*** STA Configuration ***
(InnoPhase) WiFi Remote AP SSID (default value)
(InnoPhase) WiFi Remote AP Password (default value)
(F8:F3:F4:8C:FB:22) WiFi STA MAC Address (default value)
(S) Maximum retry (default value)
WiFi Scan auth mode threshold (WPA2 PSK) ---->

[Space/Enter] Toggle/enter  [ESC] Leave menu          [S] Save
[O] Load                   [?] Symbol info            [/] Jump to symbol
[F] Toggle show-help mode  [C] Toggle show-name mode  [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced) [R] Reset to default value

```

Select the option “Include an image of the NPU in the app’s text section” as shown below.



Build the Project

Build the Project by running:

```
idf.py fullclean
idf.py build
```

This command compiles the application and all GTS-IDF components, then it generates the application binary in ELF format.

```
$idf.py build
Executing action: all (aliases: build)
Running cmake in directory /home/iio/apu/examples/wifi/iperf3/build
Executing "cmake -G Ninja -DPYTHON_DEPS_CHECKED=1
-
DPYTHON=/home/bill/.espressif/python_env/idf6.0_py3.12_env/bin/python -DGTS_
↳ PLATFORM=1 -DCCACHE_ENABLE=0
/home/iio/apu/examples/wifi/iperf3"...
-- IDF_TARGET is not set, guessed 'neon-b0' from sdkconfig
'/home/iio/apu/examples/wifi/iperf3/sdkconfig'
-- Found Git: /usr/bin/git (found version "2.43.0")
-- Set \_\_COMPONENT_REQUIRES_COMMON:
freertos;soc;sdk_common;log;inph_hal;inph_event;lwip;inph_netif;inph_wifi;
↳ inph_hw_support;heap
CMake Warning at /home/iio/apu/tools/cmake/git_submodules.cmake:52
↳ (message):
Call Stack (most recent call first):
/home/iio/apu/tools/cmake/build.cmake:109 (git_submodule_check)
/home/iio/apu/tools/cmake/build.cmake:283 (\_\_build_get_idf_git_revision)
/home/iio/apu/tools/cmake/idf.cmake:54 (\_\_build_init)
/home/iio/apu/tools/cmake/project.cmake:29 (include)
CMakeLists.txt:5 (include)
-- Minimal build - ON
-- The C compiler identification is GNU 14.2.1
```

(continues on next page)

(continued from previous page)

```
-- The CXX compiler identification is GNU 14.2.1
-- The ASM compiler identification is GNU
-- Found assembler:
/opt/neon_tools/arm-gnu-toolchain-14.2.rel1-x86_64-arm-none-eabi/bin/arm-
↳none-eabi-gcc
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler:
/opt/neon_tools/arm-gnu-toolchain-14.2.rel1-x86_64-arm-none-eabi/bin/arm-
↳none-eabi-gcc
- skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler:
/opt/neon_tools/arm-gnu-toolchain-14.2.rel1-x86_64-arm-none-eabi/bin/arm-
↳none-eabi-g++
- skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Building GTS-IDF components [main] for target neon-b0
-- target neon-b0: ===== Innophase ARM Cortex-M33 Processor!
=====
-- Project sdkconfig file
/home/iidot/apu/examples/wifi/iperf3/sdkconfig
Configuration Report

Parser Version: 1
Verbosity: default
Defaults policy: sdkconfig
Status: Finished successfully

-- Adding linker script
/home/iidot/apu/components/soc/neon-b0/ld/neon-b0.peripherals.ld
-- Firmware Signature: Git Hash = 8b85cb91 (dirty)
-- Firmware Signature: Build Time = 2026-01-14 22:35:45 UTC
-- Components: arm-cm33 embeddedRPC freertos fw_signature heap
inph_drivers inph_event inph_hal inph_hw_support inph_netif
inph_system inph_wifi log lwip main sdk_common soc streaming_bridge
-- Component paths: /home/iidot/apu/components/arm-cm33
/home/iidot/apu/components/embeddedRPC
/home/iidot/apu/components/freertos
/home/iidot/apu/components/fw_signature
/home/iidot/apu/components/heap
/home/iidot/apu/components/inph_drivers
/home/iidot/apu/components/inph_event
/home/iidot/apu/components/inph_hal
/home/iidot/apu/components/inph_hw_support
/home/iidot/apu/components/inph_netif
/home/iidot/apu/components/inph_system
/home/iidot/apu/components/inph_wifi
/home/iidot/apu/components/log
/home/iidot/apu/components/lwip
/home/iidot/apu/examples/wifi/iperf3/main
/home/iidot/apu/components/sdk_common
/home/iidot/apu/components/soc
```

(continues on next page)

(continued from previous page)

```

/home/iiot/apu/components/streaming_bridge
-- Configuring done (4.8s)
-- Generating done (0.0s)
-- Build files have been written to:
/home/iiot/apu/examples/wifi/iperf3/build
Running ninja in directory /home/iiot/apu/examples/wifi/iperf3/build
Executing "ninja all"...
[212/212] Linking CXX executable iperf3.elf
Done

```

If there are no errors, the build finishes by generating the firmware binary .elf file.

Flash onto the Device

To enable GTS-IDF flashing, you must first grant permission to access the USB device using the following commands:

```

sudo chmod a+rw /dev/ttyUSB\*
sudo chmod 666 /dev/bus/usb/\*/\*

```

Next, obtain the FTDI device serial number using the following command:

```

sudo lsusb -v

```

You should see the serial number displayed, as shown below



```

Bus 003 Device 101: ID 0403:6011 Future Technology Devices International, Ltd FT4232H Quad HS USB-UART/FIFO IC
Device Descriptor:
  bLength                18
  bDescriptorType         1
  bcdUSB                  2.00
  bDeviceClass             0 [unknown]
  bDeviceSubClass          0 [unknown]
  bDeviceProtocol          0
  bMaxPacketSize0         64
  idVendor                 0x0403 Future Technology Devices International, Ltd
  idProduct                0x6011 FT4232H Quad HS USB-UART/FIFO IC
  bcdDevice                8.00
  iManufacturer           1 FTDI
  iProduct                 2 InnoPhase T2 Evaluation Board
  iSerial                  3 2025-441
  bNumConfigurations      1

```

Navigate to the *iperf3* project directory and build the image according to section *Build the Project*. Then flash the device using (the argument sn is the FDTI serial number):

```

idf.py flash --sn=2025-441

```

If the flashing process is completed successfully, the output will appear similar to the information shown below.

```

writing imgFile: /home/iiot/apu/examples/wifi/iperf3/build/iperf3.bin
Start writing: /home/iiot/apu/examples/wifi/iperf3/build/iperf3.bin
702016 bytes written lost connection with openocd HioServer shutdown success
Done

```

Note: If you encounter the error "magic check failed", unplug jumper **J2**, press the reset button, reconnect jumper **J2**, and then run the flash command again.

Monitor the Output

Output Console

- Connect hardware:
 - Attach the USB-to-Serial FTDI board to your Linux PC.
 - Connect the board's GPIO5 pin to the UART Tx pin.
 - Connect FTDI's GND to the board's GND.
- Verify connection:
 - Once plugged in, the FTDI interface should appear on the PC as /dev/ttyUSB*.
- Open serial console:
 - Launch minicom on the Linux PC.
 - Configure the port to /dev/ttyUSB*.
 - Set the baud rate to 921600.
- Start communication:
 - Begin monitoring the console output through the UART link.

Additional Tips

Python Compatibility

GTS-IDF supports Python 3.8 or newer. It is recommended to upgrade your operating system to a recent version satisfying this requirement. Other options include the installation of Python from sources or the use of a Python version management system such as pyenv.

Flash Erase

Erasing the flash is also possible. To erase the entire flash memory, you can run the following command:

```
idf.py erase-flash --sn=2025-441
```

Failed to Start OpenOCD

If GTS-IDF flashing shows **“Failed to start OpenOCD”**, please unplug and reconnect the INP3000 board, then try flashing again.

```
xPack OpenOCD, x86_64 Open On-Chip Debugger 0.10.0+dev (2020-10-13-17:27)
Licensed under GNU GPL v2
For bug reports, read http://openocd.org/doc/doxygen/bugs.html
Failed to start OpenOCD: OpenOCD not able to connect to device.
Done
```

1.3.3 Build Your First Project

If you already have the GTS-IDF installed and are not using an IDE, you can build your first project from the command line following the *Start a Project on Linux*.

1.3.4 Uninstall GTS-IDF

If you want to remove GTS-IDF, please follow *Uninstall GTS-IDF*.

1.4 Installation and Setup for Window

This section talks about installation of IDE, adding custom IDF and tools, building examples using terminal window, setup debug configurations in IDE, and debugging in the target.

1.4.1 IDE

For ease of setup and tools installation, we will be using Espressif-IDE which is based on Eclipse CTD for working with GTS-IDF framework. Download the Espressif IDE Installer for windows (win32.win32.x86_64) from the below link,

<https://developer.espressif.com/blog/espressif-ide/espressif-ide-3-1-0-release/>

1.4.2 Installation Steps

(The <username> in below steps will be your windows username.)

1. Install python from <https://www.python.org/downloads/release/python-3130/>.

- a. Open a CMD prompt, check the version and make sure it's 3.13.0, shown below:

```
C:\Users\<username>>C:\Users\<username>\AppData\Local\Programs\Python\
Python313\python.exe --version

Python 3.13.0
```

2. By default, the IDE expects and installs all tools in *C:\Users\<username>\.espressif* folder. The same will be used as our IDF tools installation folder.

- a. Create a python env for the IDF installation in the same directory. In the CMD prompt, type the command below:

```
C:\Users\<username>>C:\Users\<username>\AppData\Local\Programs\Python\
Python313\python.exe -m venv
C:\Users\<username>\.espressif\python_env\idf6.0_py3.13_env
```

3. Add the arm gcc path – “*C:\Users\<username>\.espressif\tools\arm-none-eabi-gcc\14.3.Rel1\bin*” to System Path (environment variables Path). This will be installed as part of IDF installation, but after installing the IDE will query the version, and to pass that check we make sure the path is already accessible and available before the IDE is open.

4. Open the Espressif IDE and choose a workspace location.

- a. Navigate to Window->Preferences->Espressif, you will see the GTS-IDF tools installation directory path by default pointing to

C:\Users\<username>\.espressif folder as default tools installation folder as mentioned in Step 2. (If you want to use a different directory then change this path and redo steps 2 and 3 with the new directory path).

5. Open the Espressif->GTS-IDF Manager. In the Manager Window, click on Add-GTS-IDF.

- a. In the GTS-IDF Configuration window, check the “Use existing GTS-IDF directory from file system” and point to the “apu_uc” code downloaded from GIT.

- b. The GIT will point to default git.exe, if not available, install git for windows.
- c. For the Python path, choose the python.exe created in step 2. It will be in the following location, "C:\Users\<username>\.espressif\python_env\idf6.0_py3.13_env\Scripts\python.exe"
- d. Click on Finish. At the end of the installation, a list of targets supported by our custom IDF will be listed and it will show "Tools activated". Monitor the console for installation of tools as shown below (below logs are pruned).

```

Installing tools...
This can take a while. Please be patient.
Executing C:\Users\<username>\.espressif\python_env\idf6.0_py3.13_env\Scripts\
python.exe
C:\Users\<username>\Documents\Projects\Unified_Code\apu_uc\tools\idf_tools.py
install all
Updating C:\Users\<username>\.espressif\idf-env.json
Selected targets are: neon, neon-a0, neon-b0, helium
Current system platform: win64
Installing tools: arm-none-eabi-gcc, cmake, openocd-esp32, ninja,
ccache, dfu-util
Installing arm-none-eabi-gcc@14.3.Rel1
Downloading
https://developer.arm.com/-/..media/Files/downloads/gnu/14.3.rel1/binrel/arm-
gnu-toolchain-14.3.rel1-mingw-w64-x86_64-arm-none-eabi.zip
Destination:
C:\Users\<username>\.espressif\dist\arm-gnu-toolchain-14.3.rel1-mingw-w64-x86_
64-arm-none-eabi.zip.tmp
[=====] 100%Done
Extracting
C:\Users\<username>\.espressif\dist\arm-gnu-toolchain-14.3.rel1-mingw-w64-x86_
64-arm-none-eabi.zip
to C:\Users\<username>\.espressif\tools\arm-none-eabi-gcc\14.3.Rel1
Installing cmake@3.30.2
Using GitHub assets mirror for URL:
https://github.com/Kitware/CMake/releases/download/v3.30.2/cmake-3.30.2-
windows-x86_64.zip
=> https://dl.espressif.com/github_assets/Kitware/CMake/releases/download/v3.
30.2/cmake-3.30.2-windows-x86_64.zip
Downloading https://dl.espressif.com/github_assets/Kitware/CMake/releases/
download/v3.30.2/cmake-3.30.2-windows-x86_64.zip
Destination: C:\Users\<username>\.espressif\dist\cmake-3.30.2-windows-x86_64.
zip.tmp
[=====] 100%Done
Extracting C:\Users\<username>\.espressif\dist\cmake-3.30.2-windows-x86_64.
zip to C:\Users\<username>\.espressif\tools\cmake\3.30.2
Installing openocd-esp32@v0.12.0-esp32-20250422
.....
.....
Installing ninja@1.12.1
Using GitHub assets mirror for URL:
https://github.com/ninja-build/ninja/releases/download/v1.12.1/ninja-win.zip
=> https://dl.espressif.com/github_assets/ninja-build/ninja/releases/download/
v1.12.1/ninja-win.zip
Downloading
https://dl.espressif.com/github_assets/ninja-build/ninja/releases/download/v1.
12.1/ninja-win.zip
Destination:
C:\Users\<username>\.espressif\dist\ninja-win-v1.12.1.zip.tmp

```

(continues on next page)

(continued from previous page)

```
[=====] 100%Done
Extracting
C:\Users\<username>\.espressif\dist\ninja-win-v1.12.1.zip to
C:\Users\<username>\.espressif\tools\ninja\1.12.1
Installing ccache@4.11.2
.....
.....
Executing C:\Users\<username>\.espressif\python_env\idf6.0_py3.13_env\Scripts\
python.exe
C:\Users\<username>\Documents\Projects\Unified_Code\apu_uc\tools\idf_tools.py
install-python-env
Python 3.13.0
pip 24.2 from C:\Users\<username>\.espressif\python_env\idf6.0_py3.13_env\Lib\
site-packages\pip
(python 3.13)
Downloading https://dl.espressif.com/dl/esp-idf/esp-idf.constraints.v6.0.txt
Destination: C:\Users\<username>\.espressif\esp-idf.constraints.v6.0.txt.tmp
[=====] 100%Done
Upgrading pip...
Looking in indexes: https://pypi.org/simple, https://dl.espressif.com/pypi
Requirement already satisfied: pip in
c:\users\<username>\.espressif\python_env\idf6.0_py3.13_env\lib\site-packages
(24.2)
Collecting pip
Using cached pip-26.0.1-py3-none-any.whl.metadata (4.7 kB)
Using cached pip-26.0.1-py3-none-any.whl (1.8 MB)
.....
.....
Collecting certifi>=2017.4.17 (from requests->idf-component-manager->-r
C:\Users\<username>\Documents\Projects\Unified_Code\apu_uc\tools\requirements\
requirements.core.txt (line 17))
Using cached https://dl.espressif.com/pypi/certifi/certifi-2026.1.4-py3-none-
any.whl (152 kB)
Using cached pyparsing-3.2.5-py3-none-any.whl (113 kB)
Using cached pyusb-1.3.1-py3-none-any.whl (58 kB)
Using cached windows_curses-2.4.1-cp313-cp313-win_amd64.whl (81 kB)
Using cached requests-2.32.5-py3-none-any.whl (64 kB)
Using cached ruamel_yaml-0.19.1-py3-none-any.whl (118 kB)
Using cached truststore-0.10.4-py3-none-any.whl (18 kB)
Installing collected packages: windows-curses, reedsolo, pyserial, pygdbmi,
pyelftools, intelhex, func_timeout, bitarray, urllib3,
typing-extensions, truststore, ruamel.yaml, PyYAML, pyusb, python-dotenv,
pyparsing, pygments, pycparser, pyclang, psutil,
packaging, mdurl, jsonref, idna, freertos_gdb, esp-idf-panic-decoder,
construct, colorama, charset_normalizer, certifi, bitstring,
annotated-types, typing-inspection, tqdm, requests, pydantic-core, markdown-
it-py, click, cffi, rich, requests-toolbelt, requests-file,
pydantic, cryptography, rich_click, pydantic-settings, esp-idf-size, esp-idf-
nvs-partition-gen, esp-idf-kconfig, esp-idf-diag,
idf-component-manager, esptool, esp-coredump, esp-idf-monitor
.....
Executing C:\Users\<username>\.espressif\python_env\idf6.0_py3.13_env\Scripts\
python.exe
C:\Users\<username>\Documents\Projects\Unified_Code\apu_uc\tools\idf.py --
version
WARNING: Git version unavailable, reading from source
GTS-IDF v6.0.0
```

(continues on next page)

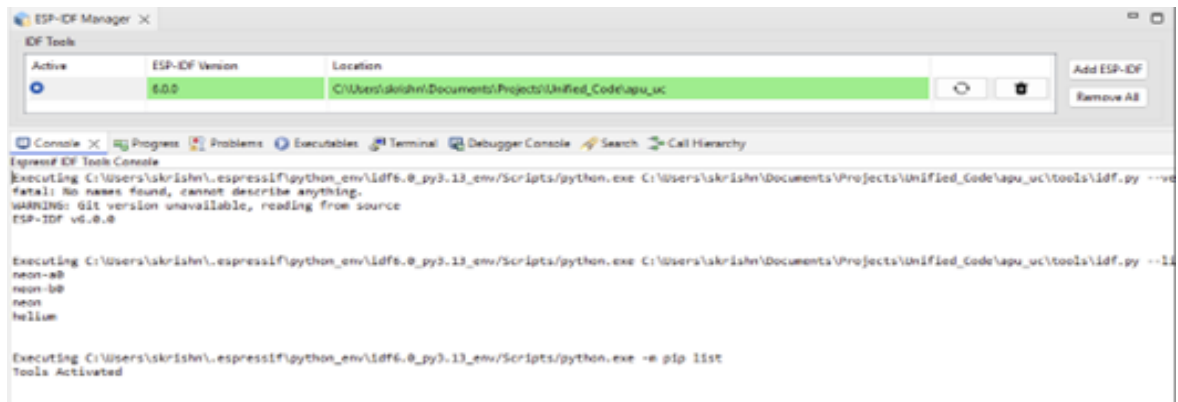
(continued from previous page)

```

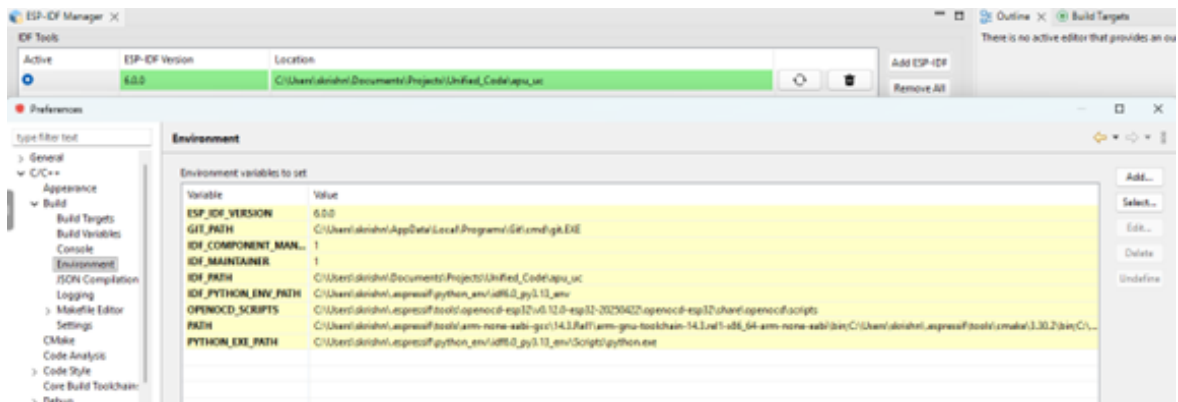
Executing C:\Users\

```

- e. After installation, you should see the custom IDF is active as shown below.



6. At this point the installation is complete (this covers both install.sh and export.sh steps used in Linux setup). Below snapshot shows the custom IDE installation being active and the IDE has set all the Window->Preferences->C/C++->Build->Environment paths.



1.4.3 Build Applications

Build using GTS-IDF Terminal Window:

The steps below show how to build an example application using a terminal window.

1. Open a terminal within the IDE by navigating to Windows->ShowView->Terminal or use the Terminal icon and open an GTS-IDF Terminal. This would automatically inherit all the paths. If required, re-confirm the paths as shown below:

```
C:\Users\<username>\Documents\Projects\Unified_Code\apu_uc>echo%IDF_PATH%
C:\Users\<username>\Documents\Projects\Unified_Code\apu_uc
C:\Users\<username>\Documents\Projects\Unified_Code\apu_uc>echo%IDF_TOOLS_PATH%
C:\Users\<username>\.espressif
C:\Users\<username>\Documents\Projects\Unified_Code\apu_uc>echo%IDF_PYTHON_ENV_
PATH%
C:\Users\<username>\.espressif\python_env\idf6.0_py3.13_env
C:\Users\<username>\Documents\Projects\Unified_Code\apu_uc>where python
C:\Users\<username>\.espressif\python_env\idf6.0_py3.13_env\Scripts\python.exe
C:\Users\<username>\Documents\Projects\Unified_Code\apu_uc>python --version
Python 3.13.0
C:\Users\<username>\Documents\Projects\Unified_Code\apu_uc>where pip
C:\Users\<username>\.espressif\python_env\idf6.0_py3.13_env\Scripts\pip.exe
C:\Users\<username>\Documents\Projects\Unified_Code\apu_uc\examples\peripherals\
pdl_main>where cmake
C:\Users\<username>\.espressif\tools\cmake\3.30.2\bin\cmake.exe
C:\Users\<username>\Documents\Projects\Unified_Code\apu_uc\examples\peripherals\
pdl_main>where ninja
C:\Users\<username>\.espressif\tools\ninja\1.12.1\ninja.exe
```

2. The following steps show how to clean and build an example application (pdl_main) in the terminal window. The idf.py path can be added to system's environment variables, otherwise use the full path as shown below:

- a. Set target; the example sets it to neon-b0 (logs pruned).

Command: *python ..\..\tools\idf.py set-target neon-b0** ***

```
C:\Users\<username>\Documents\Projects\Unified_Code\apu_uc\examples\
peripherals\pdl_main>python ..\..\tools\idf.py set-target neon-b0
Adding "set-target"'s dependency "fullclean" to list of commands with
default set of options.
Executing action: fullclean
Build directory 'C:\Users\<username>\Documents\Projects\Unified_Code\apu_uc\
examples\peripherals\pdl_main\build' not found. Nothing to clean.
Executing action: set-target
Set Target to: neon-b0, new sdkconfig will be created.
Running cmake in directory
.....
-- Configuring done (47.5s)
-- Generating done (3.3s)
-- Build files have been written to:
C:/Users/<username>/Documents/Projects/Unified_Code/apu_uc/examples/
peripherals/pdl_main/build
Done
```

- b. Navigate to the application folder. Full clean command:

python ..\..\tools\idf.py fullclean

```
C:\Users\\Documents\Projects\Unified_Code\apu_uc\examples\
↳peripherals\pdl_main>python ../../tools\idf.py fullclean
Executing action: fullclean
Executing action: remove_managed_components
Done
```

c. Build command: *python ../../tools\idf.py build*

```
C:\Users\\Documents\Projects\Unified_Code\apu_uc\examples\peripherals\
↳pdl_main>python ../../tools\idf.py build
Executing action: all (aliases: build)
Running ninja in directory
C:\Users\\Documents\Projects\Unified_Code\apu_uc\examples\peripherals\
↳pdl_main\build
Executing "ninja all"...
[422/422] Linking CXX executable pdl_main.elfDone
```

- d. Load the project in eclipse using File->Import->Espressif->Existing IDF Projects. In the import window, point the “Existing project location” to the apu_uc code downloaded from GIT and click Finish.
- e. The Project explorer window will show the entire project directory. Navigate to the application folder, apu_uc->examples->peripherals->pdl_main->build folder which will now show “pdl_main.elf” which was generated in step c.
- f. By default, it would build for Flash, to build the code for RAM, open menu config using the command: *python ../../tools\idf.py menuconfig*
 - i. Choose Build Type->Application build type (Build app runs entirely in RAM), save and exit. This would update the required config options in this example, apu_uc/examples/peripherals/pdl_main/sdkconfig file.
 - ii. Repeat steps b and c; the resulting elf file gets compiled to run in RAM.
 - iii. Make sure not to set-target at this point as it will clear config options chosen using “menuconfig” and resets it to default config.

Build using project’s build configurations settings:

TBD

1.4.4 Debug Setup

The following section talks about setting up debug configuration in the IDE. The setup uses JLink probe and custom openOCD.

Prerequisite: Plug in the J-Link probe into your computer via USB. Power on the target and connect the board to J-link using SWD cable.

Debug and Run in RAM or Flash:

The example uses a Neon-B0 ASIC:

1. Open a CMD prompt window, navigate to the folder which has the custom OpenOCD and config files. Start openOCD using the following command:

```
openocd.exe -f jlink.cfg -f t3a_swd.cfg
```

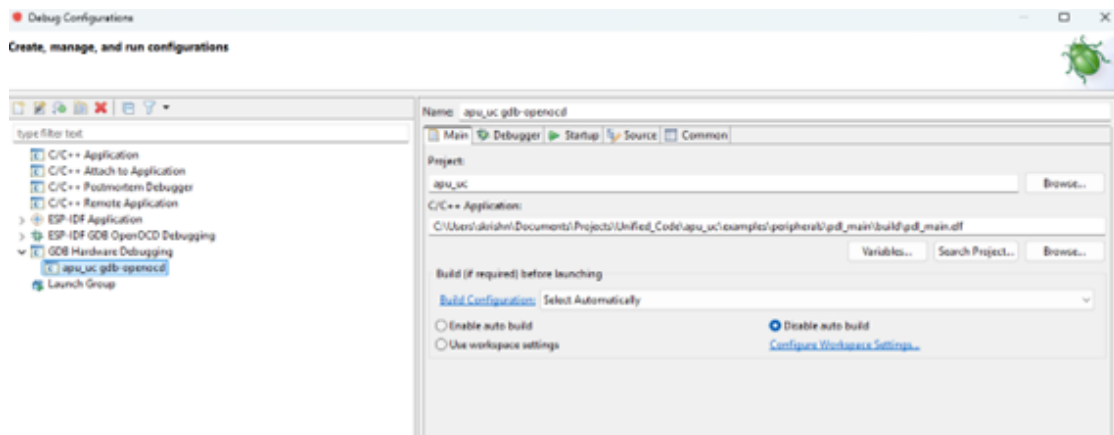
This will start the GDB server and wait for connection as shown below.

```

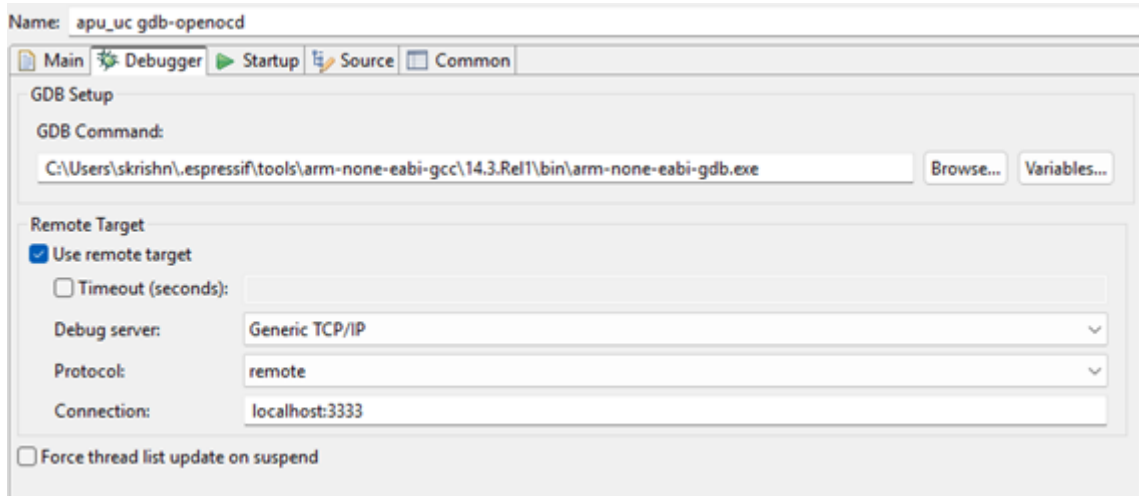
C:\Users\\Desktop\InnoOpenOCD\bin>openocd.exe -f jlink.cfg -f t3a_
↪swd.cfg
Open On-Chip Debugger 0.12.0+dev-01598-gd21ebd053-dirty (2025-07-08-23:12)
Licensed under GNU GPL v2
For bug reports, read
http://openocd.org/doc/doxygen/bugs.html
\*****\
\*****\
Info : Listening on port 6666 for tcl connections
Info : Listening on port 4444 for telnet connections
Info : J-Trace PRO V3 Cortex-M compiled Apr 15 2024 17:39:50
Info : Hardware version: 3.00
Info : VTarget = 1.818 V
Info : clock speed 4000 kHz
Info : SWD DPIDR 0x6ba02477
Info : [inno_t3a.cpu.CPU0] Cortex-M33 r1p0 processor detected
Info : [inno_t3a.cpu.CPU0] target has 8 breakpoints, 4 watchpoints
Info : [inno_t3a.cpu.CPU0] Examination succeed
Info : starting gdb server for inno_t3a.cpu.CPU0 on 3333
Info : Listening on port 3333 for gdb connections

```

2. In the IDE, click on the debug icon (green bug) -> Debug Configuration. Since we are not an GTS32 target, we will be selecting the “GDB Hardware Debugging” option. Right click and choose New Configuration.
3. In the Main tab, choose a config name, in C/C++ Application choose the elf file built for RAM memory or Flash memory. The snapshot below uses pdl_main.elf for debugging. Disable auto build.



4. In the debugger tab, “GDB Command:” Point to ARM GDB toolchain which was installed by the IDE in the .espressif/tools folder. Check “Use Remote target”, TCP and Port: localhost:3333.



5. Leave the Startup, Source and Common in their default settings and click on the Debug button.
6. This will connect to the GDB server and switch to debug window; code gets downloaded to RAM or Flash memory and halts in Reset handler.
7. Navigate to project window and click on any file in pdl_main application where you want to set breakpoints and click on the run button; it should run and halt at the set breakpoint.
8. You can now step into, step through code, add breakpoints, set watch variables and open memory windows, and continue debugging in RAM or Flash. Use the run button and do a free run to interact with the console terminal menu options printed by the pdl_main application.

Debug and Run in PSRAM:

TBD

Chapter 2

API Reference

2.1 Networking APIs

2.1.1 Wi-Fi

Introduction

The Wi-Fi libraries provide support for configuring and monitoring the Neon.B0 Wi-Fi networking functionality. This includes configuration for:

- Station mode (aka STA mode or Wi-Fi client mode). Neon.B0 connects to an access point.
- Various security modes for the above (WPA, WPA2, WPA3, etc.)
- Scanning for access points (active & passive scanning).
- Promiscuous mode for monitoring of IEEE802.11 Wi-Fi packets.

Application Examples

Several application examples demonstrate the functionality of Wi-Fi library are provided in **wifi** directory of GTS-IDF repository. Please check the README for more details.

API Reference

Wi-Fi Macros & Structures

Defines

WIFI_AP_DEFAULT_MAX_IDLE_PERIOD

Default timeout for SoftAP BSS Max Idle. Unit: 1000TUs >

WIFI_ACTIVE_SCAN_MIN_DEFAULT_TIME

Default minimum active scan time per channel

WIFI_ACTIVE_SCAN_MAX_DEFAULT_TIME

Default maximum active scan time per channel

WIFI_PASSIVE_SCAN_DEFAULT_TIME

Default passive scan time per channel

WIFI_SCAN_HOME_CHANNEL_DWELL_DEFAULT_TIME

Default time spent at home channel between scanning consecutive channels

WIFI_SCAN_PARAMS_DEFAULT_CONFIG ()**CHANNEL_TO_BIT_NUMBER (channel)****BIT_NUMBER_TO_CHANNEL (bit_number, band)****CHANNEL_TO_BIT (channel)****WIFI_PROTOCOL_11B**

802.11b protocol

WIFI_PROTOCOL_11G

802.11g protocol

WIFI_PROTOCOL_11N

802.11n protocol

WIFI_PROTOCOL_LR

Low Rate protocol

WIFI_PROTOCOL_11A

802.11a protocol

WIFI_PROTOCOL_11AC

802.11ac protocol

WIFI_PROTOCOL_11AX

802.11ax protocol

SAE_H2E_IDENTIFIER_LEN

Length of the password identifier for H2E

WIFI_VENDOR_IE_ELEMENT_ID

Vendor Information Element ID

WIFI_PROMIS_FILTER_MASK_ALL

Filter all packets

WIFI_PROMIS_FILTER_MASK_MGMT

Filter the packets with type of WIFI_PKT_MGMT

WIFI_PROMIS_FILTER_MASK_CTRL

Filter the packets with type of WIFI_PKT_CTRL

WIFI_PROMIS_FILTER_MASK_DATA

Filter the packets with type of WIFI_PKT_DATA

WIFI_PROMIS_FILTER_MASK_MISC

Filter the packets with type of WIFI_PKT_MISC

WIFI_PROMIS_FILTER_MASK_DATA_MPDU

Filter the MPDU which is a kind of WIFI_PKT_DATA

WIFI_PROMIS_FILTER_MASK_DATA_AMPDU

Filter the AMPDU which is a kind of WIFI_PKT_DATA

WIFI_PROMIS_FILTER_MASK_FCSFAIL

Filter the FCS failed packets, do not open it in general

WIFI_PROMIS_CTRL_FILTER_MASK_ALL

Filter all control packets

WIFI_PROMIS_CTRL_FILTER_MASK_WRAPPER

Filter the control packets with subtype of Control Wrapper

WIFI_PROMIS_CTRL_FILTER_MASK_BAR

Filter the control packets with subtype of Block Ack Request

WIFI_PROMIS_CTRL_FILTER_MASK_BA

Filter the control packets with subtype of Block Ack

WIFI_PROMIS_CTRL_FILTER_MASK_PSPOLL

Filter the control packets with subtype of PS-Poll

WIFI_PROMIS_CTRL_FILTER_MASK_RTS

Filter the control packets with subtype of RTS

WIFI_PROMIS_CTRL_FILTER_MASK_CTS

Filter the control packets with subtype of CTS

WIFI_PROMIS_CTRL_FILTER_MASK_ACK

Filter the control packets with subtype of ACK

WIFI_PROMIS_CTRL_FILTER_MASK_CFEND

Filter the control packets with subtype of CF-END

WIFI_PROMIS_CTRL_FILTER_MASK_CFENDACK

Filter the control packets with subtype of CF-END+CF-ACK

WIFI_EVENT_MASK_ALL

Mask all Wi-Fi events

WIFI_EVENT_MASK_NONE

Mask none of the Wi-Fi events

WIFI_EVENT_MASK_AP_PROBEREQRECVED

Mask SYSTEM_EVENT_AP_PROBEREQRECVED event

INPH_WIFI_NAN_MAX_SVC_SUPPORTED

Maximum number of NAN services supported

INPH_WIFI_NAN_DATAPATH_MAX_PEERS

Maximum number of NAN datapath peers supported

INPH_WIFI_NDP_ROLE_INITIATOR

Initiator role for NAN Data Path

INPH_WIFI_NDP_ROLE_RESPONDER

Responder role for NAN Data Path

INPH_WIFI_MAX_SVC_NAME_LEN

Maximum length of NAN service name

INPH_WIFI_MAX_FILTER_LEN

Maximum length of NAN service filter

INPH_WIFI_MAX_SVC_INFO_LEN

Maximum length of NAN service info

INPH_WIFI_MAX_FUP_SSI_LEN

Maximum length of NAN Service Specific Info in a Follow-up frame

INPH_WIFI_MAX_SVC_SSI_LEN

Maximum length of NAN Service Specific Info in Publish/Subscribe SDF's

INPH_WIFI_MAX_NEIGHBOR_REP_LEN

Maximum length of NAN Neighbor Report

WIFI_OUI_LEN

Length of OUI bytes in IE or attributes

MAX_SSID_LEN

Maximum length of SSID

MAX_PASSPHRASE_LEN

Maximum length of passphrase

MAX_WPS_AP_CRED

Maximum number of AP credentials received from WPS handshake

WIFI_STATIS_BUFFER

Buffer status

WIFI_STATIS_RXTX

RX/TX status

WIFI_STATIS_HW

Hardware status

WIFI_STATIS_DIAG

Diagnostic status

WIFI_STATIS_PS

Power save status

WIFI_STATIS_ALL

All status

WIFI_MAX_SUPPORT_COUNTRY_NUM

max number of supported countries

WIFI_MAX_REGULATORY_RULE_NUM

max number of regulatory rules

Typedefs

typedef struct *wifi_csi_info_t* **wifi_csi_info_t**

CSI data type.

typedef int (***wifi_action_rx_cb_t**)(uint8_t *hdr, uint8_t *payload, size_t len, uint8_t channel)

The Rx callback function of Action Tx operations.

Param hdr

Pointer to the IEEE 802.11 Header structure

Param payload

Pointer to the Payload following 802.11 Header

Param len

Length of the Payload

Param channel

Channel number the frame is received on

```
typedef void (*wifi_action_roc_done_cb_t)(uint32_t context, uint8_t op_id, wifi_roc_done_status_t status)
```

The callback function executed when ROC operation has ended.

Param context

rxcb registered for the corresponding ROC operation

Param op_id

ID of the corresponding ROC operation

Param status

status code of the ROC operation denoted

```
typedef wifi_tx_info_t inph_80211_tx_info_t
```

Enums

```
enum inph_wifi_mode_t
```

Wi-Fi mode type.

Values:

```
enumerator WIFI_MODE_NULL
```

Null mode

```
enumerator WIFI_MODE_STA
```

Wi-Fi station mode

```
enumerator WIFI_MODE_AP
```

Wi-Fi soft-AP mode

```
enumerator WIFI_MODE_APSTA
```

Wi-Fi station + soft-AP mode

```
enumerator WIFI_MODE_NAN
```

Wi-Fi NAN mode

```
enumerator WIFI_MODE_MAX
```

```
enum wifi_action_tx_t
```

Values:

```
enumerator WIFI_OFFCHAN_TX_CANCEL
```

Cancel off-channel transmission

enumerator **WIFI_OFFCHAN_TX_REQ**

Request off-channel transmission

enum **wifi_roc_t**

Values:

enumerator **WIFI_ROC_CANCEL**

Cancel remain on channel

enumerator **WIFI_ROC_REQ**

Request remain on channel

enum **wifi_pm_flags**

Power management flags for WiFi

Values:

enumerator **WIFI_PM_PS_POLL**

Send ps poll if a beacon was missed. This will increase power consumption slightly but decrease latency.

enumerator **WIFI_PM_DYN_LISTEN_INT**

Dynamic listen interval. Listen to each beacon if there has been traffic recently

enumerator **WIFI_PM_STA_RX_NAP**

Turn off receiver for uninteresting frames for station

enumerator **WIFI_PM_STA_ONLY_BROADCAST**

Don't receive multicast frames that are not broadcast (only effective if rx_nap is used).

enumerator **WIFI_PM_TX_PS**

Send outgoing frames without leaving WiFi power save

enumerator **WIFI_PM_MCAST_DONT_CARE**

Ignore the multicast flag in beacons. Use this function with care. Incoming broadcast ARPs or other important broadcast/multicast traffic may be missed.

enum **system_notify_msg_code**

Wi-Fi disconnection system error codes.

These error codes are used to indicate the cause of disconnection by self.

Values:

enumerator **SYSTEM_NOTIFY_MSG_AP_NOT_FOUND**

WiFi AP not found

enumerator **SYSTEM_NOTIFY_MSG_PASSPHRASE_ERR**

WiFi wrong passphrase or 4way handshake timeout

enum **wifi_err_reason_t**

Wi-Fi disconnection reason codes.

These reason codes are used to indicate the cause of disconnection.

Values:

enumerator **WIFI_REASON_UNSPECIFIED**

Unspecified reason

enumerator **WIFI_REASON_AUTH_EXPIRE**

Authentication expired

enumerator **WIFI_REASON_AUTH_LEAVE**

Deauthentication due to leaving

enumerator **WIFI_REASON_ASSOC_EXPIRE**

Deprecated, will be removed in next IDF major release

enumerator **WIFI_REASON_DISASSOC_DUE_TO_INACTIVITY**

Disassociated due to inactivity

enumerator **WIFI_REASON_ASSOC_TOOMANY**

Too many associated stations

enumerator **WIFI_REASON_NOT_AUTHED**

Deprecated, will be removed in next IDF major release

enumerator **WIFI_REASON_CLASS2_FRAME_FROM_NONAUTH_STA**

Class 2 frame received from nonauthenticated STA

enumerator **WIFI_REASON_NOT_ASSOCED**

Deprecated, will be removed in next IDF major release

enumerator **WIFI_REASON_CLASS3_FRAME_FROM_NONASSOC_STA**

Class 3 frame received from nonassociated STA

enumerator **WIFI_REASON_ASSOC_LEAVE**

Deassociated due to leaving

enumerator **WIFI_REASON_ASSOC_NOT_AUTHED**

Association but not authenticated

enumerator **WIFI_REASON_DISASSOC_PWRCAP_BAD**

Disassociated due to poor power capability

enumerator **WIFI_REASON_DISASSOC_SUPCHAN_BAD**

Disassociated due to unsupported channel

enumerator **WIFI_REASON_BSS_TRANSITION_DISASSOC**

Disassociated due to BSS transition

enumerator **WIFI_REASON_IE_INVALID**

Invalid Information Element (IE)

enumerator **WIFI_REASON_MIC_FAILURE**

MIC failure

enumerator **WIFI_REASON_4WAY_HANDSHAKE_TIMEOUT**

4-way handshake timeout

enumerator **WIFI_REASON_GROUP_KEY_UPDATE_TIMEOUT**

Group key update timeout

enumerator **WIFI_REASON_IE_IN_4WAY_DIFFERS**

IE differs in 4-way handshake

enumerator **WIFI_REASON_GROUP_CIPHER_INVALID**

Invalid group cipher

enumerator **WIFI_REASON_PAIRWISE_CIPHER_INVALID**

Invalid pairwise cipher

enumerator **WIFI_REASON_AKMP_INVALID**

Invalid AKMP

enumerator **WIFI_REASON_UNSUPP_RSN_IE_VERSION**

Unsupported RSN IE version

enumerator **WIFI_REASON_INVALID_RSN_IE_CAP**

Invalid RSN IE capabilities

enumerator **WIFI_REASON_802_1X_AUTH_FAILED**

802.1X authentication failed

enumerator **WIFI_REASON_CIPHER_SUITE_REJECTED**

Cipher suite rejected

enumerator **WIFI_REASON_TDLS_PEER_UNREACHABLE**

TDLS peer unreachable

enumerator **WIFI_REASON_TDLS_UNSPECIFIED**

TDLS unspecified

enumerator **WIFI_REASON_SSP_REQUESTED_DISASSOC**

SSP requested disassociation

enumerator **WIFI_REASON_NO_SSP_ROAMING_AGREEMENT**

No SSP roaming agreement

enumerator **WIFI_REASON_BAD_CIPHER_OR_AKM**

Bad cipher or AKM

enumerator **WIFI_REASON_NOT_AUTHORIZED_THIS_LOCATION**

Not authorized in this location

enumerator **WIFI_REASON_SERVICE_CHANGE_PERCLUDES_TS**

Service change precludes TS

enumerator **WIFI_REASON_UNSPECIFIED_QOS**

Unspecified QoS reason

enumerator **WIFI_REASON_NOT_ENOUGH_BANDWIDTH**

Not enough bandwidth

enumerator **WIFI_REASON_MISSING_ACKS**

Missing ACKs

enumerator **WIFI_REASON_EXCEEDED_TXOP**

Exceeded TXOP

enumerator **WIFI_REASON_STA_LEAVING**

Station leaving

enumerator **WIFI_REASON_END_BA**

End of Block Ack (BA)

enumerator **WIFI_REASON_UNKNOWN_BA**

Unknown Block Ack (BA)

enumerator **WIFI_REASON_TIMEOUT**

Timeout

enumerator **WIFI_REASON_PEER_INITIATED**

Peer initiated disassociation

enumerator **WIFI_REASON_AP_INITIATED**

AP initiated disassociation

enumerator **WIFI_REASON_INVALID_FT_ACTION_FRAME_COUNT**

Invalid FT action frame count

enumerator **WIFI_REASON_INVALID_PMKID**

Invalid PMKID

enumerator **WIFI_REASON_INVALID_MDE**

Invalid MDE

enumerator **WIFI_REASON_INVALID_FTE**

Invalid FTE

enumerator **WIFI_REASON_TRANSMISSION_LINK_ESTABLISH_FAILED**

Transmission link establishment failed

enumerator **WIFI_REASON_ALTERATIVE_CHANNEL_OCCUPIED**

Alternative channel occupied

enumerator **WIFI_REASON_BEACON_TIMEOUT**

Beacon timeout

enumerator **WIFI_REASON_NO_AP_FOUND**

No AP found

enumerator **WIFI_REASON_AUTH_FAIL**

Authentication failed

enumerator **WIFI_REASON_ASSOC_FAIL**

Association failed

enumerator **WIFI_REASON_HANDSHAKE_TIMEOUT**

Handshake timeout

enumerator **WIFI_REASON_CONNECTION_FAIL**

Connection failed

enumerator **WIFI_REASON_AP_TSF_RESET**

AP TSF reset

enumerator **WIFI_REASON_ROAMING**

Roaming

enumerator **WIFI_REASON_ASSOC_COMEBACK_TIME_TOO_LONG**

Association comeback time too long

enumerator **WIFI_REASON_SA_QUERY_TIMEOUT**

SA query timeout

enumerator **WIFI_REASON_NO_AP_FOUND_W_COMPATIBLE_SECURITY**

No AP found with compatible security

enumerator **WIFI_REASON_NO_AP_FOUND_IN_AUTHMODE_THRESHOLD**

No AP found in auth mode threshold

enumerator **WIFI_REASON_NO_AP_FOUND_IN_RSSI_THRESHOLD**

No AP found in RSSI threshold

enum **wifi_bandwidth_t**

Wi-Fi bandwidth type.

Values:

enumerator **WIFI_BW_HT20**

Bandwidth is HT20

enumerator **WIFI_BW20**

Bandwidth is 20 MHz

enumerator **WIFI_BW_HT40**

Bandwidth is HT40

enumerator **WIFI_BW40**

Bandwidth is 40 MHz

enumerator **WIFI_BW80**

Bandwidth is 80 MHz

enumerator **WIFI_BW160**

Bandwidth is 160 MHz

enumerator **WIFI_BW80_BW80**

Bandwidth is 80 + 80 MHz

enum **wifi_scan_method_t**

Wi-Fi scan method.

Values:

enumerator **WIFI_FAST_SCAN**

Do fast scan, scan will end after find SSID match AP

enumerator **WIFI_ALL_CHANNEL_SCAN**

All channel scan, scan will end after scan all the channel

enum **wifi_sort_method_t**

Wi-Fi sort AP method.

Values:

enumerator **WIFI_CONNECT_AP_BY_SIGNAL**

Sort match AP in scan list by RSSI

enumerator **WIFI_CONNECT_AP_BY_SECURITY**

Sort match AP in scan list by security mode

enum **wifi_ps_type_t**

Wi-Fi power save type.

Values:

enumerator **WIFI_PS_NONE**

No power save

enumerator **WIFI_PS_MIN_MODEM**

Minimum modem power saving. In this mode, station wakes up to receive beacon every DTIM period

enumerator **WIFI_PS_MAX_MODEM**

Maximum modem power saving. In this mode, interval to receive beacons is determined by the listen_interval parameter in wifi_sta_config_t

enum **wifi_band_t**

Argument structure for Wi-Fi band.

Values:

enumerator **WIFI_BAND_2G**

Band is 2.4 GHz

enumerator **WIFI_BAND_5G**

Band is 5 GHz

enum **wifi_band_mode_t**

Argument structure for Wi-Fi band mode.

Values:

enumerator **WIFI_BAND_MODE_2G_ONLY**

Wi-Fi band mode is 2.4 GHz only

enumerator **WIFI_BAND_MODE_5G_ONLY**

Wi-Fi band mode is 5 GHz only

enumerator **WIFI_BAND_MODE_AUTO**

Wi-Fi band mode is 2.4 GHz + 5 GHz

enum **wifi_2g_channel_bit_t**

Argument structure for 2.4G channels

Values:

enumerator **WIFI_CHANNEL_1**

Wi-Fi channel 1

enumerator **WIFI_CHANNEL_2**

Wi-Fi channel 2

enumerator **WIFI_CHANNEL_3**

Wi-Fi channel 3

enumerator **WIFI_CHANNEL_4**

Wi-Fi channel 4

enumerator **WIFI_CHANNEL_5**

Wi-Fi channel 5

enumerator **WIFI_CHANNEL_6**

Wi-Fi channel 6

enumerator **WIFI_CHANNEL_7**

Wi-Fi channel 7

enumerator **WIFI_CHANNEL_8**

Wi-Fi channel 8

enumerator **WIFI_CHANNEL_9**

Wi-Fi channel 9

enumerator **WIFI_CHANNEL_10**

Wi-Fi channel 10

enumerator **WIFI_CHANNEL_11**

Wi-Fi channel 11

enumerator **WIFI_CHANNEL_12**

Wi-Fi channel 12

enumerator **WIFI_CHANNEL_13**

Wi-Fi channel 13

enumerator **WIFI_CHANNEL_14**

Wi-Fi channel 14

enum **wifi_5g_channel_bit_t**

Argument structure for 5G channels

Values:

enumerator **WIFI_CHANNEL_36**

Wi-Fi channel 36

enumerator **WIFI_CHANNEL_40**

Wi-Fi channel 40

enumerator **WIFI_CHANNEL_44**

Wi-Fi channel 44

enumerator **WIFI_CHANNEL_48**

Wi-Fi channel 48

enumerator **WIFI_CHANNEL_52**

Wi-Fi channel 52

enumerator **WIFI_CHANNEL_56**

Wi-Fi channel 56

enumerator **WIFI_CHANNEL_60**

Wi-Fi channel 60

enumerator **WIFI_CHANNEL_64**

Wi-Fi channel 64

enumerator **WIFI_CHANNEL_100**

Wi-Fi channel 100

enumerator **WIFI_CHANNEL_104**

Wi-Fi channel 104

enumerator **WIFI_CHANNEL_108**

Wi-Fi channel 108

enumerator **WIFI_CHANNEL_112**

Wi-Fi channel 112

enumerator **WIFI_CHANNEL_116**

Wi-Fi channel 116

enumerator **WIFI_CHANNEL_120**

Wi-Fi channel 120

enumerator **WIFI_CHANNEL_124**

Wi-Fi channel 124

enumerator **WIFI_CHANNEL_128**

Wi-Fi channel 128

enumerator **WIFI_CHANNEL_132**

Wi-Fi channel 132

enumerator **WIFI_CHANNEL_136**

Wi-Fi channel 136

enumerator **WIFI_CHANNEL_140**

Wi-Fi channel 140

enumerator **WIFI_CHANNEL_144**

Wi-Fi channel 144

enumerator **WIFI_CHANNEL_149**

Wi-Fi channel 149

enumerator **WIFI_CHANNEL_153**

Wi-Fi channel 153

enumerator **WIFI_CHANNEL_157**

Wi-Fi channel 157

enumerator **WIFI_CHANNEL_161**

Wi-Fi channel 161

enumerator **WIFI_CHANNEL_165**

Wi-Fi channel 165

enumerator **WIFI_CHANNEL_169**

Wi-Fi channel 169

enumerator **WIFI_CHANNEL_173**

Wi-Fi channel 173

enumerator **WIFI_CHANNEL_177**

Wi-Fi channel 177

enum **wifi_sae_pwe_method_t**

Configuration for SAE PWE derivation.

Values:

enumerator **WPA3_SAE_PWE_UNSPECIFIED**

enumerator **WPA3_SAE_PWE_HUNT_AND_PECK**

enumerator **WPA3_SAE_PWE_HASH_TO_ELEMENT**

enumerator **WPA3_SAE_PWE_BOTH**

enum **wifi_sae_pk_mode_t**

Configuration for SAE-PK.

Values:

enumerator **WPA3_SAE_PK_MODE_AUTOMATIC**

enumerator **WPA3_SAE_PK_MODE_ONLY**

enumerator **WPA3_SAE_PK_MODE_DISABLED**

enum **wifi_storage_t**

Wi-Fi storage type.

Values:

enumerator **WIFI_STORAGE_FLASH**

All configuration will store in both memory and flash

enumerator **WIFI_STORAGE_RAM**

All configuration will only store in the memory

enum **wifi_vendor_ie_type_t**

Vendor Information Element type.

Determines the frame type that the IE will be associated with.

Values:

enumerator **WIFI_VND_IE_TYPE_BEACON**

Beacon frame

enumerator **WIFI_VND_IE_TYPE_PROBE_REQ**

Probe request frame

enumerator **WIFI_VND_IE_TYPE_PROBE_RESP**

Probe response frame

enumerator **WIFI_VND_IE_TYPE_ASSOC_REQ**

Association request frame

enumerator **WIFI_VND_IE_TYPE_ASSOC_RESP**

Association response frame

enum **wifi_vendor_ie_id_t**

Vendor Information Element index.

Each IE type can have up to two associated vendor ID elements.

Values:

enumerator **WIFI_VND_IE_ID_0**

Vendor ID element 0

enumerator **WIFI_VND_IE_ID_1**

Vendor ID element 1

enum **wifi_phy_mode_t**

Operation PHY mode.

Values:

enumerator **WIFI_PHY_MODE_LR**

PHY mode for Low Rate

enumerator **WIFI_PHY_MODE_11B**

PHY mode for 11b

enumerator **WIFI_PHY_MODE_11G**

PHY mode for 11g

enumerator **WIFI_PHY_MODE_11A**

PHY mode for 11a

enumerator **WIFI_PHY_MODE_HT20**

PHY mode for Bandwidth HT20

enumerator **WIFI_PHY_MODE_HT40**

PHY mode for Bandwidth HT40

enumerator **WIFI_PHY_MODE_HE20**

PHY mode for Bandwidth HE20

enumerator **WIFI_PHY_MODE_VHT20**

PHY mode for Bandwidth VHT20

enum **wifi_promiscuous_pkt_type_t**

Promiscuous frame type.

Passed to promiscuous mode RX callback to indicate the type of parameter in the buffer.

Values:

enumerator **WIFI_PKT_MGMT**

Management frame, indicates ‘buf’ argument is wifi_promiscuous_pkt_t

enumerator **WIFI_PKT_CTRL**

Control frame, indicates ‘buf’ argument is wifi_promiscuous_pkt_t

enumerator **WIFI_PKT_DATA**

Data frame, indicates ‘buf’ argument is wifi_promiscuous_pkt_t

enumerator **WIFI_PKT_MISC**

Other type, such as MIMO etc. ‘buf’ argument is wifi_promiscuous_pkt_t but the payload is zero length.

enum **wifi_ant_mode_t**

Wi-Fi antenna mode.

Values:

enumerator **WIFI_ANT_MODE_ANT0**

Enable Wi-Fi antenna 0 only

enumerator **WIFI_ANT_MODE_ANT1**

Enable Wi-Fi antenna 1 only

enumerator **WIFI_ANT_MODE_AUTO**

Enable Wi-Fi antenna 0 and 1, automatically select an antenna

enumerator **WIFI_ANT_MODE_MAX**

Invalid Wi-Fi enabled antenna

enum **wifi_roc_done_status_t**

Status codes for WIFI_EVENT_ROC_DONE evt

Values:

enumerator **WIFI_ROC_DONE**

ROC operation was completed successfully

enumerator **WIFI_ROC_FAIL**

ROC operation was cancelled

enum **wifi_nan_svc_proto_t**

Protocol types in NAN service specific info attribute.

Values:

enumerator **WIFI_SVC_PROTO_RESERVED**

Value 0 Reserved

enumerator **WIFI_SVC_PROTO_BONJOUR**

Bonjour Protocol

enumerator **WIFI_SVC_PROTO_GENERIC**

Generic Service Protocol

enumerator **WIFI_SVC_PROTO_CSA_MATTER**

CSA Matter specific protocol

enumerator **WIFI_SVC_PROTO_MAX**

Values 4-255 Reserved

enum **wifi_nan_service_type_t**

NAN Services types.

Values:

enumerator **NAN_PUBLISH_SOLICITED**

Send unicast Publish frame to Subscribers that match the requirement

enumerator **NAN_PUBLISH_UN SOLICITED**

Send broadcast Publish frames in every Discovery Window(DW)

enumerator **NAN_SUBSCRIBE_ACTIVE**

Send broadcast Subscribe frames in every DW

enumerator **NAN_SUBSCRIBE_PASSIVE**

Passively listens to Publish frames

enum **wifi_phy_rate_t**

Wi-Fi PHY rate encodings.

Note: Rate Table: MCS Rate and Guard Interval Information

MCS RATE	HT20	HT40	HE20	VHT20
WIFI_PHY_RATE_MCS0	6.5 Mbps (800 ns)	13.5 Mbps (800 ns)	8.1 Mbps (1600 ns)	6.5 Mbps (800 ns)
WIFI_PHY_RATE_MCS1	13 Mbps (800 ns)	27 Mbps (800 ns)	16.3 Mbps (1600 ns)	13 Mbps (800 ns)
WIFI_PHY_RATE_MCS2	19.5 Mbps (800 ns)	40.5 Mbps (800 ns)	24.4 Mbps (1600 ns)	19.5 Mbps (800 ns)
WIFI_PHY_RATE_MCS3	26 Mbps (800 ns)	54 Mbps (800 ns)	32.5 Mbps (1600 ns)	26 Mbps (800 ns)
WIFI_PHY_RATE_MCS4	39 Mbps (800 ns)	81 Mbps (800 ns)	48.8 Mbps (1600 ns)	39 Mbps (800 ns)
WIFI_PHY_RATE_MCS5	52 Mbps (800 ns)	108 Mbps (800 ns)	65 Mbps (1600 ns)	52 Mbps (800 ns)
WIFI_PHY_RATE_MCS6	58.5 Mbps (800 ns)	121.5 Mbps (800 ns)	73.1 Mbps (1600 ns)	58.5 Mbps (800 ns)
WIFI_PHY_RATE_MCS7	65 Mbps (800 ns)	135 Mbps (800 ns)	81.3 Mbps (1600 ns)	65 Mbps (800 ns)
WIFI_PHY_RATE_MCS8			97.5 Mbps (1600 ns)	
WIFI_PHY_RATE_MCS9			108.3 Mbps (1600 ns)	

Note:

MCS RATE	HT20	HT40	HE20	VHT20
WIFI_PHY_RATE_MCS0	7.2 Mbps (400 ns)	15 Mbps (400 ns)	8.6 Mbps (800 ns)	7.2 Mbps (400 ns)
WIFI_PHY_RATE_MCS1	14.4 Mbps (400 ns)	30 Mbps (400 ns)	17.2 Mbps (800 ns)	14.4 Mbps (400 ns)
WIFI_PHY_RATE_MCS2	21.7 Mbps (400 ns)	45 Mbps (400 ns)	25.8 Mbps (800 ns)	21.7 Mbps (400 ns)
WIFI_PHY_RATE_MCS3	28.9 Mbps (400 ns)	60 Mbps (400 ns)	34.4 Mbps (800 ns)	28.9 Mbps (400 ns)
WIFI_PHY_RATE_MCS4	43.3 Mbps (400 ns)	90 Mbps (400 ns)	51.6 Mbps (800 ns)	43.3 Mbps (400 ns)
WIFI_PHY_RATE_MCS5	57.8 Mbps (400 ns)	120 Mbps (400 ns)	68.8 Mbps (800 ns)	57.8 Mbps (400 ns)
WIFI_PHY_RATE_MCS6	65 Mbps (400 ns)	135 Mbps (400 ns)	77.4 Mbps (800 ns)	65 Mbps (400 ns)
WIFI_PHY_RATE_MCS7	72.2 Mbps (400 ns)	150 Mbps (400 ns)	86 Mbps (800 ns)	72.2 Mbps (400 ns)
WIFI_PHY_RATE_MCS8			103.2 Mbps (800 ns)	
WIFI_PHY_RATE_MCS9			114.7 Mbps (800 ns)	

Values:

enumerator **WIFI_PHY_RATE_1M_L**

1 Mbps with long preamble

enumerator **WIFI_PHY_RATE_2M_L**

2 Mbps with long preamble

enumerator **WIFI_PHY_RATE_5M_L**

5.5 Mbps with long preamble

enumerator **WIFI_PHY_RATE_11M_L**

11 Mbps with long preamble

enumerator **WIFI_PHY_RATE_2M_S**

2 Mbps with short preamble

enumerator **WIFI_PHY_RATE_5M_S**

5.5 Mbps with short preamble

enumerator **WIFI_PHY_RATE_11M_S**

11 Mbps with short preamble

enumerator **WIFI_PHY_RATE_48M**

48 Mbps

enumerator **WIFI_PHY_RATE_24M**

24 Mbps

enumerator **WIFI_PHY_RATE_12M**

12 Mbps

enumerator **WIFI_PHY_RATE_6M**

6 Mbps

enumerator **WIFI_PHY_RATE_54M**

54 Mbps

enumerator **WIFI_PHY_RATE_36M**

36 Mbps

enumerator **WIFI_PHY_RATE_18M**

18 Mbps

enumerator **WIFI_PHY_RATE_9M**

9 Mbps

enumerator **WIFI_PHY_RATE_MCS0_LGI**
MCS0 with long GI

enumerator **WIFI_PHY_RATE_MCS1_LGI**
MCS1 with long GI

enumerator **WIFI_PHY_RATE_MCS2_LGI**
MCS2 with long GI

enumerator **WIFI_PHY_RATE_MCS3_LGI**
MCS3 with long GI

enumerator **WIFI_PHY_RATE_MCS4_LGI**
MCS4 with long GI

enumerator **WIFI_PHY_RATE_MCS5_LGI**
MCS5 with long GI

enumerator **WIFI_PHY_RATE_MCS6_LGI**
MCS6 with long GI

enumerator **WIFI_PHY_RATE_MCS7_LGI**
MCS7 with long GI

enumerator **WIFI_PHY_RATE_MCS8_LGI**
MCS8 with long GI

enumerator **WIFI_PHY_RATE_MCS9_LGI**
MCS9 with long GI

enumerator **WIFI_PHY_RATE_MCS0_SGI**
MCS0 with short GI

enumerator **WIFI_PHY_RATE_MCS1_SGI**
MCS1 with short GI

enumerator **WIFI_PHY_RATE_MCS2_SGI**
MCS2 with short GI

enumerator **WIFI_PHY_RATE_MCS3_SGI**
MCS3 with short GI

enumerator **WIFI_PHY_RATE_MCS4_SGI**
MCS4 with short GI

enumerator **WIFI_PHY_RATE_MCS5_SGI**

MCS5 with short GI

enumerator **WIFI_PHY_RATE_MCS6_SGI**

MCS6 with short GI

enumerator **WIFI_PHY_RATE_MCS7_SGI**

MCS7 with short GI

enumerator **WIFI_PHY_RATE_MCS8_SGI**

MCS8 with short GI

enumerator **WIFI_PHY_RATE_MCS9_SGI**

MCS9 with short GI

enumerator **WIFI_PHY_RATE_LORA_250K**

Espressif-specific Long Range mode rate, 250 Kbps

enumerator **WIFI_PHY_RATE_LORA_500K**

Espressif-specific Long Range mode rate, 500 Kbps

enumerator **WIFI_PHY_RATE_MAX**

enum **wifi_event_sta_wps_fail_reason_t**

Argument structure for WIFI_EVENT_STA_WPS_ER_FAILED event.

Values:

enumerator **WPS_FAIL_REASON_NORMAL**

WPS normal fail reason

enumerator **WPS_FAIL_REASON_RECV_M2D**

WPS receive M2D frame

enumerator **WPS_FAIL_REASON_RECV_DEAUTH**

Recv deauth from AP while wps handshake

enumerator **WPS_FAIL_REASON_MAX**

Max WPS fail reason

enum **wifi_ftm_status_t**

FTM operation status types.

Values:

enumerator **FTM_STATUS_SUCCESS**

FTM exchange is successful

enumerator **FTM_STATUS_UNSUPPORTED**

Peer does not support FTM

enumerator **FTM_STATUS_CONF_REJECTED**

Peer rejected FTM configuration in FTM Request

enumerator **FTM_STATUS_NO_RESPONSE**

Peer did not respond to FTM Requests

enumerator **FTM_STATUS_FAIL**

Unknown error during FTM exchange

enumerator **FTM_STATUS_NO_VALID_MSMT**

FTM session did not result in any valid measurements

enumerator **FTM_STATUS_USER_TERM**

User triggered termination

enum **wifi_action_tx_status_type_t**

Status codes for WIFI_EVENT_ACTION_TX_STATUS evt There will be back to back events in success case TX_DONE and TX_DURATION_COMPLETED

Values:

enumerator **WIFI_ACTION_TX_DONE**

ACTION_TX operation was completed successfully

enumerator **WIFI_ACTION_TX_FAILED**

ACTION_TX operation failed during tx

enumerator **WIFI_ACTION_TX_DURATION_COMPLETED**

ACTION_TX operation completed it's wait duration

enumerator **WIFI_ACTION_TX_OP_CANCELLED**

ACTION_TX operation was cancelled by application or higher priority operation

enum **wps_fail_reason_t**

WPS fail reason.

Values:

enumerator **WPS_AP_FAIL_REASON_NORMAL**

WPS normal fail reason

enumerator **WPS_AP_FAIL_REASON_CONFIG**

WPS failed due to incorrect config

enumerator **WPS_AP_FAIL_REASON_AUTH**

WPS failed during auth

enumerator **WPS_AP_FAIL_REASON_MAX**

Max WPS fail reason

enum **wifi_tx_status_t**

Status of wifi sending data.

Values:

enumerator **WIFI_SEND_SUCCESS**

Sending Wi-Fi data successfully

enumerator **WIFI_SEND_FAIL**

Sending Wi-Fi data fail

struct **wifi_scan_default_params_t**

#include <inph_wifi_types_generic.h> Parameters default scan configurations.

Public Members

wifi_scan_time_t **scan_time**

Scan time per channel

uint8_t **home_chan_dwell_time**

Time spent at home channel between scanning consecutive channels.

struct **wifi_scan_threshold_t**

#include <inph_wifi_types_generic.h> Structure describing parameters for a Wi-Fi fast scan.

Public Members

int8_t **rss_i**

The minimum rssi to accept in the fast scan mode. Defaults to -127 if set to ≥ 0

wifi_auth_mode_t **authmode**

The weakest auth mode to accept in the fast scan mode Note: In case this value is not set and password is set as per WPA2 standards(password len ≥ 8), it will be defaulted to WPA2 and device won't connect to deprecated WEP/WPA networks. Please set auth mode threshold as WIFI_AUTH_WEP/WIFI_AUTH_WPA_PSK to connect to WEP/WPA networks

uint8_t **rss_i_5g_adjustment**

The RSSI value of the 5G AP is within the `rss_i_5g_adjustment` range compared to the 2G AP, the 5G AP will be given priority for connection.

struct **wifi_protocols_t**

#include <inph_wifi_types_generic.h> Description of a Wi-Fi protocols.

Public Members

uint16_t **ghz_2g**

Represents 2.4 GHz protocol, support 802.11b or 802.11g or 802.11n or 802.11ax or LR mode

uint16_t **ghz_5g**

Represents 5 GHz protocol, support 802.11a or 802.11n or 802.11ac or 802.11ax

struct **wifi_bandwidths_t**

#include <inph_wifi_types_generic.h> Description of a Wi-Fi band bandwidths.

Public Members

wifi_bandwidth_t **ghz_2g**

Represents 2.4 GHz bandwidth

wifi_bandwidth_t **ghz_5g**

Represents 5 GHz bandwidth

struct **wifi_pmf_config_t**

#include <inph_wifi_types_generic.h> Configuration structure for Protected Management Frame.

Public Members

bool **capable**

Deprecated variable. Device will always connect in PMF mode if other device also advertises PMF capability.

bool **required**

Advertises that Protected Management Frame is required. Device will not associate to non-PMF capable devices.

struct **wifi_bss_max_idle_config_t**

#include <inph_wifi_types_generic.h> Configuration structure for BSS max idle.

Public Members

uint16_t **period**

Sets BSS Max idle period (1 Unit = 1000TUs OR 1.024 Seconds). If there are no frames for this period from a STA, SoftAP will disassociate due to inactivity. Setting it to 0 disables the feature

bool **protected_keep_alive**

Requires clients to use protected keep alive frames for BSS Max Idle period

struct **wifi_ap_config_t**

#include <inph_wifi_types_generic.h> Soft-AP configuration settings for the device.

Public Members

uint8_t **ssid**[32]

SSID of soft-AP. If **ssid_len** field is 0, this must be a Null terminated string. Otherwise, length is set according to **ssid_len**.

uint8_t **password**[64]

Password of soft-AP.

uint8_t **ssid_len**

Optional length of SSID field.

uint8_t **channel**

Channel of soft-AP

wifi_auth_mode_t **authmode**

Auth mode of soft-AP. Do not support AUTH_WEP, AUTH_WAPI_PSK and AUTH_OWE in soft-AP mode. When the auth mode is set to WPA2_PSK, WPA2_WPA3_PSK or WPA3_PSK, the pairwise cipher will be overwritten with WIFI_CIPHER_TYPE_CCMP.

uint8_t **ssid_hidden**

Broadcast SSID or not, default 0, broadcast the SSID

uint8_t **max_connection**

Max number of stations allowed to connect in

uint16_t **beacon_interval**

Beacon interval which should be multiples of 100. Unit: TU(time unit, 1 TU = 1024 us). Range: 100 ~ 60000. Default value: 100

uint8_t **csa_count**

Channel Switch Announcement Count. Notify the station that the channel will switch after the **csa_count** beacon intervals. Default value: 3

uint8_t **dtim_period**

Dtim period of soft-AP. Range: 1 ~ 10. Default value: 1

wifi_cipher_type_t **pairwise_cipher**

Pairwise cipher of SoftAP, group cipher will be derived using this. Cipher values are valid starting from WIFI_CIPHER_TYPE_TKIP, enum values before that will be considered as invalid and default cipher suites(TKIP+CCMP) will be used. Valid cipher suites in softAP mode are WIFI_CIPHER_TYPE_TKIP, WIFI_CIPHER_TYPE_CCMP and WIFI_CIPHER_TYPE_TKIP_CCMP.

bool **ftm_responder**

Enable FTM Responder mode

wifi_pmf_config_t **pmf_cfg**

Configuration for Protected Management Frame

wifi_sae_pwe_method_t **sae_pwe_h2e**

Configuration for SAE PWE derivation method

uint8_t **transition_disable**

Whether to enable transition disable feature

uint8_t **sae_ext**

Enable SAE EXT feature. SOC_GCMP_SUPPORT is required for this feature.

wifi_bss_max_idle_config_t **bss_max_idle_cfg**

Configuration for bss max idle, effective if CONFIG_WIFI_BSS_MAX_IDLE_SUPPORT is enabled

uint16_t **gtk_rekey_interval**

GTK rekeying interval in seconds. If set to 0, GTK rekeying is disabled. Range: 60 ~ 65535 including 0.

struct **wifi_nan_config_t**

#include <inph_wifi_types_generic.h> NAN Discovery start configuration.

Public Members

uint8_t **op_channel**

NAN Discovery operating channel

uint8_t **master_pref**

Device's preference value to serve as NAN Master

uint8_t **scan_time**

Scan time in seconds while searching for a NAN cluster

uint16_t **warm_up_sec**

Warm up time before assuming NAN Anchor Master role

union **wifi_config_t**

#include <inph_wifi_types_generic.h> Configuration data for device's AP or STA or NAN.

The usage of this union (for ap, sta or nan configuration) is determined by the accompanying interface argument passed to `inph_wifi_set_config()` or `inph_wifi_get_config()`

Public Members

wifi_ap_config_t **ap**

Configuration of AP

wifi_sta_config_t **sta**

Configuration of STA

wifi_nan_config_t **nan**

Configuration of NAN

struct **wifi_sta_info_t**

#include <inph_wifi_types_generic.h> Description of STA associated with AP.

Public Members

uint8_t **mac[6]**

MAC address

int8_t **rss_i**

Current average rssi of sta connected

uint32_t **phy_11b**

Bit: 0 flag to identify if 11b mode is enabled or not

uint32_t **phy_11g**

Bit: 1 flag to identify if 11g mode is enabled or not

uint32_t **phy_11n**

Bit: 2 flag to identify if 11n mode is enabled or not

uint32_t **phy_1r**

Bit: 3 flag to identify if low rate is enabled or not

uint32_t **phy_11a**

Bit: 4 flag to identify if 11ax mode is enabled or not

uint32_t **phy_11ac**

Bit: 5 flag to identify if 11ax mode is enabled or not

uint32_t **phy_11ax**

Bit: 6 flag to identify if 11ax mode is enabled or not

uint32_t **is_mesh_child**

Bit: 7 flag to identify mesh child

uint32_t **reserved**

Bit: 8..31 reserved

struct **vendor_ie_data_t**

#include <inph_wifi_types_generic.h> Vendor Information Element header.

The first bytes of the Information Element will match this header. Payload follows.

Public Members

uint8_t **element_id**

Should be set to WIFI_VENDOR_IE_ELEMENT_ID (0xDD)

uint8_t **length**

Length of all bytes in the element data following this field. Minimum 4.

uint8_t **vendor_oui**[3]

Vendor identifier (OUI).

uint8_t **vendor_oui_type**

Vendor-specific OUI type.

uint8_t **payload**[0]

Payload. Length is equal to value in 'length' field, minus 4.

struct **wifi_promiscuous_filter_t**

#include <inph_wifi_types_generic.h> Mask for filtering different packet types in promiscuous mode.

Public Members

uint32_t **filter_mask**

OR of one or more filter values WIFI_PROMIS_FILTER_*

struct **wifi_ant_gpio_t**

#include <inph_wifi_types_generic.h> Wi-Fi GPIO configuration for antenna selection.

Public Members

uint8_t **gpio_select**

Whether this GPIO is connected to external antenna switch

uint8_t **gpio_num**

The GPIO number that connects to external antenna switch

struct **wifi_ant_gpio_config_t**

#include <inph_wifi_types_generic.h> Wi-Fi GPIOs configuration for antenna selection.

Public Members

wifi_ant_gpio_t **gpio_cfg**[4]

The configurations of GPIOs that connect to external antenna switch

struct **wifi_ant_config_t**

#include <inph_wifi_types_generic.h> Wi-Fi antenna configuration.

Public Members

wifi_ant_mode_t **rx_ant_mode**

Wi-Fi antenna mode for receiving

wifi_ant_t **rx_ant_default**

Default antenna mode for receiving, it's ignored if rx_ant_mode is not WIFI_ANT_MODE_AUTO

wifi_ant_mode_t **tx_ant_mode**

Wi-Fi antenna mode for transmission, it can be set to WIFI_ANT_MODE_AUTO only if rx_ant_mode is set to WIFI_ANT_MODE_AUTO

uint8_t **enabled_ant0**

Index (in antenna GPIO configuration) of enabled WIFI_ANT_MODE_ANT0

uint8_t **enabled_ant1**

Index (in antenna GPIO configuration) of enabled WIFI_ANT_MODE_ANT1

struct **wifi_action_tx_req_t**

#include <inph_wifi_types_generic.h> Action Frame Tx Request.

Public Members

wifi_interface_t **ifx**

Wi-Fi interface to send request to

uint8_t **dest_mac**[6]

Destination MAC address

wifi_action_tx_t **type**

ACTION TX operation type

uint8_t **channel**

Channel on which to perform ACTION TX Operation

uint32_t **wait_time_ms**

Duration to wait for on target channel

bool **no_ack**

Indicates no ack required

wifi_action_rx_cb_t **rx_cb**

Rx Callback to receive action frames

uint8_t **op_id**

Unique Identifier for operation provided by wifi driver

uint32_t **data_len**

Length of the appended Data

uint8_t **data**[0]

Appended Data payload

struct **wifi_roc_req_t**

#include <inph_wifi_types_generic.h> Remain on Channel request.

Public Members

wifi_interface_t **ifx**

WiFi interface to send request to

wifi_roc_t **type**

ROC operation type

uint8_t **channel**

Channel on which to perform ROC Operation

wifi_second_chan_t **sec_channel**

Secondary channel

uint32_t **wait_time_ms**

Duration to wait for on target channel

wifi_action_rx_cb_t **rx_cb**

Rx Callback to receive any response

uint8_t **op_id**

ID of this specific ROC operation provided by wifi driver

wifi_action_roc_done_cb_t **done_cb**

Callback to function that will be called upon ROC done. If assigned, WIFI_EVENT_ROC_DONE event will not be posted

struct **wifi_ftm_initiator_cfg_t**

#include <inph_wifi_types_generic.h> FTM Initiator configuration.

Public Members

uint8_t **rinph_mac**[6]

MAC address of the FTM Responder

uint8_t **channel**

Primary channel of the FTM Responder

uint8_t **frm_count**

No. of FTM frames requested in terms of 4 or 8 bursts (allowed values - 0(No pref), 16, 24, 32, 64)

uint16_t **burst_period**

Requested period between FTM bursts in 100's of milliseconds (allowed values 0(No pref) - 100)

bool **use_get_report_api**

True - Using `inph_wifi_ftm_get_report` to get FTM report, False - Using `ftm_report_data` from `WIFI_EVENT_FTM_REPORT` to get FTM report

struct **wifi_nan_wfa_ssi_t**

#include <inph_wifi_types_generic.h> WFA defined Protocol types in NAN service specific info attribute.

Public Members

uint8_t **wfa_oui**[WIFI_OUI_LEN]

WFA OUI - 0x50, 0x6F, 0x9A

wifi_nan_svc_proto_t **proto**

WFA defined protocol types

uint8_t **payload**[0]

Service Info payload

struct **wifi_nan_publish_cfg_t**

#include <inph_wifi_types_generic.h> NAN Publish service configuration parameters.

Public Members

char **service_name**[INPH_WIFI_MAX_SVC_NAME_LEN]

Service name identifier

wifi_nan_service_type_t **type**

Service type

char **matching_filter**[INPH_WIFI_MAX_FILTER_LEN]

Comma separated filters for filtering services

char **svc_info**[INPH_WIFI_MAX_SVC_INFO_LEN]

To be deprecated in next major release, use ssi instead

uint8_t **single_replied_event**

Give single Replied event or every time

uint8_t **datapath_reqd**

NAN Datapath required for the service

uint8_t **fsd_reqd**

Further Service Discovery(FSD) required

uint8_t **fsd_gas**

0 - Follow-up used for FSD, 1 - GAS used for FSD

uint8_t **reserved**

Reserved

uint16_t **ssi_len**

Length of service specific info, maximum allowed length - INPH_WIFI_MAX_SVC_SSI_LEN

uint8_t ***ssi**

Service Specific Info of type wifi_nan_wfa_ssi_t for WFA defined protocols, otherwise proprietary and defined by Applications

struct **wifi_nan_subscribe_cfg_t**

#include <inph_wifi_types_generic.h> NAN Subscribe service configuration parameters.

Public Members

char **service_name**[INPH_WIFI_MAX_SVC_NAME_LEN]

Service name identifier

wifi_nan_service_type_t **type**

Service type

char **matching_filter**[INPH_WIFI_MAX_FILTER_LEN]

Comma separated filters for filtering services

char **svc_info**[INPH_WIFI_MAX_SVC_INFO_LEN]

To be deprecated in next major release, use ssi instead

uint8_t **single_match_event**

Give single Match event(per SSI update) or every time

uint8_t **datapath_reqd**

NAN Datapath required for the service

uint8_t **fsd_reqd**

Further Service Discovery(FSD) required

uint8_t **fsd_gas**

0 - Follow-up used for FSD, 1 - GAS used for FSD

uint8_t **reserved**

Reserved

uint16_t **ssi_len**

Length of service specific info, maximum allowed length - INPH_WIFI_MAX_SVC_SSI_LEN

uint8_t ***ssi**

Service Specific Info of type wifi_nan_wfa_ssi_t for WFA defined protocols, otherwise proprietary and defined by Applications

struct **wifi_nan_followup_params_t**

#include <inph_wifi_types_generic.h> NAN Follow-up parameters.

Public Members

uint8_t **inst_id**

Own service instance id

uint8_t **peer_inst_id**

Peer's service instance id

uint8_t **peer_mac**[6]

Peer's MAC address

char **svc_info**[INPH_WIFI_MAX_SVC_INFO_LEN]

To be deprecated in next major release, use ssi instead

uint16_t **ssi_len**

Length of service specific info, maximum allowed length - INPH_WIFI_MAX_FUP_SSI_LEN

uint8_t ***ssi**

Service Specific Info of type wifi_nan_wfa_ssi_t for WFA defined protocols, otherwise proprietary and defined by Applications

struct **wifi_nan_datapath_req_t**

#include <inph_wifi_types_generic.h> NAN Datapath Request parameters.

Public Members

uint8_t **pub_id**

Publisher's service instance id

uint8_t **peer_mac**[6]

Peer's MAC address

bool **confirm_required**

NDP Confirm frame required

struct **wifi_nan_datapath_rinph_t**

#include <inph_wifi_types_generic.h> NAN Datapath Response parameters.

Public Members

bool **accept**

True - Accept incoming NDP, False - Reject it

uint8_t **ndp_id**

NAN Datapath Identifier

uint8_t **peer_mac**[6]

Peer's MAC address

struct **wifi_nan_datapath_end_req_t**

#include <inph_wifi_types_generic.h> NAN Datapath End parameters.

Public Members

uint8_t **ndp_id**

NAN Datapath Identifier

uint8_t **peer_mac**[6]

Peer's MAC address

struct **wifi_event_sta_scan_done_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_SCAN_DONE event.

Public Members

uint32_t **status**

Status of scanning APs: 0 — success, 1 - failure

uint8_t **number**

Number of scan results

uint8_t **scan_id**

Scan sequence number, used for block scan

struct **wifi_event_sta_connected_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_STA_CONNECTED event.

Public Members

uint8_t **ssid**[32]

SSID of connected AP

uint8_t **ssid_len**

SSID length of connected AP

uint8_t **bssid**[6]

BSSID of connected AP

uint8_t **channel**

Channel of connected AP

wifi_auth_mode_t **authmode**

Authentication mode used by the connection

uint16_t **aid**

Authentication id assigned by the connected AP

struct **wifi_event_sta_disconnected_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_STA_DISCONNECTED event.

Public Members

uint8_t **ssid**[32]

SSID of disconnected AP

uint8_t **ssid_len**

SSID length of disconnected AP

uint8_t **bssid**[6]

BSSID of disconnected AP

uint8_t **reason**

Disconnection reason

int8_t **rsssi**

Disconnection RSSI

struct **wifi_event_sta_authmode_change_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_STA_AUTHMODE_CHANGE event.

Public Members

wifi_auth_mode_t **old_mode**

Old auth mode of AP

wifi_auth_mode_t **new_mode**

New auth mode of AP

struct **wifi_event_sta_wps_er_pin_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_STA_WPS_ER_PIN event.

Public Members

uint8_t **pin_code**[8]

PIN code of station in enrollee mode

struct **wifi_event_sta_wps_er_success_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_STA_WPS_ER_SUCCESS event.

Public Members

uint8_t **ap_cred_cnt**

Number of AP credentials received

uint8_t **ssid**[MAX_SSID_LEN]

SSID of AP

uint8_t **passphrase**[MAX_PASSPHRASE_LEN]

Passphrase for the AP

struct *wifi_event_sta_wps_er_success_t* **ap_cred**[MAX_WPS_AP_CRED]

All AP credentials received from WPS handshake

struct **wifi_event_ap_staconnected_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_AP_STACONNECTED event.

Public Members

uint8_t **mac**[6]

MAC address of the station connected to Soft-AP

uint8_t **aid**

AID assigned by the Soft-AP to the connected station

bool **is_mesh_child**

Flag indicating whether the connected station is a mesh child

struct **wifi_event_ap_stadisconnected_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_AP_STADISCONNECTED event.

Public Members

uint8_t **mac**[6]

MAC address of the station disconnects from the soft-AP

uint8_t **aid**

AID that the Soft-AP assigned to the disconnected station

bool **is_mesh_child**

Flag indicating whether the disconnected station is a mesh child

uint16_t **reason**

Disconnection reason

struct **wifi_event_ap_probe_req_rx_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_AP_PROBEREQRECVD event.

Public Members

int **rss_i**

Received probe request signal strength

uint8_t **mac**[6]

MAC address of the station which send probe request

struct **wifi_event_bss_rssi_low_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_STA_BSS_RSSI_LOW event.

Public Members

int32_t **rssi**

RSSI value of bss

struct **wifi_event_home_channel_change_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_HOME_CHANNEL_CHANGE event.

Public Members

uint8_t **old_chan**

Old home channel of the device

wifi_second_chan_t **old_snd**

Old second channel of the device

uint8_t **new_chan**

New home channel of the device

wifi_second_chan_t **new_snd**

New second channel of the device

struct **wifi_ftm_report_entry_t**

#include <inph_wifi_types_generic.h> Structure representing a report entry for Fine Timing Measurement (FTM) in Wi-Fi.

This structure holds the information related to the FTM process between a Wi-Fi FTM Initiator and a Wi-Fi FTM Responder. FTM is used for precise distance measurement by timing the exchange of frames between devices.

Public Members

uint8_t **dlog_token**

Dialog Token of the FTM frame

int8_t **rssi**

RSSI of the FTM frame received

uint32_t **rtt**

Round Trip Time in pSec with a peer

uint64_t **t1**

Time of departure of FTM frame from FTM Responder in pSec

uint64_t **t2**

Time of arrival of FTM frame at FTM Initiator in pSec

uint64_t **t3**

Time of departure of ACK from FTM Initiator in pSec

uint64_t **t4**

Time of arrival of ACK at FTM Responder in pSec

struct **wifi_event_ftm_report_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_FTM_REPORT event.

Public Members

uint8_t **peer_mac**[6]

MAC address of the FTM Peer

wifi_ftm_status_t **status**

Status of the FTM operation

uint32_t **rtt_raw**

Raw average Round-Trip-Time with peer in Nano-Seconds

uint32_t **rtt_est**

Estimated Round-Trip-Time with peer in Nano-Seconds

uint32_t **dist_est**

Estimated one-way distance in Centi-Meters

wifi_ftm_report_entry_t ***ftm_report_data**

Pointer to FTM Report, should be freed after use. Note: Highly recommended to use API `inph_wifi_ftm_get_report` to get the report instead of using this

uint8_t **ftm_report_num_entries**

Number of entries in the FTM Report data

struct **wifi_event_action_tx_status_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_ACTION_TX_STATUS event

Public Members

wifi_interface_t **ifx**

WiFi interface to send request to

uint32_t **context**

Context to identify the request

wifi_action_tx_status_type_t **status**

Status of the operation

uint8_t **op_id**

ID of the corresponding operation that was provided during action tx request

uint8_t **channel**

Channel provided in tx request

struct **wifi_event_roc_done_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_ROC_DONE event.

Public Members

uint32_t **context**

Context to identify the initiator of the request

wifi_roc_done_status_t **status**

ROC status

uint8_t **op_id**

ID of the corresponding ROC operation

uint8_t **channel**

Channel provided in tx request

struct **wifi_event_ap_wps_rg_pin_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_AP_WPS_RG_PIN event.

Public Members

uint8_t **pin_code**[8]

PIN code of station in enrollee mode

struct **wifi_event_ap_wps_rg_fail_reason_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_AP_WPS_RG_FAILED event.

Public Members

wps_fail_reason_t **reason**

WPS failure reason wps_fail_reason_t

uint8_t **peer_macaddr**[6]

Enrollee mac address

struct **wifi_event_ap_wps_rg_success_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_AP_WPS_RG_SUCCESS event.

Public Members

uint8_t **peer_macaddr**[6]

Enrollee mac address

struct **wifi_event_nan_svc_match_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_NAN_SVC_MATCH event.

Public Members

uint8_t **subscribe_id**

Subscribe Service Identifier

uint8_t **publish_id**

Publish Service Identifier

uint8_t **pub_if_mac**[6]

NAN Interface MAC of the Publisher

bool **update_pub_id**

Indicates whether publisher's service ID needs to be updated

uint8_t **datapath_reqd**

NAN Datapath required for the service

uint8_t **fsd_reqd**

Further Service Discovery(FSD) required

uint8_t **fsd_gas**

0 - Follow-up used for FSD, 1 - GAS used for FSD

uint8_t **reserved**

Reserved

uint32_t **reserved_1**

Reserved

uint32_t **reserved_2**

Reserved

uint8_t **ssi_version**

Indicates version of SSI in Publish instance, 0 if not available

uint16_t **ssi_len**

Length of service specific info

uint8_t **ssi[]**

Service specific info of Publisher

struct **wifi_event_nan_replied_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_NAN_REPLIED event.

Public Members

uint8_t **publish_id**

Publish Service Identifier

uint8_t **subscribe_id**

Subscribe Service Identifier

uint8_t **sub_if_mac[6]**

NAN Interface MAC of the Subscriber

uint32_t **reserved_1**

Reserved

uint32_t **reserved_2**

Reserved

uint16_t **ssi_len**

Length of service specific info

uint8_t **ssi[]**

Service specific info of Subscriber

struct **wifi_event_nan_receive_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_NAN_RECEIVE event.

Public Members

uint8_t **inst_id**

Our Service Identifier

uint8_t **peer_inst_id**

Peer's Service Identifier

uint8_t **peer_if_mac[6]**

Peer's NAN Interface MAC

uint8_t **peer_svc_info[INPH_WIFI_MAX_SVC_INFO_LEN]**

To be deprecated in next major release, use ssi instead

uint32_t **reserved_1**

Reserved

uint32_t **reserved_2**

Reserved

uint16_t **ssi_len**

Length of service specific info

uint8_t **ssi[]**

Service specific info from Follow-up

struct **wifi_event_ndp_indication_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_NDP_INDICATION event.

Public Members

uint8_t **publish_id**

Publish Id for NAN Service

uint8_t **ndp_id**

NDP instance id

uint8_t **peer_nmi**[6]

Peer's NAN Management Interface MAC

uint8_t **peer_ndi**[6]

Peer's NAN Data Interface MAC

uint8_t **svc_info**[INPH_WIFI_MAX_SVC_INFO_LEN]

To be deprecated in next major release, use ssi instead

uint32_t **reserved_1**

Reserved

uint32_t **reserved_2**

Reserved

uint16_t **ssi_len**

Length of service specific info

uint8_t **ssi**[]

Service specific info from NDP/NDPE Attribute

struct **wifi_event_ndp_confirm_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_NDP_CONFIRM event.

Public Members

uint8_t **status**

NDP status code

uint8_t **ndp_id**

NDP instance id

uint8_t **peer_nmi**[6]

Peer's NAN Management Interface MAC

uint8_t **peer_ndi**[6]

Peer's NAN Data Interface MAC

uint8_t **own_ndi**[6]

Own NAN Data Interface MAC

uint8_t **svc_info**[INPH_WIFI_MAX_SVC_INFO_LEN]

To be deprecated in next major release, use ssi instead

uint32_t **reserved_1**

Reserved

uint32_t **reserved_2**

Reserved

uint16_t **ssi_len**

Length of Service Specific Info

uint8_t **ssi**[]

Service specific info from NDP/NDPE Attribute

struct **wifi_event_ndp_terminated_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_NDP_TERMINATED event.

Public Members

uint8_t **reason**

Termination reason code

uint8_t **ndp_id**

NDP instance id

uint8_t **init_ndi**[6]

Initiator's NAN Data Interface MAC

struct **wifi_event_neighbor_report_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_STA_NEIGHBOR_REP event.

Public Members

uint8_t **report**[INPH_WIFI_MAX_NEIGHBOR_REP_LEN]

Neighbor Report received from the AP (will be deprecated in next major release, use n_report instead)

uint16_t **report_len**

Length of the report

uint8_t **n_report**[]

Neighbor Report received from the AP

struct **wifi_event_ap_wrong_password_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_AP_WRONG_PASSWORD event

Public Members

uint8_t **mac**[6]

MAC address of the station trying to connect to Soft-AP

struct **wifi_tx_rate_config_t**

#include <inph_wifi_types_generic.h> Argument structure for wifi_tx_rate_config.

Public Members

wifi_phy_mode_t **phymode**

Phymode of specified interface

wifi_phy_rate_t **rate**

Rate of specified interface

bool **ersu**

Using ERSU to send frame, ERSU is a transmission mode related to 802.11 ax. ERSU is always used in long distance transmission, and its frame has lower rate compared with SU mode

bool **dcm**

Using dcm rate to send frame

struct **wifi_reg_rule_t**

#include <inph_wifi_types_generic.h> Argument structure for regulatory rule

Public Members

uint8_t **start_channel**

start channel of regulatory rule

uint8_t **end_channel**

end channel of regulatory rule

uint16_t **max_bandwidth**

max bandwidth(MHz) of regulatory rule, 1:20M, 2:40M, 3:80M, 4:160M

uint16_t **max_eirp**

indicates the maximum Equivalent Isotropically Radiated Power (EIRP), typically measured in dBm

uint16_t **is_dfs**

flag to identify dfs channel

uint16_t **reserved**

reserved

struct **wifi_regulatory_t**

#include <inph_wifi_types_generic.h> Argument structure for regdomain

Public Members

uint8_t **n_reg_rules**

number of regulatory rules

wifi_reg_rule_t **reg_rules**[WIFI_MAX_REGULATORY_RULE_NUM]

array of regulatory rules

struct **wifi_regdomain_t**

#include <inph_wifi_types_generic.h> Argument structure for regdomain

Public Members

char **cn**[2]

country code string

uint8_t **regulatory_type**

regulatory type of country

struct **wifi_tx_info_t**

#include <inph_wifi_types_generic.h> Information of wifi sending data.

Public Members

uint8_t ***des_addr**

The address of the receive device

uint8_t ***src_addr**

The address of the sending device

wifi_interface_t **ifidx**

Interface of sending 80211 tx data

uint8_t ***data**

The data for 80211 tx, start from the MAC header

uint8_t **data_len**

The frame body length for 80211 tx, excluding the MAC header

wifi_phy_rate_t **rate**

Data rate

wifi_tx_status_t **tx_status**

Status of sending 80211 tx data

struct **wifi_event_sta_beacon_offset_unstable_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_STA_BEACON_OFFSET_UNSTABLE event

Public Members

float **beacon_success_rate**

Received beacon success rate

struct **wifi_event_dpp_uri_ready_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_DPP_URI_READY event

Public Members

uint32_t **uri_data_len**

URI data length including null termination

char **uri**[]

URI data

struct **wifi_event_dpp_config_received_t**

#include <inph_wifi_types_generic.h> Argument structure for WIFI_EVENT_DPP_CFG_RECVD event

Public Members

`wifi_config_t` **wifi_cfg**

Received WIFI config in DPP

struct **wifi_event_dpp_failed_t**

`#include <inph_wifi_types_generic.h>` Argument structure for WIFI_EVENT_DPP_FAIL event

Public Members

int **failure_reason**

Failure reason

Wi-Fi Functions

`inph_err_t inph_wifi_init` (const `wifi_init_config_t` *config)

Initialize WiFi Allocate resource for WiFi driver, such as WiFi control structure, RX/TX buffer, WiFi NVS structure etc. This WiFi also starts WiFi task.

Attention

1. This API must be called before all other WiFi API can be called

Attention

2. Always use WIFI_INIT_CONFIG_DEFAULT macro to initialize the configuration to default values, this can guarantee all the fields get correct value when more fields are added into `wifi_init_config_t` in future release. If you want to set your own initial values, overwrite the default values which are set by WIFI_INIT_CONFIG_DEFAULT. Please be notified that the field 'magic' of `wifi_init_config_t` should always be WIFI_INIT_CONFIG_MAGIC!

Parameters

config – pointer to WiFi initialized configuration structure; can point to a temporary variable.

Returns

- INPH_OK: succeed
- INPH_ERR_NO_MEM: out of memory
- others: refer to error code `inph_err.h`

`inph_err_t inph_wifi_deinit` (void)

Deinit WiFi Free all resource allocated in `inph_wifi_init` and stop WiFi task.

Attention

1. This API should be called if you want to remove WiFi driver from the system

Returns

- INPH_OK: succeed
- INPH_ERR_WIFI_NOT_INIT: WiFi is not initialized by `inph_wifi_init`

`inph_err_t inph_wifi_start` (void)

Start WiFi according to current configuration. If mode is `WIFI_MODE_STA`, it creates station control block and starts station. If mode is `WIFI_MODE_AP`, it creates soft-AP control block and starts soft-AP. If mode is `WIFI_MODE_APSTA`, it creates soft-AP and station control block and starts soft-AP and station. If mode is `WIFI_MODE_NAN`, it creates NAN control block and starts NAN.

Returns

- `INPH_OK`: succeed
- `INPH_ERR_WIFI_NOT_INIT`: WiFi is not initialized by `inph_wifi_init`
- `INPH_ERR_INVALID_ARG`: It doesn't normally happen, the function called inside the API was passed invalid argument, user should check if the WiFi related config is correct
- `INPH_ERR_NO_MEM`: out of memory
- `INPH_ERR_WIFI_CONN`: WiFi internal error, station or soft-AP control block wrong
- `INPH_FAIL`: other WiFi internal errors

`inph_err_t inph_wifi_stop` (void)

Stop WiFi. If mode is `WIFI_MODE_STA`, it stops station and frees station control block. If mode is `WIFI_MODE_AP`, it stops soft-AP and frees soft-AP control block. If mode is `WIFI_MODE_APSTA`, it stops station/soft-AP and frees station/soft-AP control block. If mode is `WIFI_MODE_NAN`, it stops NAN and frees NAN control block.

Returns

- `INPH_OK`: succeed
- `INPH_ERR_WIFI_NOT_INIT`: WiFi is not initialized by `inph_wifi_init`

`inph_err_t inph_wifi_set_mode` (`wifi_mode_t mode`)

Set the WiFi operating mode.

Set the WiFi operating mode **as** station, soft-AP, station+soft-AP **or** NAN.
The default mode **is** station mode.

Parameters

mode – WiFi operating mode

Returns

- `INPH_OK`: succeed
- `INPH_ERR_WIFI_NOT_INIT`: WiFi is not initialized by `inph_wifi_init`
- `INPH_ERR_INVALID_ARG`: invalid argument
- others: refer to error code in `inph_err.h`

`inph_err_t inph_wifi_set_config` (`wifi_interface_t interface`, `wifi_config_t *conf`)

Set the configuration of the STA, AP or NAN.

Attention

1. This API can be called only when specified interface is enabled, otherwise, API fail

Attention

2. For station configuration, `ssid_set` needs to be 0; and it needs to be 1 only when users need to check the MAC address of the AP.

Attention

3. ESP devices are limited to only one channel, so when in the soft-AP+station mode, the soft-AP will adjust its channel automatically to be the same as the channel of the station.

Attention

4. The configuration will be stored in NVS for station and soft-AP

Parameters

- **interface** – interface
- **conf** – station, soft-AP or NAN configuration

Returns

- `INPH_OK`: succeed
- `INPH_ERR_WIFI_NOT_INIT`: WiFi is not initialized by `inph_wifi_init`
- `INPH_ERR_INVALID_ARG`: invalid argument
- `INPH_ERR_WIFI_IF`: invalid interface
- `INPH_ERR_WIFI_MODE`: invalid mode
- `INPH_ERR_WIFI_PASSWORD`: invalid password
- `INPH_ERR_WIFI_NVS`: WiFi internal NVS error
- `INPH_ERR_WIFI_STATE`: WiFi still connecting when invoke `inph_wifi_set_config`
- others: refer to the error code in `inph_err.h`

`inph_err_t inph_wifi_connect` (void)

Connect WiFi station to the AP.

Attention

1. This API only impact `WIFI_MODE_STA` or `WIFI_MODE_APSTA` mode

Attention

2. If station interface is connected to an AP, call `inph_wifi_disconnect` to disconnect.

Attention

3. The scanning triggered by `inph_wifi_scan_start()` will not be effective until connection between device and the AP is established. If device is scanning and connecting at the same time, it will abort scanning and return a warning message and error number `INPH_ERR_WIFI_STATE`.

Attention

4. This API attempts to connect to an Access Point (AP) only once. To enable reconnection in case of a connection failure, please use the 'failure_retry_cnt' feature in the 'wifi_sta_config_t'. Users are suggested to implement reconnection logic in their application for scenarios where the specified AP does not exist, or reconnection is desired after the device has received a disconnect event.

Returns

- `INPH_OK`: succeed
- `INPH_ERR_WIFI_NOT_INIT`: WiFi is not initialized by `inph_wifi_init`

- INPH_ERR_WIFI_NOT_STARTED: WiFi is not started by `inph_wifi_start`
- INPH_ERR_WIFI_MODE: WiFi mode error
- INPH_ERR_WIFI_CONN: WiFi internal error, station or soft-AP control block wrong
- INPH_ERR_WIFI_SSID: SSID of AP which station connects is invalid

`inph_err_t inph_wifi_disconnect` (void)

Disconnect WiFi station from the AP.

Returns

- INPH_OK: succeed
- INPH_ERR_WIFI_NOT_INIT: WiFi was not initialized by `inph_wifi_init`
- INPH_ERR_WIFI_NOT_STARTED: WiFi was not started by `inph_wifi_start`
- INPH_FAIL: other WiFi internal errors

`inph_err_t inph_wifi_scan_start` (const `wifi_scan_config_t` *config, bool block)

Scan all available APs.

Attention

If this API is called, the found APs are stored in WiFi driver dynamic allocated memory. And then can be freed in `inph_wifi_scan_get_ap_records()`, `inph_wifi_scan_get_ap_record()` or `inph_wifi_clear_ap_list()`, so call any one to free the memory once the scan is done.

Attention

The values of maximum active scan time and passive scan time per channel are limited to 1500 milliseconds. Values above 1500ms may cause station to disconnect from AP and are not recommended.

Parameters

- **config** – configuration settings for scanning, if set to NULL default settings will be used of which default values are `show_hidden:false`, `scan_type:active`, `scan_time.active.min:0`, `scan_time.active.max:120` milliseconds, `scan_time.passive:360` milliseconds `home_chan_dwell_time:30ms`
- **block** – if block is true, this API will block the caller until the scan is done, otherwise it will return immediately

Returns

- INPH_OK: succeed
- INPH_ERR_WIFI_NOT_INIT: WiFi is not initialized by `inph_wifi_init`
- INPH_ERR_WIFI_NOT_STARTED: WiFi was not started by `inph_wifi_start`
- INPH_ERR_WIFI_TIMEOUT: blocking scan is timeout
- INPH_ERR_WIFI_STATE: WiFi still connecting when invoke `inph_wifi_scan_start`
- others: refer to error code in `inph_err.h`

`inph_err_t inph_wifi_scan_stop` (void)

Stop the scan in process.

Returns

- INPH_OK: succeed

- INPH_ERR_WIFI_NOT_INIT: WiFi is not initialized by inph_wifi_init
- INPH_ERR_WIFI_NOT_STARTED: WiFi is not started by inph_wifi_start
- INPH_ERR_WIFI_STATE: WiFi is still connecting when inph_wifi_scan_stop() is invoked.

inph_err_t **inph_wifi_scan_get_ap_num** (uint16_t *number)

Get number of APs found in last scan.

Attention

This API can only be called when the scan is completed, otherwise it may get wrong value.

Parameters

number – [out] store number of APs found in last scan

Returns

- INPH_OK: succeed
- INPH_ERR_WIFI_NOT_INIT: WiFi is not initialized by inph_wifi_init
- INPH_ERR_WIFI_NOT_STARTED: WiFi is not started by inph_wifi_start
- INPH_ERR_INVALID_ARG: invalid argument

inph_err_t **inph_wifi_scan_get_ap_records** (uint16_t *number, wifi_ap_record_t *ap_records)

Retrieve the list of APs found during the last scan. The returned AP list is sorted in descending order based on RSSI.

Attention

This API will free all memory occupied by scanned AP list.

Parameters

- **number** – [inout] As input param, it stores max AP number ap_records can hold. As output param, it receives the actual AP number this API returns.
- **ap_records** – wifi_ap_record_t array to hold the found APs

Returns

- INPH_OK: succeed
- INPH_ERR_WIFI_NOT_INIT: WiFi is not initialized by inph_wifi_init
- INPH_ERR_WIFI_NOT_STARTED: WiFi is not started by inph_wifi_start
- INPH_ERR_INVALID_ARG: invalid argument
- INPH_ERR_NO_MEM: out of memory

inph_err_t **inph_wifi_clear_ap_list** (void)

Clear AP list found in last scan.

Attention

This API will free all memory occupied by scanned AP list. When the obtained AP list fails, AP records must be cleared, otherwise it may cause memory leakage.

Returns

- INPH_OK: succeed
- INPH_ERR_WIFI_NOT_INIT: WiFi is not initialized by inph_wifi_init
- INPH_ERR_WIFI_NOT_STARTED: WiFi is not started by inph_wifi_start
- INPH_ERR_WIFI_MODE: WiFi mode is wrong
- INPH_ERR_INVALID_ARG: It doesn't normally happen, the function called inside the API was passed invalid argument, user should check if the WiFi related config is correct

inph_err_t **inph_wifi_set_ps** (*wifi_ps_type_t* type)

Set current WiFi power save type.

Attention

Default power save type is WIFI_PS_NONE.

Parameters

type – power save type

Returns

INPH_OK: succeed

inph_err_t **inph_wifi_get_ps** (*wifi_ps_type_t* *type)

Get current WiFi power save type.

Attention

Default power save type is WIFI_PS_MIN_MODEM.

Parameters

type – [out] store current power save type

Returns

INPH_OK: succeed

inph_err_t **inph_wifi_set_default_wifi_sta_handlers** (void)

Sets default wifi event handlers for STA interface.

Returns

- INPH_OK on success, error returned from inph_event_handler_register if failed

inph_err_t **inph_wifi_set_default_wifi_ap_handlers** (void)

Sets default wifi event handlers for AP interface.

Returns

- INPH_OK on success, error returned from inph_event_handler_register if failed

inph_err_t **inph_wifi_set_default_wifi_nan_handlers** (void)

Sets default wifi event handlers for NAN interface.

Returns

- INPH_OK on success, error returned from inph_event_handler_register if failed

`inph_err_t inph_wifi_clear_default_wifi_driver_and_handlers` (void *inph_netif)

Clears default wifi event handlers for supplied network interface.

Parameters

inph_netif – instance of corresponding if object

Returns

- INPH_OK on success, error returned from `inph_event_handler_register` if failed

void `inph_wifi_destroy_if_driver` (wifi_netif_driver_t h)

Destroys wifi driver instance.

Parameters

h – pointer to wifi interface handle

`inph_err_t inph_wifi_get_if_mac` (wifi_netif_driver_t ifx, uint8_t mac[6])

Return mac of specified wifi driver instance.

Parameters

- **ifx** – [in] pointer to wifi interface handle
- **mac** – [out] output mac address

Returns

INPH_OK on success

`inph_err_t inph_wifi_register_if_rxcb` (wifi_netif_driver_t ifx, inph_netif_receive_t fn, void *arg)

Register interface receive callback function with argument.

Parameters

- **ifx** – [in] pointer to wifi interface handle
- **fn** – [in] function to be registered (typically `inph_netif_receive`)
- **arg** – [in] argument to be supplied to registered function (typically `inph_netif` ptr)

Returns

INPH_OK on success

2.1.2 Application Protocols

2.1.2.1 MQTT

Overview

INPH-MQTT MQTT over WSS Sample application (See the README.md file in the upper level ‘examples’ directory for more information about examples.)

This example connects to the broker `mqtt.eclipseprojects.io` over secure websockets and as a demonstration subscribes/unsubscribes and send a message on certain topic. (Please note that the public broker is maintained by the community so may not be always available, for details please see this [disclaimer](<https://www.emqx.com/en/mqtt/public-mqtt5-broker>))

It uses INPH-MQTT library which implements mqtt client to connect to mqtt broker.

How to use example

BROKER_URL Change

- If the default connection to ['broker.emqx.io'] is problematic, you can choose from the following free MQTT testing website.
- ['broker.emqx.io']
MQTT Broker Info Broker: broker.emqx.io TCP Port: 1883 WebSocket Port: 8083 SSL/TLS Port: 8883
WebSocket Secure Port: 8084 QUIC Port: 14567
- ['test.mosquitto.org']
- ['mqtt.eclipseprojects.io']
- Connection_Type Protocol Default_Port Description TCP (Non-encrypted) mqtt:// 1883 Standard MQTT port
TCP (TLS-encrypted) mqtt:// or ssl:// 8883 Secure MQTT Port WebSocket(Non-encrypted) ws:// 80/8080/8000
HTTP WebSocket Port WebSocket(TLS-encrypted) wss:// 443/8443/8081 HTTPS WebSocket port

Configure the project

- Open the project configuration menu (*idf.py menuconfig*)
- Configure Wi-Fi or Ethernet under “Example Connection Configuration” menu. See “Establishing Wi-Fi or Ethernet Connection” section in [examples/protocols/README.md](../README.md) for more details.

Note how to create a PEM certificate for mqtt.eclipseprojects.io:

PEM certificate for this example could be extracted from an `openssl s_client` command connecting to mqtt.eclipseprojects.io. In case a host operating system has `openssl` and `sed` packages installed, one could execute the following command to download and save the root certificate to a file (Note for Windows users: Both Linux like environment or Windows native packages may be used).

```
echo "" | openssl s_client -showcerts -connect mqtt.eclipseprojects.io:443 | \
↪sed -n "1,/Root/d; /BEGIN/,/END/p" | openssl x509 -outform PEM >mqtt_
↪eclipse_org.pem
```

Please note that this is not a general command for downloading a root certificate for an arbitrary host; this command works with mqtt.eclipseprojects.io as the site provides root certificate in the chain, which then could be extracted with text operation.

Build and Flash

Build the project and flash it to the board, then run monitor tool to view serial output:

```
idf.py -p PORT flash monitor
```

(To exit the serial monitor, type `Ctrl-]`.)

See the Getting Started Guide for full steps to configure and use INPH-IDF to build projects.

Example Output

API Reference

Defines

MQTT_OVER_TCP_SCHEME

MQTT_OVER_SSL_SCHEME

MQTT_OVER_WS_SCHEME

MQTT_OVER_WSS_SCHEME

MQTT_ERROR_TYPE_INPH_TLS

MQTT_ERROR_TYPE_TCP_TRANSPORT error type hold all sorts of transport layer errors, including ESP-TLS error, but in the past only the errors from MQTT_ERROR_TYPE_INPH_TLS layer were reported, so the ESP-TLS error type is re-defined here for backward compatibility

inph_mqtt_client_subscribe (client_handle, topic_type, qos_or_size)

Convenience macro to select subscribe function to use.

Notes:

- Usage of `inph_mqtt_client_subscribe_single` is the same as previous `inph_mqtt_client_subscribe`, refer to it for details.

Parameters

- **client_handle** – *MQTT* client handle
- **topic_type** – Needs to be `char*` for single subscription or `inph_mqtt_topic_t` for multiple topics
- **qos_or_size** – It's either a qos when subscribing to a single topic or the size of the subscription array when subscribing to multiple topics.

Returns

message_id of the subscribe message on success -1 on failure -2 in case of full outbox.

Typedefs

```
typedef void *inph_event_loop_handle_t
```

```
typedef void *inph_event_handler_t
```

```
typedef struct inph_mqtt_client *inph_mqtt_client_handle_t
```

```
typedef enum inph_mqtt_event_id_t inph_mqtt_event_id_t
```

MQTT event types.

User event handler receives context data in `inph_mqtt_event_t` structure with

- client – *MQTT* client handle
- various other data depending on event type

```
typedef enum inph_mqtt_connect_return_code_t inph_mqtt_connect_return_code_t
```

MQTT connection error codes propagated via ERROR event

```
typedef enum inph_mqtt_error_type_t inph_mqtt_error_type_t
```

MQTT connection error codes propagated via ERROR event

```
typedef enum inph_mqtt_transport_t inph_mqtt_transport_t
```

```
typedef enum inph_mqtt_protocol_ver_t inph_mqtt_protocol_ver_t
```

MQTT protocol version used for connection

```
typedef struct inph_mqtt_error_codes inph_mqtt_error_codes_t
```

MQTT error code structure to be passed as a contextual information into ERROR event

Important: This structure extends `inph_tls_last_error` error structure and is backward compatible with it (so might be down-casted and treated as `inph_tls_last_error` error, but recommended to update applications if used this way previously)

Use this structure directly checking `error_type` first and then appropriate error code depending on the source of the error:

error_type	related member variables	note
MQTT_ERROR_TYPE_TCP_TRANSPORT	<code>inph_tls_last_inph_err</code> , <code>inph_tls_stack_err</code> , <code>inph_tls_cert_verify_flags</code> , <code>sock_errno</code>	Error reported from tcp_transport/esp-tls
MQTT_ERROR_TYPE_CONNECTION_REFUSED	<code>connect_return_code</code>	Internal error reported from <i>MQTT</i> broker on connection

```
typedef struct inph_mqtt_event_t inph_mqtt_event_t
```

MQTT event configuration structure

```
typedef inph_mqtt_event_t *inph_mqtt_event_handle_t
```

```
typedef struct inph_mqtt_client_config_t inph_mqtt_client_config_t
```

MQTT client configuration structure

- Default values can be set via menuconfig
- All certificates and key data could be passed in PEM or DER format. PEM format must have a terminating NULL character and the related len field set to 0. DER format requires a related len field set to the correct length.

```
typedef struct topic_t inph_mqtt_topic_t
```

Topic definition struct

Enums

enum **inph_mqtt_event_id_t**

MQTT event types.

User event handler receives context data in `inph_mqtt_event_t` structure with

- client - *MQTT* client handle
- various other data depending on event type

Values:

enumerator **MQTT_EVENT_ANY**

enumerator **MQTT_EVENT_ERROR**

on error event, additional context: connection return code, error handle from `inph_tls` (if supported)

enumerator **MQTT_EVENT_CONNECTED**

connected event, additional context: `session_present` flag

enumerator **MQTT_EVENT_DISCONNECTED**

disconnected event

enumerator **MQTT_EVENT_SUBSCRIBED**

subscribed event, additional context:

- `msg_id` message id
- `error_handle` `error_type` in case subscribing failed
- data pointer to broker response, check for errors.
- `data_len` length of the data for this event

enumerator **MQTT_EVENT_UNSUBSCRIBED**

unsubscribed event, additional context: `msg_id`

enumerator **MQTT_EVENT_PUBLISHED**

published event, additional context: `msg_id`

enumerator **MQTT_EVENT_DATA**

data event, additional context:

- `msg_id` message id
- topic pointer to the received topic
- `topic_len` length of the topic
- data pointer to the received data
- `data_len` length of the data for this event
- `current_data_offset` offset of the current data for this event

- `total_data_len` total length of the data received
- `retain` retain flag of the message
- `qos` QoS level of the message
- `dup` dup flag of the message Note: Multiple `MQTT_EVENT_DATA` could be fired for one message, if it is longer than internal buffer. In that case only first event contains topic pointer and length, other contain data only with current data length and current data offset updating.

enumerator **MQTT_EVENT_BEFORE_CONNECT**

The event occurs before connecting

enumerator **MQTT_EVENT_DELETED**

Notification on delete of one message from the internal outbox, if the message couldn't have been sent or acknowledged before expiring defined in `OUTBOX_EXPIRED_TIMEOUT_MS`. (events are not posted upon deletion of successfully acknowledged messages)

- This event id is posted only if `MQTT_REPORT_DELETED_MESSAGES==1`
- Additional context: `msg_id` (id of the deleted message, always 0 for QoS = 0 messages).

enumerator **MQTT_USER_EVENT**

Custom event used to queue tasks into mqtt event handler All fields from the `inph_mqtt_event_t` type could be used to pass an additional context data to the handler.

enum **inph_mqtt_connect_return_code_t**

MQTT connection error codes propagated via ERROR event

Values:

enumerator **MQTT_CONNECTION_ACCEPTED**

Connection accepted

enumerator **MQTT_CONNECTION_REFUSE_PROTOCOL**

MQTT connection refused reason: Wrong protocol

enumerator **MQTT_CONNECTION_REFUSE_ID_REJECTED**

MQTT connection refused reason: ID rejected

enumerator **MQTT_CONNECTION_REFUSE_SERVER_UNAVAILABLE**

MQTT connection refused reason: Server unavailable

enumerator **MQTT_CONNECTION_REFUSE_BAD_USERNAME**

MQTT connection refused reason: Wrong user

enumerator **MQTT_CONNECTION_REFUSE_NOT_AUTHORIZED**

MQTT connection refused reason: Wrong username or password

enum **inph_mqtt_error_type_t**

MQTT connection error codes propagated via ERROR event

Values:

enumerator **MQTT_ERROR_TYPE_NONE**

enumerator **MQTT_ERROR_TYPE_TCP_TRANSPORT**

enumerator **MQTT_ERROR_TYPE_CONNECTION_REFUSED**

enumerator **MQTT_ERROR_TYPE_SUBSCRIBE_FAILED**

enum **inph_mqtt_transport_t**

Values:

enumerator **MQTT_TRANSPORT_UNKNOWN**

enumerator **MQTT_TRANSPORT_OVER_TCP**

MQTT over TCP, using scheme: **MQTT**

enumerator **MQTT_TRANSPORT_OVER_SSL**

MQTT over SSL, using scheme: **MQTTS**

enumerator **MQTT_TRANSPORT_OVER_WS**

MQTT over Websocket, using scheme: **ws**

enumerator **MQTT_TRANSPORT_OVER_WSS**

MQTT over Websocket Secure, using scheme: **wss**

enum **inph_mqtt_protocol_ver_t**

MQTT protocol version used for connection

Values:

enumerator **MQTT_PROTOCOL_UNDEFINED**

enumerator **MQTT_PROTOCOL_V_3_1**

enumerator **MQTT_PROTOCOL_V_3_1_1**

enumerator **MQTT_PROTOCOL_V_5**

Functions

inph_mqtt_client_handle_t **inph_mqtt_client_init** (const *inph_mqtt_client_config_t* *config)

Creates *MQTT* client handle based on the configuration.

Parameters

config – *MQTT* configuration structure

Returns

mqtt_client_handle if successfully created, NULL on error

inph_err_t **inph_mqtt_client_set_uri** (*inph_mqtt_client_handle_t* client, const char *uri)

Sets *MQTT* connection URI. This API is usually used to overrides the URI configured in inph_mqtt_client_init.

Parameters

- **client** – *MQTT* client handle
- **uri** –

Returns

INPH_FAIL if URI parse error, INPH_OK on success

inph_err_t **inph_mqtt_client_start** (*inph_mqtt_client_handle_t* client)

Starts *MQTT* client with already created client handle.

Parameters

client – *MQTT* client handle

Returns

INPH_OK on success INPH_ERR_INVALID_ARG on wrong initialization INPH_FAIL on other error

inph_err_t **inph_mqtt_client_reconnect** (*inph_mqtt_client_handle_t* client)

This api is typically used to force reconnection upon a specific event.

Parameters

client – *MQTT* client handle

Returns

INPH_OK on success INPH_ERR_INVALID_ARG on wrong initialization INPH_FAIL if client is in invalid state

inph_err_t **inph_mqtt_client_disconnect** (*inph_mqtt_client_handle_t* client)

This api is typically used to force disconnection from the broker.

Parameters

client – *MQTT* client handle

Returns

INPH_OK on success INPH_ERR_INVALID_ARG on wrong initialization

inph_err_t **inph_mqtt_client_stop** (*inph_mqtt_client_handle_t* client)

Stops *MQTT* client tasks.

- Notes:
- Cannot be called from the *MQTT* event handler

Parameters

client – *MQTT* client handle

Returns

INPH_OK on success INPH_ERR_INVALID_ARG on wrong initialization INPH_FAIL if client is in invalid state

int **inph_mqtt_client_subscribe_single** (*inph_mqtt_client_handle_t* client, const char *topic, int qos)

Subscribe the client to defined topic with defined qos.

Notes:

- Client must be connected to send subscribe message
- This API is could be executed from a user task or from a *MQTT* event callback i.e. internal *MQTT* task (API is protected by internal mutex, so it might block if a longer data receive operation is in progress.
- `inph_mqtt_client_subscribe` could be used to call this function.

Parameters

- **client** – *MQTT* client handle
- **topic** – topic filter to subscribe
- **qos** – Max qos level of the subscription

Returns

message_id of the subscribe message on success -1 on failure -2 in case of full outbox.

int **inph_mqtt_client_subscribe_multiple** (*inph_mqtt_client_handle_t* client, const *inph_mqtt_topic_t* *topic_list, int size)

Subscribe the client to a list of defined topics with defined qos.

Notes:

- Client must be connected to send subscribe message
- This API is could be executed from a user task or from a *MQTT* event callback i.e. internal *MQTT* task (API is protected by internal mutex, so it might block if a longer data receive operation is in progress.
- `inph_mqtt_client_subscribe` could be used to call this function.

Parameters

- **client** – *MQTT* client handle
- **topic_list** – List of topics to subscribe
- **size** – size of topic_list

Returns

message_id of the subscribe message on success -1 on failure -2 in case of full outbox.

int **inph_mqtt_client_unsubscribe** (*inph_mqtt_client_handle_t* client, const char *topic)

Unsubscribe the client from defined topic.

Notes:

- Client must be connected to send unsubscribe message
- It is thread safe, please refer to `inph_mqtt_client_subscribe_single` for details

Parameters

- **client** – *MQTT* client handle

- **topic** –

Returns

message_id of the subscribe message on success -1 on failure

int **inph_mqtt_client_publish** (*inph_mqtt_client_handle_t* client, const char *topic, const char *data, int len, int qos, int retain)

Client to send a publish message to the broker.

Notes:

- This API might block for several seconds, either due to network timeout (10s) or if publishing payloads longer than internal buffer (due to message fragmentation)
- Client doesn't have to be connected for this API to work, enqueueing the messages with qos>1 (returning -1 for all the qos=0 messages if disconnected). If MQTT_SKIP_PUBLISH_IF_DISCONNECTED is enabled, this API will not attempt to publish when the client is not connected and will always return -1.
- It is thread safe, please refer to `inph_mqtt_client_subscribe` for details

Parameters

- **client** – *MQTT* client handle
- **topic** – topic string
- **data** – payload string (set to NULL, sending empty payload message)
- **len** – data length, if set to 0, length is calculated from payload string
- **qos** – QoS of publish message
- **retain** – retain flag

Returns

message_id of the publish message (for QoS 0 message_id will always be zero) on success. -1 on failure, -2 in case of full outbox.

int **inph_mqtt_client_enqueue** (*inph_mqtt_client_handle_t* client, const char *topic, const char *data, int len, int qos, int retain, bool store)

Enqueue a message to the outbox, to be sent later. Typically used for messages with qos>0, but could be also used for qos=0 messages if store=true.

This API generates and stores the publish message into the internal outbox and the actual sending to the network is performed in the mqtt-task context (in contrast to the `inph_mqtt_client_publish()` which sends the publish message immediately in the user task's context). Thus, it could be used as a non blocking version of `inph_mqtt_client_publish()`.

Parameters

- **client** – *MQTT* client handle
- **topic** – topic string
- **data** – payload string (set to NULL, sending empty payload message)
- **len** – data length, if set to 0, length is calculated from payload string
- **qos** – QoS of publish message
- **retain** – retain flag
- **store** – if true, all messages are enqueued; otherwise only QoS 1 and QoS 2 are enqueued

Returns

message_id if queued successfully, -1 on failure, -2 in case of full outbox.

`inph_err_t inph_mqtt_client_destroy` (*inph_mqtt_client_handle_t* client)

Destroys the client handle.

Notes:

- Cannot be called from the *MQTT* event handler

Parameters

client – *MQTT* client handle

Returns

INPH_OK INPH_ERR_INVALID_ARG on wrong initialization

`inph_err_t inph_mqtt_set_config` (*inph_mqtt_client_handle_t* client, const *inph_mqtt_client_config_t* *config)

Set configuration structure, typically used when updating the config (i.e. on “before_connect” event).

Notes:

- When calling this function make sure to have all the intendend configurations set, otherwise default values are set.

Parameters

- **client** – *MQTT* client handle
- **config** – *MQTT* configuration structure

Returns

INPH_ERR_NO_MEM if failed to allocate INPH_ERR_INVALID_ARG if conflicts on transport configuration. INPH_OK on success

`inph_err_t inph_mqtt_client_register_event` (*inph_mqtt_client_handle_t* client, *inph_mqtt_event_id_t* event, *inph_event_handler_t* event_handler, void *event_handler_arg)

Registers *MQTT* event.

Parameters

- **client** – *MQTT* client handle
- **event** – event type
- **event_handler** – handler callback
- **event_handler_arg** – handlers context

Returns

INPH_ERR_NO_MEM if failed to allocate INPH_ERR_INVALID_ARG on wrong initialization
INPH_OK on success

`inph_err_t inph_mqtt_client_unregister_event` (*inph_mqtt_client_handle_t* client, *inph_mqtt_event_id_t* event, *inph_event_handler_t* event_handler)

Unregisters mqtt event.

Parameters

- **client** – mqtt client handle
- **event** – event ID

- **event_handler** – handler to unregister

Returns

INPH_ERR_NO_MEM if failed to allocate INPH_ERR_INVALID_ARG on invalid event ID
INPH_OK on success

int **inph_mqtt_client_get_outbox_size** (*inph_mqtt_client_handle_t* client)

Get outbox size.

Parameters

client – *MQTT* client handle

Returns

outbox size 0 on wrong initialization

inph_err_t **inph_mqtt_dispatch_custom_event** (*inph_mqtt_client_handle_t* client, *inph_mqtt_event_t* *event)

Dispatch user event to the mqtt internal event loop.

Parameters

- **client** – *MQTT* client handle
- **event** – *MQTT* event handle structure

Returns

INPH_OK on success INPH_ERR_TIMEOUT if the event couldn't be queued (ref also CONFIG_MQTT_EVENT_QUEUE_SIZE)

inph_transport_handle_t **inph_mqtt_client_get_transport** (*inph_mqtt_client_handle_t* client, char *transport_scheme)

Get a transport from the scheme.

Allows extra settings to be made on the selected transport, for convenience the scheme used by the mqtt client are defined as MQTT_OVER_TCP_SCHEME, MQTT_OVER_SSL_SCHEME, MQTT_OVER_WS_SCHEME and MQTT_OVER_WSS_SCHEME. If the transport_scheme is NULL and the client was set with a custom transport the custom transport will be returned.

Notes:

- This function should be called only on MQTT_EVENT_BEFORE_CONNECT.
- The intention is to provide a way to set different configurations than the ones available from client config.
- If inph_mqtt_client_destroy is called the returned pointer will be invalidated.
- All the required settings should be made in the MQTT_EVENT_BEFORE_CONNECT event handler

Parameters

- **client** – *MQTT* client handle
- **transport_scheme** – Transport handle to search for.

Returns

the transport handle NULL in case of error

struct **inph_mqtt_error_codes**

#include <mqtt_client.h> *MQTT* error code structure to be passed as a contextual information into ERROR event

Important: This structure extends `inph_tls_last_error` error structure and is backward compatible with it (so might be down-casted and treated as `inph_tls_last_error` error, but recommended to update applications if used this way previously)

Use this structure directly checking `error_type` first and then appropriate error code depending on the source of the error:

error_type	related member variables	note
<code>MQTT_ERROR_TYPE_TCP_TRANSPORT</code>	<code>inph_tls_last_inph_err</code> , <code>inph_tls_stack_err</code> , <code>inph_tls_cert_verify_flags</code> , <code>sock_errno</code>	Error reported from tcp_transport/esp-tls
<code>MQTT_ERROR_TYPE_CONNECTION_REFUSED</code>	<code>connect_return_code</code>	Internal error reported from <i>MQTT</i> broker on connection

Public Members

`inph_err_t inph_tls_last_inph_err`

last `inph_err` code reported from esp-tls component

`int inph_tls_stack_err`

tls specific error code reported from underlying tls stack

`int inph_tls_cert_verify_flags`

tls flags reported from underlying tls stack during certificate verification

`inph_mqtt_error_type_t error_type`

error type referring to the source of the error

`inph_mqtt_connect_return_code_t connect_return_code`

connection refused error code reported from MQTT* broker on connection

`int inph_transport_sock_errno`

errno from the underlying socket

struct `inph_mqtt_event_t`

#include <mqtt_client.h> MQTT event configuration structure

Public Members

`inph_mqtt_event_id_t event_id`

MQTT event type

`inph_mqtt_client_handle_t client`

MQTT client handle for this event

char *`data`

Data associated with this event

int **data_len**

Length of the data for this event

int **total_data_len**

Total length of the data (longer data are supplied with multiple events)

int **current_data_offset**

Actual offset for the data associated with this event

char ***topic**

Topic associated with this event

int **topic_len**

Length of the topic for this event associated with this event

int **msg_id**

MQTT messaged id of message

int **session_present**

MQTT session_present flag for connection event

inph_mqtt_error_codes_t ***error_handle**

esp-mqtt error handle including esp-tls errors as well as internal *MQTT* errors

bool **retain**

Retained flag of the message associated with this event

int **qos**

QoS of the messages associated with this event

bool **dup**

dup flag of the message associated with this event

inph_mqtt_protocol_ver_t **protocol_ver**

MQTT protocol version used for connection, defaults to value from menuconfig

struct **inph_mqtt_client_config_t**

#include <mqtt_client.h> *MQTT* client configuration structure

- Default values can be set via menuconfig
- All certificates and key data could be passed in PEM or DER format. PEM format must have a terminating NULL character and the related len field set to 0. DER format requires a related len field set to the correct length.

Public Members

struct *inph_mqtt_client_config_t::broker_t* **broker**

Broker address and security verification

struct *inph_mqtt_client_config_t::credentials_t* **credentials**

User credentials for broker

struct *inph_mqtt_client_config_t::session_t* **session**

MQTT session configuration.

struct *inph_mqtt_client_config_t::network_t* **network**

Network configuration

struct *inph_mqtt_client_config_t::task_t* **task**

FreeRTOS task configuration.

struct *inph_mqtt_client_config_t::buffer_t* **buffer**

Buffer size configuration.

struct *inph_mqtt_client_config_t::outbox_config_t* **outbox**

Outbox configuration.

struct **broker_t**

#include <mqtt_client.h> Broker related configuration

Public Members

struct *inph_mqtt_client_config_t::broker_t::address_t* **address**

Broker address configuration

struct *inph_mqtt_client_config_t::broker_t::verification_t* **verification**

Security verification of the broker

struct **address_t**

#include <mqtt_client.h> Broker address

- uri have precedence over other fields
- If uri isn't set at least hostname, transport and port should.

Public Members

const char ***uri**

Complete *MQTT* broker URI

const char ***hostname**

Hostname, to set ipv4 pass it as string)

inph_mqtt_transport_t **transport**

Selects transport

const char ***path**

Path in the URI

uint32_t **port**

MQTT server port

struct **verification_t**

#include <mqtt_client.h> Broker identity verification

If fields are not set broker's identity isn't verified. it's recommended to set the options in this struct for security reasons.

Public Members

bool **use_global_ca_store**

Use a global ca_store, look esp-tls documentation for details.

inph_err_t (***crt_bundle_attach**)(void *conf)

Pointer to ESP x509 Certificate Bundle attach function for the usage of certificate bundles. Client only attach the bundle, the clean up must be done by the user.

const char ***certificate**

Certificate data, default is NULL. It's not copied nor freed by the client, user needs to clean up.

size_t **certificate_len**

Length of the buffer pointed to by certificate.

const struct psk_key_hint ***psk_hint_key**

Pointer to PSK struct defined in *inph_tls.h* to enable PSK authentication (as alternative to certificate verification). PSK is enabled only if there are no other ways to verify broker. It's not copied nor freed by the client, user needs to clean up.

bool **skip_cert_common_name_check**

Skip any validation of server certificate CN field, this reduces the security of TLS and makes the *MQTT* client susceptible to MITM attacks

const char ****alpn_protos**

NULL-terminated list of supported application protocols to be used for ALPN.

const char ***common_name**

Pointer to the string containing server certificate common name. If non-NULL, server certificate CN must match this name, If NULL, server certificate CN must match hostname. This is ignored if skip_cert_common_name_check=true. It's not copied nor freed by the client, user needs to clean up.

struct **buffer_t**

#include <mqtt_client.h> Client buffer size configuration

Client have two buffers for input and output respectively.

Public Members

int **size**

size of *MQTT* send/receive buffer

int **out_size**

size of *MQTT* output buffer. If not defined, defaults to the size defined by *buffer_size*

struct **credentials_t**

#include <mqtt_client.h> Client related credentials for authentication.

Public Members

const char ***username**

MQTT username

const char ***client_id**

Set *MQTT* client identifier. Ignored if set_null_client_id == true If NULL set the default client id.

bool **set_null_client_id**

Selects a NULL client id

struct *inph_mqtt_client_config_t::credentials_t::authentication_t* **authentication**

Client authentication

struct **authentication_t**

#include <mqtt_client.h> Client authentication

Fields related to client authentication by broker

For mutual authentication using TLS, user could select certificate and key, secure element or digital signature peripheral if available.

Public Members

const char ***password**

MQTT password

const char ***certificate**

Certificate for ssl mutual authentication, not required if mutual authentication is not needed. Must be provided with `key`. It's not copied nor freed by the client, user needs to clean up.

size_t **certificate_len**

Length of the buffer pointed to by certificate.

const char ***key**

Private key for SSL mutual authentication, not required if mutual authentication is not needed. If it is not NULL, also `certificate` has to be provided. It's not copied nor freed by the client, user needs to clean up.

size_t **key_len**

Length of the buffer pointed to by key.

const char ***key_password**

Client key decryption password, not PEM nor DER, if provided `key_password_len` must be correctly set.

int **key_password_len**

Length of the password pointed to by `key_password`

bool **use_secure_element**

Enable secure element for SSL connection

void ***ds_data**

Carrier of handle for digital signature parameters, digital signature peripheral is available in some Espressif devices. It's not copied nor freed by the client, user needs to clean up.

bool **use_ecdsa_peripheral**

Enable ECDSA peripheral, available in some Espressif devices.

uint8_t **ecdsa_key_efuse_blk**

ECDSA key block number from efuse, available in some Espressif devices.

struct **network_t**

#include <mqtt_client.h> Network related configuration

Public Members

int **reconnect_timeout_ms**

Reconnect to the broker after this value in milliseconds if auto reconnect is not disabled (defaults to 10s)

int **timeout_ms**

Abort network operation if it is not completed after this value, in milliseconds (defaults to 10s).

int **refresh_connection_after_ms**

Refresh connection after this value (in milliseconds)

bool **disable_auto_reconnect**

Client will reconnect to server (when errors/disconnect). Set `disable_auto_reconnect=true` to disable

inph_transport_keep_alive_t **tcp_keep_alive_cfg**

Transport keep-alive config

inph_transport_handle_t **transport**

Custom transport handle to use, leave it NULL to allow MQTT client create or recreate its own. Warning: The transport should be valid during the client lifetime and is destroyed when `inph_mqtt_client_destroy` is called.

struct ifreq ***if_name**

The name of interface for data to go through. Use the default interface without setting

struct **outbox_config_t**

#include <mqtt_client.h> Client outbox configuration options.

Public Members

uint64_t **limit**

Size limit for the outbox in bytes.

struct **session_t**

#include <mqtt_client.h> MQTT Session related configuration

Public Members

struct *inph_mqtt_client_config_t::session_t::last_will_t* **last_will**

Last will configuration

bool **disable_clean_session**

MQTT clean session, default clean_session is true

int **keepalive**

MQTT keepalive, default is 120 seconds When configuring this value, keep in mind that the client attempts to communicate with the broker at half the interval that is actually set. This conservative approach allows for more attempts before the broker's timeout occurs

bool **disable_keepalive**

Set `disable_keepalive=true` to turn off keep-alive mechanism, keepalive is active by default. Note: setting the config value `keepalive` to 0 doesn't disable keepalive feature, but uses a default keepalive period

inph_mqtt_protocol_ver_t **protocol_ver**

MQTT protocol version used for connection.

int **message_retransmit_timeout**

timeout for retransmitting of failed packet

struct **last_will_t**

#include <mqtt_client.h> Last Will and Testament message configuration.

Public Members

const char ***topic**

LWT (Last Will and Testament) message topic

const char ***msg**

LWT message, may be NULL terminated

int **msg_len**

LWT message length, if msg isn't NULL terminated must have the correct length

int **qos**

LWT message QoS

int **retain**

LWT retained message flag

struct **task_t**

#include <mqtt_client.h> Client task configuration

Public Members

int **priority**
MQTT task priority

int **stack_size**
MQTT task stack size

struct **topic_t**
#include <mqtt_client.h> Topic definition struct

Public Members

const char ***filter**
Topic filter to subscribe

int **qos**
Max QoS level of the subscription

Typedefs

typedef struct inph_mqtt_client ***inph_mqtt5_client_handle_t**

typedef enum *mqtt5_error_reason_code_t* **inph_mqtt5_error_reason_code_t**
MQTT5 protocol error reason code, more details refer to MQTT5 protocol document section 2.4

typedef struct mqtt5_user_property_list_t ***mqtt5_user_property_handle_t**
MQTT5 user property handle

Enums

enum **mqtt5_error_reason_code_t**
MQTT5 protocol error reason code, more details refer to MQTT5 protocol document section 2.4
Values:

enumerator **MQTT5_UNSPECIFIED_ERROR**

enumerator **MQTT5_MALFORMED_PACKET**

enumerator **MQTT5_PROTOCOL_ERROR**

enumerator **MQTT5_IMPLEMENT_SPECIFIC_ERROR**

enumerator **MQTT5_UNSUPPORTED_PROTOCOL_VER**

enumerator **__attribute__**

enumerator **MQTT5_INVALID_CLIENT_ID**

enumerator **MQTT5_BAD_USERNAME_OR_PWD**

enumerator **MQTT5_NOT_AUTHORIZED**

enumerator **MQTT5_SERVER_UNAVAILABLE**

enumerator **MQTT5_SERVER_BUSY**

enumerator **MQTT5_BANNED**

enumerator **MQTT5_SERVER_SHUTTING_DOWN**

enumerator **MQTT5_BAD_AUTH_METHOD**

enumerator **MQTT5_KEEP_ALIVE_TIMEOUT**

enumerator **MQTT5_SESSION_TAKEN_OVER**

enumerator **__attribute__**

enumerator **MQTT5_TOPIC_FILTER_INVALID**

enumerator **__attribute__**

enumerator **MQTT5_TOPIC_NAME_INVALID**

enumerator **MQTT5_PACKET_IDENTIFIER_IN_USE**

enumerator **MQTT5_PACKET_IDENTIFIER_NOT_FOUND**

enumerator **MQTT5_RECEIVE_MAXIMUM_EXCEEDED**

enumerator **__attribute__**

enumerator **MQTT5_TOPIC_ALIAS_INVALID**

enumerator **MQTT5_PACKET_TOO_LARGE**

enumerator **MQTT5_MESSAGE_RATE_TOO_HIGH**

enumerator **MQTT5_QUOTA_EXCEEDED**

enumerator **MQTT5_ADMINISTRATIVE_ACTION**

enumerator **__attribute__**

enumerator **MQTT5_PAYLOAD_FORMAT_INVALID**

enumerator **MQTT5_RETAIN_NOT_SUPPORT**

enumerator **MQTT5_QOS_NOT_SUPPORT**

enumerator **MQTT5_USE_ANOTHER_SERVER**

enumerator **MQTT5_SERVER_MOVED**

enumerator **MQTT5_SHARED_SUBSCR_NOT_SUPPORTED**

enumerator **MQTT5_CONNECTION_RATE_EXCEEDED**

enumerator **MQTT5_MAXIMUM_CONNECT_TIME**

enumerator **MQTT5_SUBSCRIBE_IDENTIFIER_NOT_SUPPORT**

enumerator **MQTT5_WILDCARD_SUBSCRIBE_NOT_SUPPORT**

Functions

`inph_err_t inph_mqtt5_client_set_connect_property` (*inph_mqtt5_client_handle_t* client, const *inph_mqtt5_connection_property_config_t* *connect_property)

Set MQTT5 client connect property configuration.

Parameters

- **client** – mqtt client handle
- **connect_property** – connect property

Returns

INPH_ERR_NO_MEM if failed to allocate INPH_ERR_INVALID_ARG on wrong initialization
INPH_FAIL on fail INPH_OK on success

`inph_err_t inph_mqtt5_client_set_publish_property` (*inph_mqtt5_client_handle_t* client, const *inph_mqtt5_publish_property_config_t* *property)

Set MQTT5 client publish property configuration.

This API will not store the publish property, it is one-time configuration. Before call `inph_mqtt_client_publish` to publish data, call this API to set publish property if have

Parameters

- **client** – mqtt client handle
- **property** – publish property

Returns

INPH_ERR_INVALID_ARG on wrong initialization INPH_FAIL on fail INPH_OK on success

`inph_err_t inph_mqtt5_client_set_subscribe_property` (*inph_mqtt5_client_handle_t* client, const *inph_mqtt5_subscribe_property_config_t* *property)

Set MQTT5 client subscribe property configuration.

This API will not store the subscribe property, it is one-time configuration. Before call `inph_mqtt_client_subscribe` to subscribe topic, call this API to set subscribe property if have

Parameters

- **client** – mqtt client handle
- **property** – subscribe property

Returns

INPH_ERR_INVALID_ARG on wrong initialization INPH_FAIL on fail INPH_OK on success

`inph_err_t inph_mqtt5_client_set_unsubscribe_property` (*inph_mqtt5_client_handle_t* client, const *inph_mqtt5_unsubscribe_property_config_t* *property)

Set MQTT5 client unsubscribe property configuration.

This API will not store the unsubscribe property, it is one-time configuration. Before call `inph_mqtt_client_unsubscribe` to unsubscribe topic, call this API to set unsubscribe property if have

Parameters

- **client** – mqtt client handle
- **property** – unsubscribe property

Returns

INPH_ERR_INVALID_ARG on wrong initialization INPH_FAIL on fail INPH_OK on success

`inph_err_t inph_mqtt5_client_set_disconnect_property` (*inph_mqtt5_client_handle_t* client, const *inph_mqtt5_disconnect_property_config_t* *property)

Set MQTT5 client disconnect property configuration.

This API will not store the disconnect property, it is one-time configuration. Before call `inph_mqtt_client_disconnect` to disconnect connection, call this API to set disconnect property if have

Parameters

- **client** – mqtt client handle
- **property** – disconnect property

Returns

INPH_ERR_NO_MEM if failed to allocate INPH_ERR_INVALID_ARG on wrong initialization
 INPH_FAIL on fail INPH_OK on success

```
inph_err_t inph_mqtt5_client_set_user_property (mqtt5_user_property_handle_t *user_property,
                                              inph_mqtt5_user_property_item_t item[], uint8_t
                                              item_num)
```

Set MQTT5 client user property configuration.

This API will allocate memory for user_property, please DO NOT forget call inph_mqtt5_client_delete_user_property after you use it. Before publish data, subscribe topic, unsubscribe, etc, call this API to set user property if have

Parameters

- **user_property** – user_property handle
- **item** – array of user property data (eg. {{“var”, “val”}, {“other”, “2”}})
- **item_num** – number of items in user property data

Returns

INPH_ERR_NO_MEM if failed to allocate INPH_FAIL on fail INPH_OK on success

```
inph_err_t inph_mqtt5_client_get_user_property (mqtt5_user_property_handle_t user_property,
                                              inph_mqtt5_user_property_item_t *item, uint8_t
                                              *item_num)
```

Get MQTT5 client user property.

This API can use with inph_mqtt5_client_get_user_property_count to get list count of user property. And malloc number of count item array memory to store the user property data. Please DO NOT forget the item memory, key and value point in item memory when get user property data successfully.

Parameters

- **user_property** – user_property handle
- **item** – point that store user property data
- **item_num** – number of items in user property data

Returns

INPH_ERR_NO_MEM if failed to allocate INPH_FAIL on fail INPH_OK on success

```
uint8_t inph_mqtt5_client_get_user_property_count (mqtt5_user_property_handle_t user_property)
```

Get MQTT5 client user property list count.

Parameters

user_property – user_property handle

Returns

user property list count

```
void inph_mqtt5_client_delete_user_property (mqtt5_user_property_handle_t user_property)
```

Free the user property list.

This API will free the memory in user property list and free user_property itself

Parameters

user_property – user_property handle

struct **inph_mqtt5_connection_property_config_t**

#include <mqtt5_client.h> MQTT5 protocol connect properties and will properties configuration, more details refer to MQTT5 protocol document section 3.1.2.11 and 3.3.2.3

Public Members

uint32_t **session_expiry_interval**

The interval time of session expiry

uint32_t **maximum_packet_size**

The maximum packet size that we can receive

uint16_t **receive_maximum**

The maximum packet count that we process concurrently

uint16_t **topic_alias_maximum**

The maximum topic alias that we support

bool **request_rinph_info**

This value to request Server to return Response information

bool **request_problem_info**

This value to indicate whether the reason string or user properties are sent in case of failures

mqtt5_user_property_handle_t **user_property**

The handle for user property, call function inph_mqtt5_client_set_user_property to set it

uint32_t **will_delay_interval**

The time interval that server delays publishing will message

uint32_t **message_expiry_interval**

The time interval that message expiry

bool **payload_format_indicator**

This value is to indicator will message payload format

const char ***content_type**

This value is to indicator will message content type, use a MIME content type string

const char ***response_topic**

Topic name for a response message

const char ***correlation_data**

Binary data for receiver to match the response message

uint16_t **correlation_data_len**

The length of correlation data

mqtt5_user_property_handle_t **will_user_property**

The handle for will message user property, call function `inph_mqtt5_client_set_user_property` to set it

struct **inph_mqtt5_publish_property_config_t**

#include <mqtt5_client.h> MQTT5 protocol publish properties configuration, more details refer to MQTT5 protocol document section 3.3.2.3

Public Members

bool **payload_format_indicator**

This value is to indicator publish message payload format

uint32_t **message_expiry_interval**

The time interval that message expiry

uint16_t **topic_alias**

An integer value to identify the topic instead of using topic name string

const char ***response_topic**

Topic name for a response message

const char ***correlation_data**

Binary data for receiver to match the response message

uint16_t **correlation_data_len**

The length of correlation data

const char ***content_type**

This value is to indicator publish message content type, use a MIME content type string

mqtt5_user_property_handle_t **user_property**

The handle for user property, call function `inph_mqtt5_client_set_user_property` to set it

struct **inph_mqtt5_subscribe_property_config_t**

#include <mqtt5_client.h> MQTT5 protocol subscribe properties configuration, more details refer to MQTT5 protocol document section 3.8.2.1

Public Members

uint16_t **subscribe_id**

A variable byte represents the identifier of the subscription

bool **no_local_flag**

Subscription Option to allow that server publish message that client sent

bool **retain_as_published_flag**

Subscription Option to keep the retain flag as published option

uint8_t **retain_handle**

Subscription Option to handle retain option

bool **is_share_subscribe**

Whether subscribe is a shared subscription

const char ***share_name**

The name of shared subscription which is a part of \$share/{share_name}/{topic}

mqtt5_user_property_handle_t **user_property**

The handle for user property, call function `inph_mqtt5_client_set_user_property` to set it

struct **inph_mqtt5_unsubscribe_property_config_t**

#include <mqtt5_client.h> MQTT5 protocol unsubscribe properties configuration, more details refer to MQTT5 protocol document section 3.10.2.1

Public Members

bool **is_share_subscribe**

Whether subscribe is a shared subscription

const char ***share_name**

The name of shared subscription which is a part of \$share/{share_name}/{topic}

mqtt5_user_property_handle_t **user_property**

The handle for user property, call function `inph_mqtt5_client_set_user_property` to set it

struct **inph_mqtt5_disconnect_property_config_t**

#include <mqtt5_client.h> MQTT5 protocol disconnect properties configuration, more details refer to MQTT5 protocol document section 3.14.2.2

Public Members

uint32_t **session_expiry_interval**

The interval time of session expiry

uint8_t **disconnect_reason**

The reason that connection disconnect, refer to mqtt5_error_reason_code

mqtt5_user_property_handle_t **user_property**

The handle for user property, call function `inph_mqtt5_client_set_user_property` to set it

struct **inph_mqtt5_event_property_t**

#include <mqtt5_client.h> MQTT5 protocol for event properties

Public Members

bool **payload_format_indicator**

Payload format of the message

char ***response_topic**

Response topic of the message

int **response_topic_len**

Response topic length of the message

char ***correlation_data**

Correlation data of the message

uint16_t **correlation_data_len**

Correlation data length of the message

char ***content_type**

Content type of the message

int **content_type_len**

Content type length of the message

uint16_t **subscribe_id**

Subscription identifier of the message

mqtt5_user_property_handle_t **user_property**

The handle for user property, call function `inph_mqtt5_client_delete_user_property` to free the memory

struct **inph_mqtt5_user_property_item_t**

#include <mqtt5_client.h> MQTT5 protocol for user property

Public Members

const char ***key**

Item key name

const char ***value**

Item value string

2.1.2.2 SNTP

Example: using LwIP SNTP module and time functions

This example demonstrates the use of LwIP SNTP module to obtain time from Internet servers. See the README.md file in the upper level 'examples' directory for more information about examples.

Configuring the Example

Open the project configuration menu (*idf.py menuconfig*):

- Configure Wi-Fi or Ethernet or Thread under “Example Connection Configuration” menu. See “Establishing Wi-Fi or Ethernet or Thread Connection” section in [examples/protocols/README.md](../README.md) for more details.
- Select one method to synchronize time out of the three available in *CONFIG_SNTP_TIME_SYNC_METHOD* (default *update time immediately when received*).
- The time synchronization period is defined by *CONFIG_LWIP_SNTP_UPDATE_DELAY*. This option allows you to set the time synchronization period (default is one hour). This option does not affect this example because after synchronization the program goes into deep sleep for 10 seconds. If you modify this example or use the code from this example, keep in mind that this option will trigger time synchronization. You can change it in *Component config->LWIP->SNTP->Request interval to update time (ms)* menu.

Obtaining time using LwIP SNTP module

When this example boots first time after INPH32 is reset, it connects to WiFi and obtains time using SNTP. See *initialize_sntp* function for details.

Obtaining time using LwIP SNTP-over-DHCP module

NTP server addresses could be automatically acquired via DHCP server option 42. This could be useful on closed environments where public NTPs are not accessible or to prefer local servers and reduce traffic to the outer world. See following menuconfig options:

- *Component config->LWIP->SNTP->Maximum number of NTP servers*
- *Component config->LWIP->SNTP->Request NTP servers from DHCP*
- *Component config->LWIP->SNTP->Maximum number of NTP servers acquired via DHCP*
- *Component config->LWIP->Enable LWIP Debug->Enable SNTP debug messages*

Please note, that *dhcp_set_ntp_servers()* does not only set NTP servers provided by DHCP, but also resets all other NTP server configured before. If you want to keep both manually configured and DHCP obtained NTP servers, please use the API in this order: - Enable SNTP-over-DHCP before getting the IP using *sntp_servermode_dhcp()* - Set the static NTP servers after receiving the DHCP lease using *sntp_setserver()*

Timekeeping

Once time is synchronized, INPH32 will perform timekeeping using built-in timers.

- RTC clock is used to maintain accurate time when chip is in deep sleep mode

- High-resolution timer is used to provide time at microsecond accuracy when INPH32 is running.

Timekeeping using RTC timer is demonstrated in this example by going into deep sleep mode. After wake up, INPH32 will print current time without connecting to WiFi.

To use this functionality, make sure “Timers used for gettimeofday function” option in “INPH32-specific config” menu of menuconfig is set to “RTC and high-resolution timer” or “RTC”.

Working with time

To get current time, `[gettimeofday]`(<http://man7.org/linux/man-pages/man2/gettimeofday.2.html>) function may be used. Additionally the following [standard C library functions](<https://en.cppreference.com/w/cpp/header/ctime>) can be used to obtain time and manipulate it:

`gettimeofday time asctime clock ctime difftime gmtime localtime mktime strftime`

To set time, `[settimeofday]`(<http://man7.org/linux/man-pages/man2/settimeofday.2.html>) POSIX function can be used. It is used internally in LwIP SNTP library to set current time when response from NTP server is received.

Timezones

-To set local timezone, use `[utc time]` add/subtract the corresponding time zone difference.

-For the current newlibc-nano, if the TZ environment variable has been set previously, there will be issues with `['tzset']`. So, avoid using `['setenv']` and `['tzset']`.

Additional options

This example can use 3 time synchronization method:

- *update time immediately when received* - update time immediately as received an answer from SNTP server. Allows syncing the time without any additional code from the user side, and use a callback function or getting status for notification of the process of sync.
- *update time with smooth method (adjtime)* - time synchronization will use the `adjtime` function to smoothly update the time. Allows syncing the time without any additional code from the user side, and use a callback function or getting status for notification of the process of sync.
- *custom implementation* - allows replacing the built-in time synchronization functionality. This option redefines `sntp_sync_time()` function.

Useful API functions

- `sntp_set_time_sync_notification_cb()` - use this function to set a callback function to notify about the time synchronization process.
- `sntp_get_sync_status()` and `sntp_set_sync_status()` - get/set time synchronization status.
- `sntp_get_sync_mode()` and `sntp_set_sync_mode()` - get/set the sync mode. Allowable two mode: `SNTP_SYNC_MODE_IMMED` and `SNTP_SYNC_MODE_SMOOTH`.

Mode of update time

`sntp_set_sync_mode()` - Set the sync mode of the system time. It has two mode:

- `SNTP_SYNC_MODE_IMMED` - Update system time immediately when receiving a response from the SNTP server.
- `SNTP_SYNC_MODE_SMOOTH` - Smooth time updating. Time error is gradually reduced using `adjtime` function. If the difference between SNTP response time and system time is large (more than 35 minutes) then update immediately. This mode uses `adjtime()` function.

Adjtime() `int adjtime(const struct timeval *delta, struct timeval *outdelta)`

`adjtime()` is a libc function that is called automatically in “smooth” time update mode, but can also be called from custom time synchronization code. If the time error is less than 35 minutes then `adjtime` function will start smooth adjusting otherwise the return value is -1.

This function speeds up or slows down the system clock in order to make a gradual adjustment. This ensures that the calendar time reported by the system clock is always monotonically increasing, which might not happen if you simply set the clock. If adjusting the system clock by *adjtime()* is already done during the second call *adjtime()*, and the delta of the second call is not NULL, the earlier tuning is stopped, but the already completed part of the adjustment is not canceled.

The delta argument specifies a relative adjustment to be made to the clock time. If negative, the system clock is slowed down for a while until it has lost this much elapsed time. If positive, the system clock is speeded up for a while.

If the *olddelta* argument is not a null pointer, the *adjtime* function returns information about any previous time adjustment that has not yet completed.

The return value is 0 on success and -1 on failure.

To stop the smooth time adjustment, you need to record the current time using the function *settimeofday()*.

Example Output

```
I (269) wifi_init: create WIFI interface: MAC f0:f3:f4:8c:fb:22
I (269) wifi_init: rx ba win: 6
I (269) wifi_init: accept mbox: 6
I (269) wifi_init: tcpip mbox: 32
I (269) wifi_init: udp mbox: 6
I (270) wifi_init: tcp mbox: 6
I (270) wifi_init: tcp tx win: 5760
I (270) wifi_init: tcp rx win: 5760
I (271) wifi_init: tcp mss: 1440
I (271) wifi_init: WiFi IRAM OP enabled
I (272) wifi_init: WiFi RX IRAM OP enabled
I (273) wifi_init_default: WIFI mac address: f0 f3 f4 8c fb 22
W (274) inph_netif_lwip: CONFIG_LWIP_INPH_GRATUITOUS_ARP not enabled, but
↳inph_netif configured with INPH_NETIF_FLAG_GARP
I (275) example: wifi_init_sta finished.
I (10604) inph_netif_handlers: sta ip: 192.168.31.127, mask: 255.255.255.0,
↳gw: 192.168.31.1
I (10604) example: got ip:192.168.31.127
I (10605) example: Time is not set yet. Connecting to WiFi and getting time
↳over NTP.
I (10605) example: Initializing and starting SNTP
I (10606) example: List of configured NTP servers:
I (10607) example: server 0: pool.ntp.org
I (12607) example: Waiting for system time to be set... (1/15)
I (14607) example: Waiting for system time to be set... (2/15)
I (16607) example: Waiting for system time to be set... (3/15)
I (18607) example: Waiting for system time to be set... (4/15)
I (18637) example: Notification of a time synchronization event
I (18638) example: SNTP synchronized: 2026-01-05 09:04:31 UTC
I (18638) example: Local time (Beijing): 2026-01-05 17:04:31
I (18638) example: New York time: 2026-01-05 04:04:31
I (18639) example: London time: 2026-01-05 09:04:31
I (18639) example: APU free memory: 226 KiB
```

API Reference

struct **inph_sntp_config**

#include <inph_netif_sntp.h> SNTP configuration struct.

Public Members

bool **smooth_sync**
set to true if smooth sync required

bool **server_from_dhcp**
set to true to request NTP server config from DHCP

bool **wait_for_sync**
if true, we create a semaphore to signal time sync event

bool **start**
set to true to automatically start the SNTP service

inph_sntp_time_cb_t **sync_cb**
optionally sets callback function on time sync event

bool **renew_servers_after_new_IP**
this is used to refresh server list if NTP provided by DHCP (which cleans other pre-configured servers)

ip_event_t **ip_event_to_renew**
set the IP event id on which we refresh server list (if renew_servers_after_new_IP=true)

size_t **index_of_first_server**
refresh server list after this server (if renew_servers_after_new_IP=true)

size_t **num_of_servers**
number of preconfigured NTP servers

const char ***servers**[CONFIG_LWIP_SNTP_MAX_SERVERS]
list of servers

Defines

inph_sntp_sync_time
Aliases for inph_sntp prefixed API (inherently thread safe)

inph_sntp_set_sync_mode

inph_sntp_get_sync_mode

inph_sntp_get_sync_status

inph_sntp_set_sync_status

`inph_sntp_set_time_sync_notification_cb`

`inph_sntp_set_sync_interval`

`inph_sntp_get_sync_interval`

`inph_sntp_restart`

`SNTP_OPMODE_POLL`

Typedefs

`typedef void (*sntp_sync_time_cb_t)(struct timeval *tv)`
SNTP callback function for notifying about time sync event.

Param tv

Time received from SNTP server.

Enums

`enum sntp_sync_mode_t`
SNTP time update mode.

Values:

enumerator `SNTP_SYNC_MODE_IMMED`

Update system time immediately when receiving a response from the SNTP server.

enumerator `SNTP_SYNC_MODE_SMOOTH`

Smooth time updating. Time error is gradually reduced using `adjtime` function. If the difference between SNTP response time and system time is large (more than 35 minutes) then update immediately.

`enum sntp_sync_status_t`
SNTP sync status.

Values:

enumerator `SNTP_SYNC_STATUS_RESET`

enumerator `SNTP_SYNC_STATUS_COMPLETED`

enumerator `SNTP_SYNC_STATUS_IN_PROGRESS`

enum **inph_sntp_operatingmode_t**

SNTP operating modes per lwip SNTP module.

Values:

enumerator **INPH_SNTP_OPMODE_POLL**

enumerator **INPH_SNTP_OPMODE_LISTENONLY**

Functions

void **sntp_sync_time** (struct timeval *tv)

This function updates the system time.

This is a weak-linked function. It is possible to replace all SNTP update functionality by placing a `sntp_sync_time()` function in the app firmware source. If the default implementation is used, calling `sntp_set_sync_mode()` allows the time synchronization mode to be changed to instant or smooth. If a callback function is registered via `sntp_set_time_sync_notification_cb()`, it will be called following time synchronization.

Parameters

tv – Time received from SNTP server.

void **sntp_set_sync_mode** (*sntp_sync_mode_t* sync_mode)

Set the sync mode.

Modes allowed: **SNTP_SYNC_MODE_IMMED** and **SNTP_SYNC_MODE_SMOOTH**.

Parameters

sync_mode – Sync mode.

sntp_sync_mode_t **sntp_get_sync_mode** (void)

Get set sync mode.

Returns

SNTP_SYNC_MODE_IMMED: Update time immediately. **SNTP_SYNC_MODE_SMOOTH**: Smooth time updating.

sntp_sync_status_t **sntp_get_sync_status** (void)

Get status of time sync.

After the update is completed, the status will be returned as **SNTP_SYNC_STATUS_COMPLETED**. After that, the status will be reset to **SNTP_SYNC_STATUS_RESET**. If the update operation is not completed yet, the status will be **SNTP_SYNC_STATUS_RESET**. If a smooth mode was chosen and the synchronization is still continuing (adjtime works), then it will be **SNTP_SYNC_STATUS_IN_PROGRESS**.

Returns

SNTP_SYNC_STATUS_RESET: Reset status. **SNTP_SYNC_STATUS_COMPLETED**: Time is synchronized. **SNTP_SYNC_STATUS_IN_PROGRESS**: Smooth time sync in progress.

void **sntp_set_sync_status** (*sntp_sync_status_t* sync_status)

Set status of time sync.

Parameters

sync_status – status of time sync (see `sntp_sync_status_t`)

void **sntp_set_time_sync_notification_cb** (*sntp_sync_time_cb_t* callback)

Set a callback function for time synchronization notification.

Parameters

callback – a callback function

void **sntp_set_sync_interval** (uint32_t interval_ms)

Set the sync interval of SNTP operation.

Note: SNTPv4 RFC 4330 enforces a minimum sync interval of 15 seconds. This sync interval will be used in the next attempt update time through SNTP. To apply the new sync interval call the sntp_restart() function, otherwise, it will be applied after the last interval expired.

Parameters

interval_ms – The sync interval in ms. It cannot be lower than 15 seconds, otherwise 15 seconds will be set.

uint32_t **sntp_get_sync_interval** (void)

Get the sync interval of SNTP operation.

Returns

the sync interval

bool **sntp_restart** (void)

Restart SNTP.

Returns

True - Restart False - SNTP was not initialized yet

void **inph_sntp_setoperatingmode** (*inph_sntp_operatingmode_t* operating_mode)

Sets SNTP operating mode. The mode has to be set before init.

Parameters

operating_mode – Desired operating mode

void **inph_sntp_init** (void)

Init and start SNTP service.

void **inph_sntp_stop** (void)

Stops SNTP service.

void **inph_sntp_setserver** (u8_t idx, const ip_addr_t *addr)

Sets SNTP server address.

Parameters

- **idx** – Index of the server
- **addr** – IP address of the server

void **inph_sntp_setservername** (u8_t idx, const char *server)

Sets SNTP hostname.

Parameters

- **idx** – Index of the server
- **server** – Name of the server

const char ***inph_sntp_getservername** (u8_t idx)

Gets SNTP server name.

Parameters**idx** – Index of the server**Returns**

Name of the server

```
const ip_addr_t *inph_sntp_getserver (u8_t idx)
```

Get SNTP server IP.

Parameters**idx** – Index of the server**Returns**

IP address of the server

```
bool inph_sntp_enabled (void)
```

Checks if sntp is enabled.

Returns

true if sntp module is enabled

```
uint8_t inph_sntp_getreachability (uint8_t idx)
```

Gets the server reachability shift register as described in RFC 5905.

Parameters**idx** – Index of the SNTP server**Returns**

reachability shift register

```
inph_sntp_operatingmode_t inph_sntp_getoperatingmode (void)
```

Get the configured operating mode.

Returns

operating mode enum

```
static inline static __attribute__ ((deprecated("use inph_sntp_setoperatingmode() instead"))
```

if not build within lwip, provide translating inlines, that will warn about thread safety

```
static inline static __attribute__ ((deprecated("use inph_sntp_servermode_dhcp() instead"))
```

```
static inline static __attribute__ ((deprecated("use inph_sntp_setservername() instead"))
```

```
static inline static __attribute__ ((deprecated("use inph_sntp_init() instead")) void sntp
```

```
static inline static __attribute__ ((deprecated("use inph_sntp_getservername() instead"))
```

```
static inline static __attribute__ ((deprecated("use inph_sntp_getserver() instead")) cons
```

```
static inline static __attribute__ ((deprecated("use inph_sntp_getreachability() instead"))
```

```
static inline static __attribute__ ((deprecated("use inph_sntp_getoperatingmode() instead"))
```

Variables

```
const char * server {inph_sntp_setservername(idx, server)}
```

2.1.2.3 HTTP Client

Overview

`esp_http_client` component provides a set of APIs for making HTTP/S requests from ESP-IDF applications. The steps to use these APIs are as follows:

- `esp_http_client_init()`: Creates an `esp_http_client_handle_t` instance, i.e., an HTTP client handle based on the given `esp_http_client_config_t` configuration. This function must be the first to be called; default values are assumed for the configuration values that are not explicitly defined by the user.
- `esp_http_client_perform()`: Performs all operations of the `esp_http_client` - opening the connection, exchanging data, and closing the connection (as required), while blocking the current task before its completion. All related events are invoked through the event handler (as specified in `esp_http_client_config_t`).
- `esp_http_client_cleanup()`: Closes the connection (if any) and frees up all the memory allocated to the HTTP client instance. This must be the last function to be called after the completion of operations.

Basic HTTP Request

Check out the example functions `http_rest_with_url` and `http_rest_with_hostname_path` in the application example for implementation details.

Persistent Connections

Persistent connection means that the HTTP client can reuse the same connection for several exchanges. If the server does not request to close the connection with the `Connection: close` header, the connection is not dropped but is instead kept open and used for further requests.

To allow ESP HTTP client to take full advantage of persistent connections, one should make as many requests as possible using the same handle instance. Check out the example functions `http_rest_with_url` and `http_rest_with_hostname_path` in the application example. Here, once the connection is created, multiple requests (GET, POST, PUT, etc.) are made before the connection is closed.

Use Secure Element (ATECC608) for TLS

A secure element (ATECC608) can be also used for the underlying TLS connection in the HTTP client connection. Please refer to the **ATECC608A (Secure Element) with ESP-TLS** section in the ESP-TLS documentation for more details. The secure element support has to be first enabled in menuconfig through `CONFIG_ESP_TLS_USE_SECURE_ELEMENT`. Then the HTTP client can be configured to use secure element as follows:

```
esp_http_client_config_t cfg = {
    /* other configurations options */
    .use_secure_element = true,
};
```

HTTPS Request

ESP HTTP client supports SSL connections using **mbedTLS**, with the `url` configuration starting with `https` scheme or `transport_type` set to `HTTP_TRANSPORT_OVER_SSL`. HTTPS support can be configured via `CONFIG_ESP_HTTP_CLIENT_ENABLE_HTTPS` (enabled by default).

Note: While making HTTPS requests, if server verification is needed, an additional root certificate (in PEM format) needs to be provided to the `cert_pem` member in the `esp_http_client_config_t` configuration. Users can also use the ESP x509 Certificate Bundle for server verification using the `crt_bundle_attach` member of the `esp_http_client_config_t` configuration.

Check out the example functions `https_with_url` and `https_with_hostname_path` in the application example for implementation details of the above note.

HTTP Stream

Some applications need to open the connection and control the exchange of data actively (data streaming). In such cases, the application flow is different from regular requests. Example flow is given below:

- `esp_http_client_init()`: Create a HTTP client handle.
- `esp_http_client_set_*` or `esp_http_client_delete_*`: Modify the HTTP connection parameters (optional).
- `esp_http_client_open()`: Open the HTTP connection with `write_len` parameter (content length that needs to be written to server), set `write_len=0` for read-only connection.
- `esp_http_client_write()`: Write data to server with a maximum length equal to `write_len` of `esp_http_client_open()` function; no need to call this function for `write_len=0`.
- `esp_http_client_fetch_headers()`: Read the HTTP Server response headers, after sending the request headers and server data (if any). Returns the `content-length` from the server and can be succeeded by `esp_http_client_get_status_code()` for getting the HTTP status of the connection.
- `esp_http_client_read()`: Read the HTTP stream.
- `esp_http_client_close()`: Close the connection.
- `esp_http_client_cleanup()`: Release allocated resources.

Check out the example function `http_perform_as_stream_reader` in the application example for implementation details.

HTTP Authentication

ESP HTTP client supports both Basic and Digest Authentication.

- Users can provide the username and password in the `url` or the `username` and `password` members of the `esp_http_client_config_t` configuration. For `auth_type = HTTP_AUTH_TYPE_BASIC`, the HTTP client takes only one perform operation to pass the authentication process.
- If `auth_type = HTTP_AUTH_TYPE_NONE`, but the `username` and `password` fields are present in the configuration, the HTTP client takes two perform operations. The client will receive the 401 Unauthorized header in its first attempt to connect to the server. Based on this information, it decides which authentication method to choose and performs it in the second operation.
- Check out the example functions `http_auth_basic`, `http_auth_basic_redirect` (for Basic authentication) and `http_auth_digest` (for Digest authentication) in the application example for implementation details.
- Currently, Digest authentication supports only MD5 and SHA-256 algorithms.

Examples of Authentication Configuration

- Authentication with URI


```
esp_http_client_config_t config = {
    .url = "http://user:passwd@httpbin.org/basic-auth/user/passwd",
    .auth_type = HTTP_AUTH_TYPE_BASIC,
};
```

- Authentication with username and password entry

```
esp_http_client_config_t config = {
    .url = "http://httpbin.org/basic-auth/user/passwd",
    .username = "user",
    .password = "passwd",
    .auth_type = HTTP_AUTH_TYPE_BASIC,
};
```

Event Handling

ESP HTTP Client supports event handling by triggering an event handler corresponding to the event which takes place. `esp_http_client_event_id_t` contains all the events which could occur while performing an HTTP request using the ESP HTTP Client.

To enable event handling, you just need to set a callback function using the `esp_http_client_config_t::event_handler` member.

ESP HTTP Client Diagnostic Information

Diagnostic information could be helpful to gain insights into a problem. In the case of ESP HTTP Client, the diagnostic information can be collected by registering an event handler with the Event Loop library. This feature has been added by keeping in mind the **ESP Insights** framework which collects the diagnostic information. However, this feature can also be used without any dependency on the ESP Insights framework for the diagnostic purpose. Event handler can be registered to the event loop using the `esp_event_handler_register()` function.

Expected data types for different HTTP Client events in the event loop are as follows:

- `HTTP_EVENT_ERROR`: `esp_http_client_handle_t`
- `HTTP_EVENT_ON_CONNECTED`: `esp_http_client_handle_t`
- `HTTP_EVENT_HEADERS_SENT`: `esp_http_client_handle_t`
- `HTTP_EVENT_ON_HEADER`: `esp_http_client_handle_t`
- `HTTP_EVENT_ON_HEADERS_COMPLETE`: `esp_http_client_handle_t`
- `HTTP_EVENT_ON_STATUS_CODE`: `esp_http_client_handle_t`
- `HTTP_EVENT_ON_DATA`: `esp_http_client_on_data_t`
- `HTTP_EVENT_ON_FINISH`: `esp_http_client_handle_t`
- `HTTP_EVENT_DISCONNECTED`: `esp_http_client_handle_t`
- `HTTP_EVENT_REDIRECT`: `esp_http_client_redirect_event_data_t`

The `esp_http_client_handle_t` received along with the event data will be valid until `HTTP_EVENT_DISCONNECTED` is not received. This handle has been sent primarily to differentiate between different client connections and must not be used for any other purpose, as it may change based on client connection state.

TLS Protocol Version

TLS protocol version to be used for the underlying TLS connection can be set in `esp_http_client_config_t`. Please refer to the **TLS Protocol Version** section in the `/api-reference/protocols/esp_tls` for more details.

The TLS protocol version for the HTTP client can be configured as follows:

```
#include "esp_http_client.h"
esp_http_client_config_t config = {
    .tls_version = ESP_HTTP_CLIENT_TLS_VER_TLS_1_2,
};
```

API Reference

Defines

DEFAULT_HTTP_BUF_SIZE

ESP_ERR_HTTP_BASE

Starting number of HTTP error codes

ESP_ERR_HTTP_MAX_REDIRECT

The error exceeds the number of HTTP redirects

ESP_ERR_HTTP_CONNECT

Error open the HTTP connection

ESP_ERR_HTTP_WRITE_DATA

Error write HTTP data

ESP_ERR_HTTP_FETCH_HEADER

Error read HTTP header from server

ESP_ERR_HTTP_INVALID_TRANSPORT

There are no transport support for the input scheme

ESP_ERR_HTTP_CONNECTING

HTTP connection hasn't been established yet

ESP_ERR_HTTP_EAGAIN

Mapping of errno EAGAIN to inph_err_t

ESP_ERR_HTTP_CONNECTION_CLOSED

Read FIN from peer and the connection closed

ESP_ERR_HTTP_NOT_MODIFIED

HTTP 304 Not Modified, no update available

ESP_ERR_HTTP_RANGE_NOT_SATISFIABLE

HTTP 416 Range Not Satisfiable, requested range in header is incorrect

ESP_ERR_HTTP_READ_TIMEOUT

HTTP data read timeout

ESP_ERR_HTTP_INCOMPLETE_DATA

Incomplete data received, less than Content-Length or last chunk

Typedefs

typedef struct esp_http_client ***esp_http_client_handle_t**

typedef struct *esp_http_client_event* ***esp_http_client_event_handle_t**

typedef struct *esp_http_client_event* **esp_http_client_event_t**

HTTP Client events data.

typedef struct *esp_http_client_on_data* **esp_http_client_on_data_t**

Argument structure for HTTP_EVENT_ON_DATA event.

typedef struct *esp_http_client_redirect_event_data* **esp_http_client_redirect_event_data_t**

Argument structure for HTTP_EVENT_REDIRECT event.

typedef inph_err_t (***http_event_handle_cb**)(*esp_http_client_event_t* *evt)

Enums

enum **esp_http_client_event_id_t**

HTTP Client events id.

Values:

enumerator **HTTP_EVENT_ERROR**

This event occurs when there are any errors during execution

enumerator **HTTP_EVENT_ON_CONNECTED**

Once the HTTP has been connected to the server, no data exchange has been performed

enumerator **HTTP_EVENT_HEADERS_SENT**

After sending all the headers to the server

enumerator **HTTP_EVENT_HEADER_SENT**

This header has been kept for backward compatibility and will be deprecated in future versions esp-idf

enumerator **HTTP_EVENT_ON_HEADER**

Occurs when receiving each header sent from the server

enumerator **HTTP_EVENT_ON_HEADERS_COMPLETE**

Occurs when all headers are received on the client side

enumerator **HTTP_EVENT_ON_STATUS_CODE**

Occurs when receiving the HTTP status code from the server

enumerator **HTTP_EVENT_ON_DATA**

Occurs when receiving data from the server, possibly multiple portions of the packet

enumerator **HTTP_EVENT_ON_FINISH**

Occurs when complete data is received

enumerator **HTTP_EVENT_DISCONNECTED**

The connection has been disconnected

enumerator **HTTP_EVENT_REDIRECT**

Intercepting HTTP redirects to handle them manually

enum **esp_http_client_transport_t**

HTTP Client transport.

Values:

enumerator **HTTP_TRANSPORT_UNKNOWN**

Unknown

enumerator **HTTP_TRANSPORT_OVER_TCP**

Transport over tcp

enumerator **HTTP_TRANSPORT_OVER_SSL**

Transport over ssl

enum **esp_http_client_proto_ver_t**

Values:

enumerator **ESP_HTTP_CLIENT_TLS_VER_ANY**

enumerator **ESP_HTTP_CLIENT_TLS_VER_TLS_1_2**

enumerator **ESP_HTTP_CLIENT_TLS_VER_TLS_1_3**

enumerator **ESP_HTTP_CLIENT_TLS_VER_MAX**

enum **esp_http_client_ecdsa_curve_t**

ECDSA curve options for TLS connections.

Values:

enumerator **ESP_HTTP_CLIENT_ECDSA_CURVE_SECP256R1**

Use SECP256R1 curve

enumerator **ESP_HTTP_CLIENT_ECDSA_CURVE_MAX**

to indicate max

enum **esp_http_client_method_t**

HTTP method.

Values:

enumerator **HTTP_METHOD_GET**

HTTP GET Method

enumerator **HTTP_METHOD_POST**

HTTP POST Method

enumerator **HTTP_METHOD_PUT**

HTTP PUT Method

enumerator **HTTP_METHOD_PATCH**

HTTP PATCH Method

enumerator **HTTP_METHOD_DELETE**

HTTP DELETE Method

enumerator **HTTP_METHOD_HEAD**

HTTP HEAD Method

enumerator **HTTP_METHOD_NOTIFY**

HTTP NOTIFY Method

enumerator **HTTP_METHOD_SUBSCRIBE**

HTTP SUBSCRIBE Method

enumerator **HTTP_METHOD_UNSUBSCRIBE**

HTTP UNSUBSCRIBE Method

enumerator **HTTP_METHOD_OPTIONS**

HTTP OPTIONS Method

enumerator **HTTP_METHOD_COPY**

HTTP COPY Method

enumerator **HTTP_METHOD_MOVE**

HTTP MOVE Method

enumerator **HTTP_METHOD_LOCK**

HTTP LOCK Method

enumerator **HTTP_METHOD_UNLOCK**

HTTP UNLOCK Method

enumerator **HTTP_METHOD_PROPFIND**

HTTP PROPFIND Method

enumerator **HTTP_METHOD_PROPPATCH**

HTTP PROPPATCH Method

enumerator **HTTP_METHOD_MKCOL**

HTTP MKCOL Method

enumerator **HTTP_METHOD_REPORT**

HTTP REPORT Method

enumerator **HTTP_METHOD_MAX**

enum **esp_http_client_auth_type_t**

HTTP Authentication type.

Values:

enumerator **HTTP_AUTH_TYPE_NONE**

No authentication

enumerator **HTTP_AUTH_TYPE_BASIC**

HTTP Basic authentication

enumerator **HTTP_AUTH_TYPE_DIGEST**

HTTP Digest authentication

enum **esp_http_client_addr_type_t**

Values:

enumerator **HTTP_ADDR_TYPE_UNSPEC**

Unspecified address family.

enumerator **HTTP_ADDR_TYPE_INET**

IPv4 address family.

enumerator **HTTP_ADDR_TYPE_INET6**

IPv6 address family.

enum **esp_http_client_tls_dyn_buf_strategy_t**

Values:

enumerator **HTTP_TLS_DYN_BUF_RX_STATIC**

Strategy to disable dynamic RX buffer allocations and convert to static allocation post-handshake, reducing memory fragmentation

enumerator **HTTP_TLS_DYN_BUF_STRATEGY_MAX**

to indicate max

enum **esp_http_state_t**

HTTP Client states.

Values:

enumerator **HTTP_STATE_UNINIT**

HTTP client uninitialized

enumerator **HTTP_STATE_INIT**

HTTP client initialized

enumerator **HTTP_STATE_CONNECTING**

HTTP client connecting to server

enumerator **HTTP_STATE_CONNECTED**

HTTP client connected to server

enumerator **HTTP_STATE_REQ_COMPLETE_HEADER**

HTTP request headers sent

enumerator **HTTP_STATE_REQ_COMPLETE_DATA**

HTTP request data sent

enumerator **HTTP_STATE_RES_COMPLETE_HEADER**

HTTP response headers received

enumerator **HTTP_STATE_RES_ON_DATA_START**

HTTP response data started

enumerator **HTTP_STATE_RES_COMPLETE_DATA**

HTTP response data completed

enumerator **HTTP_STATE_CLOSE**

HTTP client connection closed

enum **HttpStatus_Code**

Enum for the HTTP status codes.

Values:

enumerator **HttpStatus_Ok**

enumerator **HttpStatus_PartialContent**

enumerator **HttpStatus_MultipleChoices**

enumerator **HttpStatus_MovedPermanently**

enumerator **HttpStatus_Found**

enumerator **HttpStatus_SeeOther**

enumerator **HttpStatus_NotModified**

enumerator **HttpStatus_TemporaryRedirect**

enumerator **HttpStatus_PermanentRedirect**

enumerator **HttpStatus_BadRequest**

enumerator **HttpStatus_Unauthorized**

enumerator **HttpStatus_Forbidden**

enumerator **HttpStatus_NotFound**

enumerator **HttpStatus_RangeNotSatisfiable**

enumerator **HttpStatus_InternalError**

Functions

INPH_EVENT_DECLARE_BASE (ESP_HTTP_CLIENT_EVENT)

esp_http_client_handle_t **esp_http_client_init** (const *esp_http_client_config_t* *config)

Start a HTTP session This function must be the first function to call, and it returns a *esp_http_client_handle_t* that you must use as input to other functions in the interface. This call **MUST** have a corresponding call to *esp_http_client_cleanup* when the operation is complete.

Parameters

config – [in] The configurations, see *http_client_config_t*

Returns

- `esp_http_client_handle_t`
- NULL if any errors

`inph_err_t esp_http_client_perform(esp_http_client_handle_t client)`

Invoke this function after `esp_http_client_init` and all the options calls are made, and will perform the transfer as described in the options. It must be called with the same `esp_http_client_handle_t` as input as the `esp_http_client_init` call returned. `esp_http_client_perform` performs the entire request in either blocking or non-blocking manner. By default, the API performs request in a blocking manner and returns when done, or if it failed, and in non-blocking manner, it returns if EAGAIN/EWOULDBLOCK or EINPROGRESS is encountered, or if it failed. And in case of non-blocking request, the user may call this API multiple times unless request & response is complete or there is a failure. To enable non-blocking `esp_http_client_perform()`, `is_async` member of `esp_http_client_config_t` must be set while making a call to `esp_http_client_init()` API. You can do any amount of calls to `esp_http_client_perform` while using the same `esp_http_client_handle_t`. The underlying connection may be kept open if the server allows it. If you intend to transfer more than one file, you are even encouraged to do so. `esp_http_client` will then attempt to reuse the same connection for the following transfers, thus making the operations faster, less CPU intense and using less network resources. Just note that you will have to use `esp_http_client_set_*` between the invokes to set options for the following `esp_http_client_perform`.

Note: You must never call this function simultaneously from two places using the same client handle. Let the function return first before invoking it another time. If you want parallel transfers, you must use several `esp_http_client_handle_t`. This function include `esp_http_client_open` -> `esp_http_client_write` -> `esp_http_client_fetch_headers` -> `esp_http_client_read` (and option) `esp_http_client_close`.

Parameters

client – The `esp_http_client` handle

Returns

- ESP_OK on successful
- ESP_FAIL on error
- ESP_ERR_HTTP_CONNECTING is timed-out before connection is made
- ESP_ERR_HTTP_WRITE_DATA is timed-out before request fully sent
- ESP_ERR_HTTP_EAGAIN is timed-out before any data was ready
- ESP_ERR_HTTP_READ_TIMEOUT if read operation times out
- ESP_ERR_HTTP_INCOMPLETE_DATA if read operation returns less data than expected
- ESP_ERR_HTTP_CONNECTION_CLOSED if server closes the connection

`inph_err_t esp_http_client_prepare(esp_http_client_handle_t client)`

Prepare HTTP client for a new request This function initializes the client state and prepares authentication if needed. It should be called before sending a request.

Parameters

client – [in] The `esp_http_client` handle

Returns

- ESP_OK on successful
- ESP_FAIL on error

`inph_err_t esp_http_client_request_send(esp_http_client_handle_t client, int write_len)`

Send HTTP request headers and data. This function sends the HTTP request line, headers, and any post data to the server.

Parameters

- **client** – [in] The `esp_http_client` handle
- **write_len** – [in] Length of data to write (for POST/PUT requests)

Returns

- ESP_OK on successful
- ESP_FAIL on error
- ESP_ERR_HTTP_WRITE_DATA if write operation fails

`inph_err_t esp_http_client_cancel_request(esp_http_client_handle_t client)`

Cancel an ongoing HTTP request. This API closes the current socket and opens a new socket with the same `esp_http_client` context.

Parameters

client – The `esp_http_client` handle

Returns

- ESP_OK on successful
- ESP_FAIL on error
- ESP_ERR_INVALID_ARG
- ESP_ERR_INVALID_STATE

`inph_err_t esp_http_client_set_url(esp_http_client_handle_t client, const char *url)`

Set URL for client, when performing this behavior, the options in the URL will replace the old ones.

Parameters

- **client** – [in] The `esp_http_client` handle
- **url** – [in] The url

Returns

- ESP_OK
- ESP_FAIL

`inph_err_t esp_http_client_set_post_field(esp_http_client_handle_t client, const char *data, int len)`

Set post data, this function must be called before `esp_http_client_perform`. Note: Post data persists between requests and must be cleared manually. To avoid post data carryover, set the post data to NULL before starting any subsequent requests.

Parameters

- **client** – [in] The `esp_http_client` handle
- **data** – [in] post data pointer
- **len** – [in] post length

Returns

- ESP_OK

- ESP_FAIL

int **esp_http_client_get_post_field** (*esp_http_client_handle_t* client, char **data)

Get current post field information.

Parameters

- **client** – [in] The client
- **data** – [out] Point to post data pointer

Returns

Size of post data

inph_err_t **esp_http_client_set_header** (*esp_http_client_handle_t* client, const char *key, const char *value)

Set http request header, this function must be called after esp_http_client_init and before any perform function.

Parameters

- **client** – [in] The esp_http_client handle
- **key** – [in] The header key
- **value** – [in] The header value

Returns

- ESP_OK
- ESP_FAIL

inph_err_t **esp_http_client_get_header** (*esp_http_client_handle_t* client, const char *key, char **value)

Get http request header. The value parameter will be set to NULL if there is no header which is same as the key specified, otherwise the address of header value will be assigned to value parameter. This function must be called after esp_http_client_init.

Parameters

- **client** – [in] The esp_http_client handle
- **key** – [in] The header key
- **value** – [out] The header value

Returns

- ESP_OK
- ESP_FAIL

inph_err_t **esp_http_client_get_username** (*esp_http_client_handle_t* client, char **value)

Get http request username. The address of username buffer will be assigned to value parameter. This function must be called after esp_http_client_init.

Parameters

- **client** – [in] The esp_http_client handle
- **value** – [out] The username value

Returns

- ESP_OK
- ESP_ERR_INVALID_ARG

`inph_err_t esp_http_client_set_username` (*esp_http_client_handle_t* client, const char *username)

Set http request username. The value of username parameter will be assigned to username buffer. If the username parameter is NULL then username buffer will be freed.

Parameters

- **client** – [in] The esp_http_client handle
- **username** – [in] The username value

Returns

- ESP_OK
- ESP_ERR_INVALID_ARG

`inph_err_t esp_http_client_get_password` (*esp_http_client_handle_t* client, char **value)

Get http request password. The address of password buffer will be assigned to value parameter. This function must be called after esp_http_client_init.

Parameters

- **client** – [in] The esp_http_client handle
- **value** – [out] The password value

Returns

- ESP_OK
- ESP_ERR_INVALID_ARG

`inph_err_t esp_http_client_set_password` (*esp_http_client_handle_t* client, const char *password)

Set http request password. The value of password parameter will be assigned to password buffer. If the password parameter is NULL then password buffer will be freed.

Parameters

- **client** – [in] The esp_http_client handle
- **password** – [in] The password value

Returns

- ESP_OK
- ESP_ERR_INVALID_ARG

`inph_err_t esp_http_client_set_auth_type` (*esp_http_client_handle_t* client, *esp_http_client_auth_type_t* auth_type)

Set http request auth_type.

Parameters

- **client** – [in] The esp_http_client handle
- **auth_type** – [in] The esp_http_client auth type

Returns

- ESP_OK
- ESP_ERR_INVALID_ARG

`inph_err_t esp_http_client_get_user_data (esp_http_client_handle_t client, void **data)`

Get http request user_data. The value stored from the `esp_http_client_config_t` will be written to the address passed into data.

Parameters

- **client** – [in] The `esp_http_client` handle
- **data** – [out] A pointer to the pointer that will be set to user_data.

Returns

- ESP_OK
- ESP_ERR_INVALID_ARG

`inph_err_t esp_http_client_set_user_data (esp_http_client_handle_t client, void *data)`

Set http request user_data. The value passed in +data+ will be available during event callbacks. No memory management will be performed on the user's behalf.

Parameters

- **client** – [in] The `esp_http_client` handle
- **data** – [in] The pointer to the user data

Returns

- ESP_OK
- ESP_ERR_INVALID_ARG

`int esp_http_client_get_errno (esp_http_client_handle_t client)`

Get HTTP client session errno.

Parameters

- **client** – [in] The `esp_http_client` handle

Returns

- (-1) if invalid argument
- errno

`inph_err_t esp_http_client_get_and_clear_last_tls_error (esp_http_client_handle_t client, int *esp_tls_error_code, int *esp_tls_flags)`

Returns last error in `esp_tls` with detailed mbedtls related error codes. The error information is cleared internally upon return.

Parameters

- **client** – [in] The `esp_http_client` handle
- **esp_tls_error_code** – [out] last error code returned from mbedtls api (set to zero if none) This pointer could be NULL if caller does not care about `esp_tls_code`
- **esp_tls_flags** – [out] last certification verification flags (set to zero if none) This pointer could be NULL if caller does not care about `esp_tls_code`

Returns

- ESP_FAIL if invalid parameters
- ESP_OK (0) if no error occurred

- specific error code (based on ESP_ERR_ESP_TLS_BASE) otherwise

`inph_err_t esp_http_client_set_method` (*esp_http_client_handle_t* client, *esp_http_client_method_t* method)

Set http request method.

Parameters

- **client** – [in] The esp_http_client handle
- **method** – [in] The method

Returns

- ESP_OK
- ESP_ERR_INVALID_ARG

`inph_err_t esp_http_client_set_timeout_ms` (*esp_http_client_handle_t* client, int timeout_ms)

Set http request timeout.

Parameters

- **client** – [in] The esp_http_client handle
- **timeout_ms** – [in] The timeout value

Returns

- ESP_OK
- ESP_ERR_INVALID_ARG

`inph_err_t esp_http_client_delete_header` (*esp_http_client_handle_t* client, const char *key)

Delete http request header.

Parameters

- **client** – [in] The esp_http_client handle
- **key** – [in] The key

Returns

- ESP_OK
- ESP_FAIL

`inph_err_t esp_http_client_delete_all_headers` (*esp_http_client_handle_t* client)

Delete all http request headers.

Parameters

- **client** – [in] The esp_http_client handle

Returns

- ESP_OK
- ESP_FAIL

`inph_err_t esp_http_client_open` (*esp_http_client_handle_t* client, int write_len)

This function will be open the connection, write all header strings and return.

Parameters

- **client** – [in] The esp_http_client handle
- **write_len** – [in] HTTP Content length need to write to the server

Returns

- ESP_OK
- ESP_FAIL
- ESP_ERR_HTTP_CONNECTING if timed-out before connection is made

int **esp_http_client_write** (*esp_http_client_handle_t* client, const char *buffer, int len)

This function will write data to the HTTP connection previously opened by esp_http_client_open()

Parameters

- **client** – [in] The esp_http_client handle
- **buffer** – The buffer
- **len** – [in] This value must not be larger than the write_len parameter provided to esp_http_client_open()

Returns

- (-1) if any errors
- Length of data written

int64_t **esp_http_client_fetch_headers** (*esp_http_client_handle_t* client)

This function need to call after esp_http_client_open, it will read from http stream, process all receive headers.

Parameters

client – [in] The esp_http_client handle

Returns

- (0) if stream doesn't contain content-length header, or chunked encoding (checked by esp_http_client_is_chunked_response)
- (-1: ESP_FAIL) if any errors
- (-ESP_ERR_HTTP_EAGAIN = -0x7007) if call is timed-out before any data was ready
- Download data length defined by content-length header

bool **esp_http_client_is_chunked_response** (*esp_http_client_handle_t* client)

Check response data is chunked.

Parameters

client – [in] The esp_http_client handle

Returns

true or false

int **esp_http_client_read** (*esp_http_client_handle_t* client, char *buffer, int len)

Read data from http stream.

Note: (-ESP_ERR_HTTP_EAGAIN = -0x7007) is returned when call is timed-out before any data was ready

Parameters

- **client** – [in] The esp_http_client handle
- **buffer** – The buffer

- **len** – [in] The length

Returns

- (-1) if any errors
- Length of data was read

int **esp_http_client_get_status_code** (*esp_http_client_handle_t* client)

Get http response status code, the valid value if this function invoke after `esp_http_client_perform`

Parameters

client – [in] The `esp_http_client` handle

Returns

Status code

int64_t **esp_http_client_get_content_length** (*esp_http_client_handle_t* client)

Get http response content length (from header Content-Length) the valid value if this function invoke after `esp_http_client_perform`

Parameters

client – [in] The `esp_http_client` handle

Returns

- (-1) Chunked transfer
- Content-Length value as bytes

inph_err_t **esp_http_client_close** (*esp_http_client_handle_t* client)

Close http connection, still kept all http request resources.

Parameters

client – [in] The `esp_http_client` handle

Returns

- ESP_OK
- ESP_FAIL

inph_err_t **esp_http_client_cleanup** (*esp_http_client_handle_t* client)

This function must be the last function to call for an session. It is the opposite of the `esp_http_client_init` function and must be called with the same handle as input that a `esp_http_client_init` call returned. This might close all connections this handle has used and possibly has kept open until now. Don't call this function if you intend to transfer more files, re-using handles is a key to good performance with `esp_http_client`.

Parameters

client – [in] The `esp_http_client` handle

Returns

- ESP_OK
- ESP_FAIL

esp_http_client_transport_t **esp_http_client_get_transport_type** (*esp_http_client_handle_t* client)

Get transport type.

Parameters

client – [in] The `esp_http_client` handle

Returns

- HTTP_TRANSPORT_UNKNOWN
- HTTP_TRANSPORT_OVER_TCP
- HTTP_TRANSPORT_OVER_SSL

`inph_err_t esp_http_client_set_redirection` (*esp_http_client_handle_t* client)

Set redirection URL. When received the 30x code from the server, the client stores the redirect URL provided by the server. This function will set the current URL to redirect to enable client to execute the redirection request. When `disable_auto_redirect` is set, the client will not call this function but the event `HTTP_EVENT_REDIRECT` will be dispatched giving the user control over the redirection event.

Parameters

client – [in] The `esp_http_client` handle

Returns

- ESP_OK
- ESP_FAIL

`inph_err_t esp_http_client_reset_redirect_counter` (*esp_http_client_handle_t* client)

Reset the redirection counter. This is useful to reset redirect counter in cases where the same handle is used for multiple requests.

Parameters

client – [in] The `esp_http_client` handle

Returns

- ESP_OK
- ESP_ERR_INVALID_ARG

`inph_err_t esp_http_client_set_auth_data` (*esp_http_client_handle_t* client, const char *auth_data, int len)

On receiving a custom authentication header, this API can be invoked to set the authentication information from the header. This API can be called from the event handler.

Parameters

- **client** – [in] The `esp_http_client` handle
- **auth_data** – [in] The authentication data received in the header
- **len** – [in] length of `auth_data`.

Returns

- ESP_ERR_INVALID_ARG
- ESP_OK

`inph_err_t esp_http_client_add_auth` (*esp_http_client_handle_t* client)

On receiving HTTP Status code 401, this API can be invoked to add authorization information.

Note: There is a possibility of receiving body message with redirection status codes, thus make sure to flush off body data after calling this API.

Parameters

client – [in] The `esp_http_client` handle

Returns

- ESP_OK: Successfully added the authentication information.
- ESP_ERR_INVALID_ARG: Invalid client handle passed.
- ESP_ERR_NO_MEM: Memory allocation failed for the required fields.
- ESP_ERR_NOT_SUPPORTED: Unsupported authentication type in the header.
- ESP_FAIL: Failed to add authentication information due to other reasons.

bool **esp_http_client_is_complete_data_received** (*esp_http_client_handle_t* client)

Checks if entire data in the response has been read without any error.

Parameters

client – [in] The esp_http_client handle

Returns

- true
- false

int **esp_http_client_read_response** (*esp_http_client_handle_t* client, char *buffer, int len)

Helper API to read larger data chunks This is a helper API which internally calls esp_http_client_read multiple times till the end of data is reached or till the buffer gets full.

Parameters

- **client** – [in] The esp_http_client handle
- **buffer** – The buffer
- **len** – [in] The buffer length

Returns

- Length of data was read

inph_err_t **esp_http_client_flush_response** (*esp_http_client_handle_t* client, int *len)

Process all remaining response data This uses an internal buffer to repeatedly receive, parse, and discard response data until complete data is processed. As no additional user-supplied buffer is required, this may be preferable to esp_http_client_read_response in situations where the content of the response may be ignored.

Parameters

- **client** – [in] The esp_http_client handle
- **len** – Length of data discarded

Returns

- ESP_OK If successful, len will have discarded length
- ESP_FAIL If failed to read response
- ESP_ERR_INVALID_ARG If the client is NULL

inph_err_t **esp_http_client_get_url** (*esp_http_client_handle_t* client, char *url, const int len)

Get URL from client.

Parameters

- **client** – [in] The esp_http_client handle
- **url** – [inout] The buffer to store URL

- **len** – [in] The buffer length

Returns

- ESP_OK
- ESP_FAIL

`inph_err_t esp_http_client_get_chunk_length (esp_http_client_handle_t client, int *len)`

Get Chunk-Length from client.

Parameters

- **client** – [in] The esp_http_client handle
- **len** – [out] Variable to store length

Returns

- ESP_OK If successful, len will have length of current chunk
- ESP_FAIL If the server is not a chunked server
- ESP_ERR_INVALID_ARG If the client or len are NULL

`esp_http_state_t esp_http_client_get_state (esp_http_client_handle_t client)`

Get the current state of the HTTP client.

Parameters

client – [in] The HTTP client handle

Returns

Current state of the HTTP client

`bool esp_http_client_is_persistent_connection (esp_http_client_handle_t client)`

Check if persistent connection is supported by the server.

Parameters

client – [in] The HTTP client handle

Returns

true if persistent connection is supported, false otherwise

`struct esp_http_client_event`

`#include <esp_http_client.h>` HTTP Client events data.

Public Members

`esp_http_client_event_id_t event_id`

event_id, to know the cause of the event

`esp_http_client_handle_t client`

esp_http_client_handle_t context

`void *data`

data of the event

int **data_len**

data length of data

void ***user_data**

user_data context, from esp_http_client_config_t user_data

char ***header_key**

For HTTP_EVENT_ON_HEADER event_id, it's store current http header key

char ***header_value**

For HTTP_EVENT_ON_HEADER event_id, it's store current http header value

struct **esp_http_client_on_data**

#include <esp_http_client.h> Argument structure for HTTP_EVENT_ON_DATA event.

Public Members

esp_http_client_handle_t **client**

Client handle

int64_t **data_process**

Total data processed

struct **esp_http_client_redirect_event_data**

#include <esp_http_client.h> Argument structure for HTTP_EVENT_REDIRECT event.

Public Members

esp_http_client_handle_t **client**

Client handle

int **status_code**

Status Code

struct **esp_http_client_config_t**

#include <esp_http_client.h> HTTP configuration.

Public Members

const char ***url**

HTTP URL, the information on the URL is most important, it overrides the other fields below, if any

const char ***host**

Domain or IP as string

int **port**

Port to connect, default depend on esp_http_client_transport_t (80 or 443)

const char ***username**

Using for Http authentication

const char ***password**

Using for Http authentication

esp_http_client_auth_type_t **auth_type**

Http authentication type, see esp_http_client_auth_type_t

const char ***path**

HTTP Path, if not set, default is /

const char ***query**

HTTP query

const char ***cert_pem**

SSL server certification, PEM format as string, if the client requires to verify server

const char ***cert_der**

SSL server certification, DER format as binary, if the client requires to verify server

union esp_http_client_config_t

size_t **cert_len**

Length of the buffer pointed to by cert_pem or cert_der. PEM Certificate - Length of the buffer pointed to by cert_pem. Length should be the length of the certificate including NULL terminator or 0. DER Certificate - Length of the buffer pointed to by cert_der. Should be the length of the certificate.

const char ***client_cert_pem**

SSL client certification, PEM format as string, if the server requires to verify client

const char ***client_cert_der**

SSL client certification, DER format as binary, if the server requires to verify client

union esp_http_client_config_t

size_t client_cert_len

Length of the buffer pointed to by client_cert_pem or client_cert_der. PEM Certificate - Length of the buffer pointed to by client_cert_pem. Length should be the length of the certificate including NULL terminator or 0. DER Certificate - Length of the buffer pointed to by client_cert_der. Should be the length of the certificate.

const char *client_key_pem

SSL client key, PEM format as string, if the server requires to verify client

size_t client_key_len

Length of the buffer pointed to by client_key_pem. May be 0 for null-terminated pem

const char *client_key_password

Client key decryption password string

size_t client_key_password_len

String length of the password pointed to by client_key_password

esp_http_client_proto_ver_t tls_version

TLS protocol version of the connection, e.g., TLS 1.2, TLS 1.3 (default - no preference)

const char *user_agent

The User Agent string to send with HTTP requests

esp_http_client_method_t method

HTTP Method

int timeout_ms

Network timeout in milliseconds. 0 defaults to 5 seconds

bool disable_auto_redirect

Disable HTTP automatic redirects

int max_redirection_count

Max number of redirections on receiving HTTP redirect status code, using default value if zero

int max_authorization_retries

Max connection retries on receiving HTTP unauthorized status code, using default value if zero. Disables authorization retry if -1

http_event_handle_cb event_handler

HTTP Event Handle

esp_http_client_transport_t transport_type

HTTP transport type, see esp_http_client_transport_t

int **buffer_size**

HTTP receive buffer size

int **buffer_size_tx**

HTTP transmit buffer size

void ***user_data**

HTTP user_data context

bool **is_async**

Set asynchronous mode, only supported with HTTPS for now

bool **use_global_ca_store**

Use a global ca_store for all the connections in which this bool is set.

bool **skip_cert_common_name_check**

Skip any validation of server certificate CN field

const char ***common_name**

Pointer to the string containing server certificate common name. If non-NULL, server certificate CN must match this name, If NULL, server certificate CN must match hostname.

inph_err_t (***crt_bundle_attach**)(void *conf)

Function pointer to esp_crt_bundle_attach. Enables the use of certification bundle for server verification, must be enabled in menuconfig

bool **keep_alive_enable**

Enable keep-alive timeout

int **keep_alive_idle**

Keep-alive idle time. Default is 5 (second)

int **keep_alive_interval**

Keep-alive interval time. Default is 5 (second)

int **keep_alive_count**

Keep-alive packet retry send count. Default is 3 counts

struct ifreq ***if_name**

The name of interface for data to go through. Use the default interface without setting

esp_http_client_addr_type_t **addr_type**

Address type used in http client configurations

2.2 Peripherals API

2.2.1 UART

Introduction

A Universal Asynchronous Receiver/Transmitter(UART) is a hardware feature that handles communication (i.e., timing requirements and data framing) using widely-adopted asynchronous serial communication interfaces, such as RS232, RS422, and RS485. A UART provides a widely adopted and cheap method to realize full-duplex or half-duplex data exchange among different devices.

The GTS32 chip has 3 UART controllers (also referred to as ports), each featuring an identical set of registers to simplify programming and for more flexibility.

Each UART controller is independently configurable with parameters such as baud rate, data bit length, bit ordering, number of stop bits, parity bit, etc. All the regular UART controllers are compatible with UART-enabled devices from various manufacturers and can also support Infrared Data Association (IrDA) protocols.

Functional Overview

The overview describes how to establish communication between an GTS32 and other UART devices using the functions and data types of the UART driver. A typical programming work flow is broken down into the sections provided below:

1. *Set Communication Parameters* - Setting baud rate, data bits, stop bits, etc.
2. *Set Communication Pins* - Assigning pins for connection to a device
3. *Install Drivers* - Allocating GTS32's resources for the UART driver
4. *Run UART Communication* - Sending/receiving data
5. *Use Interrupts* - Triggering interrupts on specific communication events
6. *Deleting a Driver* - Freeing allocated resources if a UART communication is no longer required

Steps 1 to 3 comprise the configuration stage. Step 4 is where the UART starts operating. Steps 5 and 6 are optional.

API Reference

Defines

UART_PIN_NO_CHANGE

UART_NUM_0

UART_NUM_1

UART_NUM_2

UART_NUM_MAX

UART_HW_FIFO_LEN (uart_num)

Typedefs

```
typedef int uart_port_t
```

```
typedef void (*uart_select_notif_callback_t)(uart_port_t uart_num, uart_select_notif_t uart_select_notif,  
BaseType_t *task_woken)
```

```
typedef void *uart_isr_handle_t
```

Enums

```
enum uart_word_length_t
```

Values:

```
enumerator UART_DATA_5_BITS
```

```
enumerator UART_DATA_6_BITS
```

```
enumerator UART_DATA_7_BITS
```

```
enumerator UART_DATA_8_BITS
```

```
enum uart_stop_bits_t
```

Values:

```
enumerator UART_STOP_BITS_1
```

```
enumerator UART_STOP_BITS_2
```

```
enum uart_parity_t
```

Values:

```
enumerator UART_PARITY_DISABLE
```

```
enumerator UART_PARITY_EVEN
```

```
enumerator UART_PARITY_ODD
```

```
enum uart_hw_flowcontrol_t
```

Values:

```
enumerator UART_HW_FLOWCTRL_DISABLE
```

enumerator **UART_HW_FLOWCTRL_CTS_RTS**

enum **uart_sclk_t**

Values:

enumerator **UART_SCLK_DEFAULT**

enum **uart_mode_t**

Values:

enumerator **UART_MODE_UART**

enumerator **UART_MODE_RS485**

enumerator **UART_MODE_IRDA**

enum **uart_signal_inv_t**

Values:

enumerator **UART_SIGNAL_INV_DISABLE**

enumerator **UART_SIGNAL_INV_TXD**

enumerator **UART_SIGNAL_INV_RXD**

enumerator **UART_SIGNAL_INV_RTS**

enumerator **UART_SIGNAL_INV_CTS**

enum **uart_event_type_t**

Values:

enumerator **UART_DATA**

enumerator **UART_BREAK**

enumerator **UART_BUFFER_FULL**

enumerator **UART_FIFO_OVF**

enumerator **UART_FRAME_ERR**

enumerator **UART_PARITY_ERR**

enumerator **UART_DATA_BREAK**

enumerator **UART_PATTERN_DET**

enumerator **UART_WAKEUP**

enumerator **UART_EVENT_MAX**

enum **uart_select_notif_t**

Values:

enumerator **UART_SELECT_READ_NOTIF**

enumerator **UART_SELECT_WRITE_NOTIF**

enumerator **UART_SELECT_ERROR_NOTIF**

Functions

`esp_err_t uart_driver_install (uart_port_t uart_num, int rx_buffer_size, int tx_buffer_size, int queue_size, QueueHandle_t *uart_queue, int intr_alloc_flags)`

`esp_err_t uart_driver_delete (uart_port_t uart_num)`

`bool uart_is_driver_installed (uart_port_t uart_num)`

`esp_err_t uart_set_word_length (uart_port_t uart_num, uart_word_length_t data_bit)`

`esp_err_t uart_get_word_length (uart_port_t uart_num, uart_word_length_t *data_bit)`

`esp_err_t uart_set_stop_bits (uart_port_t uart_num, uart_stop_bits_t stop_bit)`

`esp_err_t uart_get_stop_bits (uart_port_t uart_num, uart_stop_bits_t *stop_bit)`

`esp_err_t uart_set_parity (uart_port_t uart_num, uart_parity_t parity_mode)`

`esp_err_t uart_get_parity (uart_port_t uart_num, uart_parity_t *parity_mode)`

`esp_err_t uart_set_baudrate (uart_port_t uart_num, uint32_t baud_rate)`

`esp_err_t uart_get_baudrate (uart_port_t uart_num, uint32_t *baudrate)`

`esp_err_t uart_get_sclk_freq (uart_sclk_t sclk, uint32_t *out_freq_hz)`

`esp_err_t uart_set_line_inverse (uart_port_t uart_num, uint32_t inverse_mask)`

`esp_err_t uart_set_sw_flow_ctrl (uart_port_t uart_num, bool enable, uint8_t rx_thresh_xon, uint8_t rx_thresh_xoff)`

`esp_err_t uart_set_hw_flow_ctrl (uart_port_t uart_num, uart_hw_flowcontrol_t flow_ctrl, uint8_t rx_thresh)`

```

esp_err_t uart_get_hw_flow_ctrl (uart_port_t uart_num, uart_hw_flowcontrol_t *flow_ctrl)

esp_err_t uart_clear_intr_status (uart_port_t uart_num, uint32_t clr_mask)

esp_err_t uart_enable_intr_mask (uart_port_t uart_num, uint32_t enable_mask)

esp_err_t uart_disable_intr_mask (uart_port_t uart_num, uint32_t disable_mask)

int uart_pattern_pop_pos (uart_port_t uart_num)

int uart_pattern_get_pos (uart_port_t uart_num)

esp_err_t uart_pattern_queue_reset (uart_port_t uart_num, int queue_length)

esp_err_t uart_enable_pattern_det_baud_intr (uart_port_t uart_num, char pattern_chr, uint8_t
                                             chr_num, int chr_tout, int post_idle, int pre_idle)

esp_err_t uart_disable_pattern_det_intr (uart_port_t uart_num)

esp_err_t uart_enable_rx_intr (uart_port_t uart_num)

esp_err_t uart_disable_rx_intr (uart_port_t uart_num)

esp_err_t uart_enable_tx_intr (uart_port_t uart_num, int enable, int thresh)

esp_err_t uart_disable_tx_intr (uart_port_t uart_num)

esp_err_t uart_set_pin (uart_port_t uart_num, int tx_io_num, int rx_io_num, int rts_io_num, int cts_io_num)

esp_err_t uart_set_rts (uart_port_t uart_num, int level)

esp_err_t uart_set_dtr (uart_port_t uart_num, int level)

esp_err_t uart_set_tx_idle_num (uart_port_t uart_num, uint16_t idle_num)

esp_err_t uart_param_config (uart_port_t uart_num, const uart_config_t *uart_config)

esp_err_t uart_intr_config (uart_port_t uart_num, const uart_intr_config_t *intr_conf)

esp_err_t uart_wait_tx_done (uart_port_t uart_num, TickType_t ticks_to_wait)

int uart_tx_chars (uart_port_t uart_num, const char *buffer, uint32_t len)

int uart_write_bytes (uart_port_t uart_num, const void *src, size_t size)

int uart_write_bytes_with_break (uart_port_t uart_num, const void *src, size_t size, int brk_len)

int uart_read_bytes (uart_port_t uart_num, void *buf, uint32_t length, TickType_t ticks_to_wait)

esp_err_t uart_get_buffered_data_len (uart_port_t uart_num, size_t *size)

esp_err_t uart_get_tx_buffer_free_size (uart_port_t uart_num, size_t *size)

esp_err_t uart_flush (uart_port_t uart_num)

esp_err_t uart_flush_input (uart_port_t uart_num)

esp_err_t uart_set_mode (uart_port_t uart_num, uart_mode_t mode)

esp_err_t uart_set_rx_full_threshold (uart_port_t uart_num, int threshold)

```

```

esp_err_t uart_set_tx_empty_threshold(uart_port_t uart_num, int threshold)
esp_err_t uart_set_rx_timeout(uart_port_t uart_num, const uint8_t tout_thresh)
esp_err_t uart_get_collision_flag(uart_port_t uart_num, bool *collision_flag)
esp_err_t uart_set_wakeup_threshold(uart_port_t uart_num, int wakeup_threshold)
esp_err_t uart_get_wakeup_threshold(uart_port_t uart_num, int *out_wakeup_threshold)
esp_err_t uart_wait_tx_idle_polling(uart_port_t uart_num)
esp_err_t uart_set_loop_back(uart_port_t uart_num, bool loop_back_en)
void uart_set_always_rx_timeout(uart_port_t uart_num, bool always_rx_timeout)
esp_err_t uart_detect_bitrate_start(uart_port_t uart_num, const uart_bitrate_detect_config_t *config)
esp_err_t uart_detect_bitrate_stop(uart_port_t uart_num, bool reset, uart_bitrate_res_t *result)
void uart_set_select_notif_callback(uart_port_t uart_num, uart_select_notif_callback_t
                                   uart_select_notif_callback)

portMUX_TYPE *uart_get_selectlock(void)

struct uart_config_t

```

Public Members

```

int baud_rate

uart_word_length_t data_bits

uart_parity_t parity

uart_stop_bits_t stop_bits

uart_hw_flowcontrol_t flow_ctrl

uint8_t rx_flow_ctrl_thresh

uart_sclk_t source_clk

uint32_t allow_pd

uint32_t backup_before_sleep

struct uart_config_t flags

struct uart_intr_config_t

```

Public Members

uint32_t **intr_enable_mask**

uint8_t **rx_timeout_thresh**

uint8_t **txfifo_empty_intr_thresh**

uint8_t **rxfifo_full_thresh**

struct **uart_event_t**

Public Members

uart_event_type_t **type**

size_t **size**

bool **timeout_flag**

struct **uart_bitrate_detect_config_t**

Public Members

int **rx_io_num**

uart_sclk_t **source_clk**

struct **uart_bitrate_res_t**

Public Members

uint32_t **clk_freq_hz**

uint32_t **pos_period**

uint32_t **neg_period**

2.2.2 GPIO

Introduction

The GPIO libraries provide support for configuring and monitoring the Neon.B0 GPIO (General Purpose Input/Output) peripheral functionality.

Application Examples

Several application examples demonstrate the functionality of the GPIO library. Please check the README for more details.

API Reference

Defines

GPIO_PIN_COUNT

GPIO_IS_VALID_GPIO (gpio_num)

Check whether it is a valid GPIO number.

GPIO_IS_VALID_OUTPUT_GPIO (gpio_num)

Check whether it can be a valid GPIO number of output mode.

Typedefs

typedef void (***gpio_isr_t**)(void *arg)

GPIO interrupt handler.

Param arg

User registered data

Functions

inph_err_t **gpio_isr_handler_add** (gpio_num_t gpio_num, *gpio_isr_t* isr_handler, void *args)

Add ISR handler for the corresponding GPIO pin.

Parameters

- **gpio_num** – GPIO number
- **isr_handler** – ISR handler function for the corresponding GPIO number.
- **args** – parameter for ISR handler.

Returns

- INPH_OK Success
- INPH_ERR_INVALID_STATE Wrong state, the ISR service has not been initialized.
- INPH_ERR_INVALID_ARG Parameter error

`inph_err_t gpio_isr_handler_remove (gpio_num_t gpio_num)`

Remove ISR handler for the corresponding GPIO pin.

Parameters

gpio_num – GPIO number

Returns

- INPH_OK Success
- INPH_ERR_INVALID_STATE Wrong state, the ISR service has not been initialized.
- INPH_ERR_INVALID_ARG Parameter error

`inph_err_t gpio_config (const gpio_config_t *pGPIOConfig)`

GPIO common configuration.

Configure GPIO's Mode, pull-up, PullDown, IntrType

Note: This function always overwrite all the current IO configurations

Parameters

pGPIOConfig – Pointer to GPIO configure struct

Returns

- INPH_OK success
- INPH_ERR_INVALID_ARG Parameter error

`inph_err_t gpio_set_intr_type (gpio_num_t gpio_num, gpio_int_type_t intr_type)`

GPIO set interrupt trigger type.

Parameters

- **gpio_num** – GPIO number. If you want to set the trigger type of e.g. of GPIO16, gpio_num should be GPIO_NUM_16 (16);
- **intr_type** – Interrupt type, select from gpio_int_type_t

Returns

- INPH_OK Success
- INPH_ERR_INVALID_ARG Parameter error

`inph_err_t gpio_install_isr_service (int intr_alloc_flags)`

Install the GPIO driver's ETS_GPIO_INTR_SOURCE ISR handler service, which allows per-pin GPIO interrupt handlers.

This function is incompatible with `gpio_isr_register()` - if that function is used, a single global ISR is registered for all GPIO interrupts. If this function is used, the ISR service provides a global GPIO ISR and individual pin handlers are registered via the `gpio_isr_handler_add()` function.

Parameters

intr_alloc_flags – Flags used to allocate the interrupt. One or multiple (ORred) INPH_INTR_FLAG_* values. See `inph_intr_alloc.h` for more info.

Returns

- INPH_OK Success
- INPH_ERR_NO_MEM No memory to install this service
- INPH_ERR_INVALID_STATE ISR service already installed.
- INPH_ERR_NOT_FOUND No free interrupt found with the specified flags
- INPH_ERR_INVALID_ARG GPIO error

int **gpio_get_level** (gpio_num_t gpio_num)

GPIO get input level.

Warning: If the pad is not configured for input (or input and output) the returned value is always 0.

Parameters

gpio_num – GPIO number. If you want to get the logic level of e.g. pin GPIO16, gpio_num should be GPIO_NUM_16 (16);

Returns

- 0 the GPIO input level is 0
- 1 the GPIO input level is 1

inph_err_t **gpio_set_level** (gpio_num_t gpio_num, uint32_t level)

GPIO set output level.

Note: This function is allowed to be executed when Cache is disabled within ISR context, by enabling CONFIG_GPIO_CTRL_FUNC_IN_IRAM

Parameters

- **gpio_num** – GPIO number. If you want to set the output level of e.g. GPIO16, gpio_num should be GPIO_NUM_16 (16);
- **level** – Output level. 0: low ; 1: high

Returns

- INPH_OK Success
- INPH_ERR_INVALID_ARG GPIO number error

inph_err_t **gpio_set_direction** (gpio_num_t gpio_num, gpio_mode_t mode)

GPIO set direction.

Configure GPIO mode,such as output_only,input_only,output_and_input

Note: This function always overwrite all the current modes that have applied on the IO pin

Parameters

- **gpio_num** – Configure GPIO pins number, it should be GPIO number. If you want to set direction of e.g. GPIO16, gpio_num should be GPIO_NUM_16 (16);
- **mode** – GPIO direction

Returns

- INPH_OK Success
- INPH_ERR_INVALID_ARG GPIO error

inph_err_t **gpio_set_pull_mode** (gpio_num_t gpio_num, gpio_pull_mode_t pull)

Configure GPIO internal pull-up/pull-down resistors.

Note: This function always overwrite the current pull-up/pull-down configurations

Note: INPH32: Only pins that support both input & output have integrated pull-up and pull-down resistors. Input-only GPIOs 34-39 do not.

Parameters

- **gpio_num** – GPIO number. If you want to set pull up or down mode for e.g. GPIO16, gpio_num should be GPIO_NUM_16 (16);
- **pull** – GPIO pull up/down mode.

Returns

- INPH_OK Success
- INPH_ERR_INVALID_ARG : Parameter error

inph_err_t **gpio_reset_pin** (gpio_num_t gpio_num)

Reset a GPIO to a certain state (select gpio function, enable pullup and disable input and output).

Parameters

gpio_num – GPIO number.

Returns

- INPH_OK Success
- INPH_ERR_INVALID_ARG Parameter error

struct **gpio_config_t**

#include <gpio.h> Configuration parameters of GPIO pad for gpio_config function.

Public Members

uint64_t **pin_bit_mask**

GPIO pin: set with bit mask, each bit maps to a GPIO

gpio_mode_t **mode**

GPIO mode: set input/output mode

gpio_pullup_t **pull_up_en**

GPIO pull-up

gpio_pulldown_t **pull_down_en**

GPIO pull-down

gpio_int_type_t **intr_type**

GPIO interrupt type

Chapter 3

Disclaimers

Limited warranty and liability — Information in this document is believed to be accurate and reliable. However, InnoPhase IoT Incorporated does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and assumes no liability associated with the use of such information. InnoPhase IoT Incorporated takes no responsibility for the content in this document if provided by an information source outside of InnoPhase IoT Incorporated.

InnoPhase IoT Incorporated disclaims liability for any indirect, incidental, punitive, special or consequential damages associated with the use of this document, applications and any products associated with information in this document, whether or not such damages are based on tort (including negligence), warranty, including warranty of merchantability, warranty of fitness for a particular purpose, breach of contract or any other legal theory. Further, InnoPhase IoT Incorporated accepts no liability and makes no warranty, express or implied, for any assistance given with respect to any applications described herein or customer product design, or the application or use by any customer's third-party customer(s).

Notwithstanding any damages that a customer might incur for any reason whatsoever, InnoPhase IoT Incorporated' aggregate and cumulative liability for the products described herein shall be limited in accordance with the Terms and Conditions of identified in the commercial sale documentation for such InnoPhase IoT Incorporated products.

Right to make changes — InnoPhase IoT Incorporated reserves the right to make changes to information published in this document, including, without limitation, changes to any specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use — InnoPhase IoT Incorporated products are not designed, authorized or warranted to be suitable for use in life support, life-critical or safety-critical systems or equipment, nor in applications where failure or malfunction of an InnoPhase IoT Incorporated product can reasonably be expected to result in personal injury, death or severe property or environmental damage. InnoPhase IoT Incorporated and its suppliers accept no liability for inclusion and/or use of InnoPhase IoT Incorporated products in such equipment or applications and such inclusion and/or use is at the customer's own risk.

All trademarks, trade names and registered trademarks mentioned in this document are property of their respective owners and are hereby acknowledged.

Index

B

BIT_NUMBER_TO_CHANNEL (*C macro*), 22

C

CHANNEL_TO_BIT (*C macro*), 22

CHANNEL_TO_BIT_NUMBER (*C macro*), 22

D

DEFAULT_HTTP_BUF_SIZE (*C macro*), 119

E

ESP_ERR_HTTP_BASE (*C macro*), 119

ESP_ERR_HTTP_CONNECT (*C macro*), 119

ESP_ERR_HTTP_CONNECTING (*C macro*), 119

ESP_ERR_HTTP_CONNECTION_CLOSED (*C macro*), 119

ESP_ERR_HTTP_EAGAIN (*C macro*), 119

ESP_ERR_HTTP_FETCH_HEADER (*C macro*), 119

ESP_ERR_HTTP_INCOMPLETE_DATA (*C macro*), 119

ESP_ERR_HTTP_INVALID_TRANSPORT (*C macro*), 119

ESP_ERR_HTTP_MAX_REDIRECT (*C macro*), 119

ESP_ERR_HTTP_NOT_MODIFIED (*C macro*), 119

ESP_ERR_HTTP_RANGE_NOT_SATISFIABLE (*C macro*), 119

ESP_ERR_HTTP_READ_TIMEOUT (*C macro*), 119

ESP_ERR_HTTP_WRITE_DATA (*C macro*), 119

esp_http_client_add_auth (*C++ function*), 134

esp_http_client_addr_type_t (*C++ enum*), 123

esp_http_client_addr_type_t::HTTP_ADDR_TYPE_INET (*C++ enumerator*), 123

esp_http_client_addr_type_t::HTTP_ADDR_TYPE_INET6 (*C++ enumerator*), 123

esp_http_client_addr_type_t::HTTP_ADDR_TYPE_UNSPEC (*C++ enumerator*), 123

esp_http_client_auth_type_t (*C++ enum*), 123

esp_http_client_auth_type_t::HTTP_AUTH_TYPE_BASIC (*C++ enumerator*), 123

esp_http_client_auth_type_t::HTTP_AUTH_TYPE_DIGEST (*C++ enumerator*), 123

esp_http_client_auth_type_t::HTTP_AUTH_TYPE_NONE (*C++ enumerator*), 123

esp_http_client_cancel_request (*C++ function*), 127

esp_http_client_cleanup (*C++ function*), 133

esp_http_client_close (*C++ function*), 133

esp_http_client_config_t (*C++ struct*), 137

esp_http_client_config_t::addr_type (*C++ member*), 140

esp_http_client_config_t::auth_type (*C++ member*), 138

esp_http_client_config_t::buffer_size (*C++ member*), 139

esp_http_client_config_t::buffer_size_tx (*C++ member*), 140

esp_http_client_config_t::cert_der (*C++ member*), 138

esp_http_client_config_t::cert_len (*C++ member*), 138

esp_http_client_config_t::cert_pem (*C++ member*), 138

esp_http_client_config_t::client_cert_der (*C++ member*), 138

esp_http_client_config_t::client_cert_len (*C++ member*), 139

esp_http_client_config_t::client_cert_pem (*C++ member*), 138

esp_http_client_config_t::client_key_len (*C++ member*), 139

esp_http_client_config_t::client_key_password (*C++ member*), 139

esp_http_client_config_t::client_key_password_len (*C++ member*), 139

esp_http_client_config_t::client_key_pem (*C++ member*), 139

esp_http_client_config_t::common_name (*C++ member*), 140

esp_http_client_config_t::crt_bundle_attach (*C++ member*), 140

esp_http_client_config_t::disable_auto_redirect (*C++ member*), 139

esp_http_client_config_t::event_handler (*C++ member*), 139

esp_http_client_config_t::host (*C++ member*), 138

esp_http_client_config_t::if_name (*C++ member*), 140

esp_http_client_config_t::is_async (C++ member), 140	ber), 136
esp_http_client_config_t::keep_alive_count (C++ member), 140	esp_http_client_event::data (C++ member), 136
esp_http_client_config_t::keep_alive_enable (C++ member), 140	esp_http_client_event::data_len (C++ member), 136
esp_http_client_config_t::keep_alive_idle (C++ member), 140	esp_http_client_event::event_id (C++ member), 136
esp_http_client_config_t::keep_alive_interval (C++ member), 140	esp_http_client_event::header_key (C++ member), 137
esp_http_client_config_t::max_authorization_retry_count (C++ member), 139	esp_http_client_event::header_value (C++ member), 137
esp_http_client_config_t::max_redirection_count (C++ member), 139	esp_http_client_event::user_data (C++ member), 137
esp_http_client_config_t::method (C++ member), 139	esp_http_client_event_handle_t (C++ type), 120
esp_http_client_config_t::password (C++ member), 138	esp_http_client_event_id_t (C++ enum), 120
esp_http_client_config_t::path (C++ member), 138	esp_http_client_event_id_t::HTTP_EVENT_DISCONNECTED (C++ enumerator), 121
esp_http_client_config_t::port (C++ member), 138	esp_http_client_event_id_t::HTTP_EVENT_ERROR (C++ enumerator), 120
esp_http_client_config_t::query (C++ member), 138	esp_http_client_event_id_t::HTTP_EVENT_HEADER_SENT (C++ enumerator), 120
esp_http_client_config_t::skip_cert_common_name_check (C++ member), 140	esp_http_client_event_id_t::HTTP_EVENT_HEADERS_SENT (C++ enumerator), 120
esp_http_client_config_t::timeout_ms (C++ member), 139	esp_http_client_event_id_t::HTTP_EVENT_ON_CONNECTED (C++ enumerator), 120
esp_http_client_config_t::tls_version (C++ member), 139	esp_http_client_event_id_t::HTTP_EVENT_ON_DATA (C++ enumerator), 121
esp_http_client_config_t::transport_type (C++ member), 139	esp_http_client_event_id_t::HTTP_EVENT_ON_FINISH (C++ enumerator), 121
esp_http_client_config_t::url (C++ member), 138	esp_http_client_event_id_t::HTTP_EVENT_ON_HEADER (C++ enumerator), 120
esp_http_client_config_t::use_global_ca_cert (C++ member), 140	esp_http_client_event_id_t::HTTP_EVENT_ON_HEADERS_COMPLETE (C++ enumerator), 120
esp_http_client_config_t::user_agent (C++ member), 139	esp_http_client_event_id_t::HTTP_EVENT_ON_STATUS_COMPLETE (C++ enumerator), 120
esp_http_client_config_t::user_data (C++ member), 140	esp_http_client_event_id_t::HTTP_EVENT_REDIRECT (C++ enumerator), 121
esp_http_client_config_t::username (C++ member), 138	esp_http_client_event_t (C++ type), 120
esp_http_client_delete_all_headers (C++ function), 131	esp_http_client_fetch_headers (C++ function), 132
esp_http_client_delete_header (C++ function), 131	esp_http_client_flush_response (C++ function), 135
esp_http_client_ecdsa_curve_t (C++ enum), 121	esp_http_client_get_and_clear_last_tls_error (C++ function), 130
esp_http_client_ecdsa_curve_t::ESP_HTTP_CLIENT_ECDSA_CURVE_MAX (C++ enumerator), 122	esp_http_client_get_chunk_length (C++ function), 136
esp_http_client_ecdsa_curve_t::ESP_HTTP_CLIENT_ECDSA_CURVE_SECP256R1 (C++ enumerator), 121	esp_http_client_get_content_length (C++ function), 136
esp_http_client_event (C++ struct), 136	esp_http_client_get_errno (C++ function), 130
esp_http_client_event::client (C++ member), 136	esp_http_client_get_header (C++ function), 128
	esp_http_client_get_password (C++ function), 136

tion), 129
 esp_http_client_get_post_field (C++ function), 128
 esp_http_client_get_state (C++ function), 136
 esp_http_client_get_status_code (C++ function), 133
 esp_http_client_get_transport_type (C++ function), 133
 esp_http_client_get_url (C++ function), 135
 esp_http_client_get_user_data (C++ function), 129
 esp_http_client_get_username (C++ function), 128
 esp_http_client_handle_t (C++ type), 120
 esp_http_client_init (C++ function), 125
 esp_http_client_is_chunked_response (C++ function), 132
 esp_http_client_is_complete_data_received (C++ function), 135
 esp_http_client_is_persistent_connection (C++ function), 136
 esp_http_client_method_t (C++ enum), 122
 esp_http_client_method_t::HTTP_METHOD_COPY (C++ enumerator), 122
 esp_http_client_method_t::HTTP_METHOD_DELETE (C++ enumerator), 122
 esp_http_client_method_t::HTTP_METHOD_GET (C++ enumerator), 122
 esp_http_client_method_t::HTTP_METHOD_HEAD (C++ enumerator), 122
 esp_http_client_method_t::HTTP_METHOD_LOCK (C++ enumerator), 122
 esp_http_client_method_t::HTTP_METHOD_MAX (C++ enumerator), 123
 esp_http_client_method_t::HTTP_METHOD_MKCOL (C++ enumerator), 123
 esp_http_client_method_t::HTTP_METHOD_MOVE (C++ enumerator), 122
 esp_http_client_method_t::HTTP_METHOD_NOTIFY (C++ enumerator), 122
 esp_http_client_method_t::HTTP_METHOD_OPTIONS (C++ enumerator), 122
 esp_http_client_method_t::HTTP_METHOD_PATCH (C++ enumerator), 122
 esp_http_client_method_t::HTTP_METHOD_POST (C++ enumerator), 122
 esp_http_client_method_t::HTTP_METHOD_PROBE (C++ enumerator), 123
 esp_http_client_method_t::HTTP_METHOD_PROPPATCH (C++ enumerator), 123
 esp_http_client_method_t::HTTP_METHOD_PUT (C++ enumerator), 122
 esp_http_client_method_t::HTTP_METHOD_REDIRECT (C++ enumerator), 123
 esp_http_client_method_t::HTTP_METHOD_SUBSCRIBE (C++ enumerator), 122
 esp_http_client_method_t::HTTP_METHOD_UNLOCK (C++ enumerator), 123
 esp_http_client_method_t::HTTP_METHOD_UNSUBSCRIBE (C++ enumerator), 122
 esp_http_client_on_data (C++ struct), 137
 esp_http_client_on_data::client (C++ member), 137
 esp_http_client_on_data::data_process (C++ member), 137
 esp_http_client_on_data_t (C++ type), 120
 esp_http_client_open (C++ function), 131
 esp_http_client_perform (C++ function), 126
 esp_http_client_prepare (C++ function), 126
 esp_http_client_proto_ver_t (C++ enum), 121
 esp_http_client_proto_ver_t::ESP_HTTP_CLIENT_TLS_V1 (C++ enumerator), 121
 esp_http_client_proto_ver_t::ESP_HTTP_CLIENT_TLS_V1_1 (C++ enumerator), 121
 esp_http_client_proto_ver_t::ESP_HTTP_CLIENT_TLS_V1_2 (C++ enumerator), 121
 esp_http_client_proto_ver_t::ESP_HTTP_CLIENT_TLS_V1_3 (C++ enumerator), 121
 esp_http_client_read (C++ function), 132
 esp_http_client_read_response (C++ function), 135
 esp_http_client_redirect_event_data (C++ struct), 137
 esp_http_client_redirect_event_data::client (C++ member), 137
 esp_http_client_redirect_event_data::status_code (C++ member), 137
 esp_http_client_redirect_event_data_t (C++ type), 120
 esp_http_client_request_send (C++ function), 126
 esp_http_client_reset_redirect_counter (C++ function), 134
 esp_http_client_set_auth_data (C++ function), 134
 esp_http_client_set_auth_type (C++ function), 129
 esp_http_client_set_header (C++ function), 128
 esp_http_client_set_method (C++ function), 131
 esp_http_client_set_password (C++ function), 129
 esp_http_client_set_post_field (C++ function), 127
 esp_http_client_set_redirection (C++

<i>function</i>), 134	gpio_config_t::pull_up_en (C++ member), 151
esp_http_client_set_timeout_ms (C++ function), 131	gpio_get_level (C++ function), 150
esp_http_client_set_url (C++ function), 127	gpio_install_isr_service (C++ function), 149
esp_http_client_set_user_data (C++ function), 130	GPIO_IS_VALID_GPIO (C macro), 148
esp_http_client_set_username (C++ function), 128	GPIO_IS_VALID_OUTPUT_GPIO (C macro), 148
esp_http_client_tls_dyn_buf_strategy_t (C++ enum), 123	gpio_isr_handler_add (C++ function), 148
esp_http_client_tls_dyn_buf_strategy_t::HTTP_TLSDYN_BUF_STRATEGY_DEFAULT (C++ enumerator), 124	gpio_isr_handler_remove (C++ function), 148
esp_http_client_tls_dyn_buf_strategy_t::HTTP_TLSDYN_BUF_STRATEGY_MAX (C++ enumerator), 124	gpio_isr_t (C++ type), 148
esp_http_client_transport_t (C++ enum), 121	GPIO_PIN_COUNT (C macro), 148
esp_http_client_transport_t::HTTP_TRANSPORT_OVER_SSL (C++ enumerator), 121	gpio_set_direction (C++ function), 150
esp_http_client_transport_t::HTTP_TRANSPORT_OVER_TCP (C++ enumerator), 121	GPIO_PIN_COUNT (C macro), 148
esp_http_client_transport_t::HTTP_TRANSPORT_UNKNOWN (C++ enumerator), 121	gpio_set_level (C++ function), 150
esp_http_client_write (C++ function), 132	gpio_set_pull_mode (C++ function), 151
esp_http_state_t (C++ enum), 124	http_event_handle_cb (C++ type), 120
esp_http_state_t::HTTP_STATE_CLOSE (C++ enumerator), 124	HttpStatus_Code (C++ enum), 124
esp_http_state_t::HTTP_STATE_CONNECTED (C++ enumerator), 124	HttpStatus_Code::HttpStatus_BadRequest (C++ enumerator), 125
esp_http_state_t::HTTP_STATE_CONNECTING (C++ enumerator), 124	HttpStatus_Code::HttpStatus_Forbidden (C++ enumerator), 125
esp_http_state_t::HTTP_STATE_INIT (C++ enumerator), 124	HttpStatus_Code::HttpStatus_Found (C++ enumerator), 125
esp_http_state_t::HTTP_STATE_REQ_COMPLETE_DATA (C++ enumerator), 124	HttpStatus_Code::HttpStatus_InternalServerError (C++ enumerator), 125
esp_http_state_t::HTTP_STATE_REQ_COMPLETE_HEADER (C++ enumerator), 124	HttpStatus_Code::HttpStatus_MovedPermanently (C++ enumerator), 125
esp_http_state_t::HTTP_STATE_RES_COMPLETE_DATA (C++ enumerator), 124	HttpStatus_Code::HttpStatus_MultipleChoices (C++ enumerator), 125
esp_http_state_t::HTTP_STATE_RES_COMPLETE_HEADER (C++ enumerator), 124	HttpStatus_Code::HttpStatus_NotFound (C++ enumerator), 125
esp_http_state_t::HTTP_STATE_RES_ON_DATA_START (C++ enumerator), 124	HttpStatus_Code::HttpStatus_NotModified (C++ enumerator), 125
esp_http_state_t::HTTP_STATE_UNINIT (C++ enumerator), 124	HttpStatus_Code::HttpStatus_Ok (C++ enumerator), 125
G	HttpStatus_Code::HttpStatus_PartialContent (C++ enumerator), 125
gpio_config (C++ function), 149	HttpStatus_Code::HttpStatus_PermanentRedirect (C++ enumerator), 125
gpio_config_t (C++ struct), 151	HttpStatus_Code::HttpStatus_RangeNotSatisfiable (C++ enumerator), 125
gpio_config_t::intr_type (C++ member), 152	HttpStatus_Code::HttpStatus_SeeOther (C++ enumerator), 125
gpio_config_t::mode (C++ member), 151	HttpStatus_Code::HttpStatus_TemporaryRedirect (C++ enumerator), 125
gpio_config_t::pin_bit_mask (C++ member), 151	HttpStatus_Code::HttpStatus_Unauthorized (C++ enumerator), 125
gpio_config_t::pull_down_en (C++ member), 151	I
	inph_80211_tx_info_t (C++ type), 26
	INPH_EVENT_DECLARE_BASE (C++ function), 125
	inph_event_handler_t (C++ type), 81

inph_event_loop_handle_t (C++ type), 81	inph_mqtt5_disconnect_property_config_t::disconnect
inph_mqtt5_client_delete_user_property	(C++ member), 107
(C++ function), 103	inph_mqtt5_disconnect_property_config_t::session_ex
inph_mqtt5_client_get_user_property	(C++ member), 107
(C++ function), 103	inph_mqtt5_disconnect_property_config_t::user_prope
inph_mqtt5_client_get_user_property_count	(C++ member), 107
(C++ function), 103	inph_mqtt5_error_reason_code_t (C++ type),
inph_mqtt5_client_handle_t (C++ type), 99	99
inph_mqtt5_client_set_connect_property	inph_mqtt5_event_property_t (C++ struct),
(C++ function), 101	107
inph_mqtt5_client_set_disconnect_property	inph_mqtt5_event_property_t::content_type
(C++ function), 102	(C++ member), 107
inph_mqtt5_client_set_publish_property	inph_mqtt5_event_property_t::content_type_len
(C++ function), 101	(C++ member), 107
inph_mqtt5_client_set_subscribe_property	inph_mqtt5_event_property_t::correlation_data
(C++ function), 102	(C++ member), 107
inph_mqtt5_client_set_unsubscribe_property	inph_mqtt5_event_property_t::correlation_data_len
(C++ function), 102	(C++ member), 107
inph_mqtt5_client_set_user_property	inph_mqtt5_event_property_t::payload_format_indicat
(C++ function), 103	(C++ member), 107
inph_mqtt5_connection_property_config_t	inph_mqtt5_event_property_t::response_topic
(C++ struct), 104	(C++ member), 107
inph_mqtt5_connection_property_config_t::inph_mqtt5_type	inph_mqtt5_event_property_t::response_topic_len
(C++ member), 104	(C++ member), 107
inph_mqtt5_connection_property_config_t::inph_mqtt5_event_data	inph_mqtt5_event_property_t::subscribe_id
(C++ member), 104	(C++ member), 107
inph_mqtt5_connection_property_config_t::inph_mqtt5_event_data	inph_mqtt5_event_property_t::user_property
(C++ member), 105	(C++ member), 107
inph_mqtt5_connection_property_config_t::inph_mqtt5_packet_size	inph_mqtt5_publish_property_config_t
(C++ member), 104	(C++ struct), 105
inph_mqtt5_connection_property_config_t::inph_mqtt5_publish	inph_mqtt5_publish_property_config_t::content_type
(C++ member), 104	(C++ member), 105
inph_mqtt5_connection_property_config_t::inph_mqtt5_publish	inph_mqtt5_publish_property_config_t::correlation_c
(C++ member), 104	(C++ member), 105
inph_mqtt5_connection_property_config_t::inph_mqtt5_publish	inph_mqtt5_publish_property_config_t::correlation_c
(C++ member), 104	(C++ member), 105
inph_mqtt5_connection_property_config_t::inph_mqtt5_publish	inph_mqtt5_publish_property_config_t::message_expi
(C++ member), 104	(C++ member), 105
inph_mqtt5_connection_property_config_t::inph_mqtt5_publish	inph_mqtt5_publish_property_config_t::payload_forma
(C++ member), 104	(C++ member), 105
inph_mqtt5_connection_property_config_t::inph_mqtt5_publish	inph_mqtt5_publish_property_config_t::response_top
(C++ member), 104	(C++ member), 105
inph_mqtt5_connection_property_config_t::inph_mqtt5_publish	inph_mqtt5_publish_property_config_t::topic_alias
(C++ member), 104	(C++ member), 105
inph_mqtt5_connection_property_config_t::inph_mqtt5_publish	inph_mqtt5_publish_property_config_t::user_property
(C++ member), 104	(C++ member), 105
inph_mqtt5_connection_property_config_t::inph_mqtt5_publish	inph_mqtt5_subscribe_property_config_t
(C++ member), 104	(C++ struct), 105
inph_mqtt5_connection_property_config_t::inph_mqtt5_publish	inph_mqtt5_subscribe_property_config_t::is_share_s
(C++ member), 104	(C++ member), 106
inph_mqtt5_connection_property_config_t::inph_mqtt5_publish	inph_mqtt5_subscribe_property_config_t::no_local_f
(C++ member), 105	(C++ member), 106
inph_mqtt5_disconnect_property_config_t	inph_mqtt5_subscribe_property_config_t::retain_as_p
(C++ struct), 106	(C++ member), 106

© 2025 InnoPhase IoT, Inc. | All Rights Reserved | Confidential 160 of 177

inph_mqtt_client_config_t::network_t	(C++ struct), 96	inph_mqtt_client_config_t::task_t::priority	(C++ member), 99
inph_mqtt_client_config_t::network_t::disable_mqtt_create_connection	(C++ member), 97	inph_mqtt_client_config_t::task_t::stack_size	(C++ member), 99
inph_mqtt_client_config_t::network_t::if_inph_mqtt_client_destroy	(C++ member), 97	(C++ function), 89	
inph_mqtt_client_config_t::network_t::reconnect_timeout_ms	(C++ member), 97	inph_mqtt_client_disconnect	(C++ function), 86
inph_mqtt_client_config_t::network_t::refresh_connection_get_outbox_size	(C++ member), 97	(C++ function), 90	
inph_mqtt_client_config_t::network_t::tcp_keepalive_interval	(C++ member), 97	inph_mqtt_client_enqueue	(C++ function), 88
inph_mqtt_client_config_t::network_t::timeouts	(C++ member), 97	inph_mqtt_client_handle_t	(C++ type), 81
inph_mqtt_client_config_t::network_t::transport	(C++ member), 97	inph_mqtt_client_init	(C++ function), 86
inph_mqtt_client_config_t::outbox	(C++ member), 93	inph_mqtt_client_publish	(C++ function), 88
inph_mqtt_client_config_t::outbox_config_t	(C++ struct), 97	inph_mqtt_client_reconnect	(C++ function), 86
inph_mqtt_client_config_t::outbox_config_t	(C++ member), 97	inph_mqtt_client_register_event	(C++ function), 89
inph_mqtt_client_config_t::session	(C++ member), 93	inph_mqtt_client_set_uri	(C++ function), 86
inph_mqtt_client_config_t::session_t	(C++ struct), 97	inph_mqtt_client_start	(C++ function), 86
inph_mqtt_client_config_t::session_t::disable_session	(C++ member), 98	inph_mqtt_client_stop	(C++ function), 86
inph_mqtt_client_config_t::session_t::disable_keepalive	(C++ member), 98	inph_mqtt_client_subscribe	(C macro), 81
inph_mqtt_client_config_t::session_t::keepalive	(C++ member), 98	inph_mqtt_client_subscribe_multiple	(C++ function), 87
inph_mqtt_client_config_t::session_t::last_will	(C++ member), 98	inph_mqtt_client_subscribe_single	(C++ function), 87
inph_mqtt_client_config_t::session_t::last_will	(C++ member), 98	inph_mqtt_client_unregister_event	(C++ function), 89
inph_mqtt_client_config_t::session_t::last_will	(C++ member), 98	inph_mqtt_client_unsubscribe	(C++ function), 87
inph_mqtt_client_config_t::session_t::last_will	(C++ member), 98	inph_mqtt_connect_return_code_t	(C++ enum), 84
inph_mqtt_client_config_t::session_t::last_will	(C++ member), 98	inph_mqtt_connect_return_code_t	(C++ enum), 84
inph_mqtt_client_config_t::session_t::last_will	(C++ member), 98	inph_mqtt_connect_return_code_t::MQTT_CONNECTION_ACCEPTED	
inph_mqtt_client_config_t::session_t::last_will	(C++ member), 98	inph_mqtt_connect_return_code_t::MQTT_CONNECTION_REFUSED	
inph_mqtt_client_config_t::session_t::last_will	(C++ member), 98	inph_mqtt_connect_return_code_t::MQTT_CONNECTION_REFUSED_BAD_USER_IDENTIFIER	
inph_mqtt_client_config_t::session_t::last_will	(C++ member), 98	inph_mqtt_connect_return_code_t::MQTT_CONNECTION_REFUSED_BAD_PROTOCOL_VERSION	
inph_mqtt_client_config_t::session_t::last_will	(C++ member), 98	inph_mqtt_connect_return_code_t::MQTT_CONNECTION_REFUSED_IDENTIFIER_REJECTED	
inph_mqtt_client_config_t::session_t::last_will	(C++ member), 98	inph_mqtt_connect_return_code_t::MQTT_CONNECTION_REFUSED_SERVER_SHUTTING_DOWN	
inph_mqtt_client_config_t::session_t::message	(C++ member), 98	inph_mqtt_dispatch_custom_event	(C++ function), 90
inph_mqtt_client_config_t::session_t::protocol	(C++ member), 98	inph_mqtt_error_codes	(C++ struct), 90
inph_mqtt_client_config_t::task	(C++ member), 93	inph_mqtt_error_codes::connect_return_code	(C++ member), 91
inph_mqtt_client_config_t::task_t	(C++ struct), 98	inph_mqtt_error_codes::error_type	(C++ member), 91

<code>inph_mqtt_error_codes::inph_tls_cert_verify_flag</code> (<i>C++ member</i>), 92	
<code>(C++ member), 91</code>	<code>inph_mqtt_event_t::event_id</code> (<i>C++ member</i>),
<code>inph_mqtt_error_codes::inph_tls_last_inph_err</code> 91	
<code>(C++ member), 91</code>	<code>inph_mqtt_event_t::msg_id</code> (<i>C++ member</i>), 92
<code>inph_mqtt_error_codes::inph_tls_stack_err</code>	<code>inph_mqtt_event_t::protocol_ver</code> (<i>C++</i>
<code>(C++ member), 91</code>	<code>member), 92</code>
<code>inph_mqtt_error_codes::inph_transport_sock</code>	<code>inph_mqtt_event_t::qos</code> (<i>C++ member</i>), 92
<code>(C++ member), 91</code>	<code>inph_mqtt_event_t::retain</code> (<i>C++ member</i>), 92
<code>inph_mqtt_error_codes_t</code> (<i>C++ type</i>), 82	<code>inph_mqtt_event_t::session_present</code> (<i>C++</i>
<code>inph_mqtt_error_type_t</code> (<i>C++ enum</i>), 84	<code>member), 92</code>
<code>inph_mqtt_error_type_t</code> (<i>C++ type</i>), 82	<code>inph_mqtt_event_t::topic</code> (<i>C++ member</i>), 92
<code>inph_mqtt_error_type_t::MQTT_ERROR_TYPE_CONNECTION_REFUSED</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ mem-</i>
<code>(C++ enumerator), 85</code>	<code>ber), 92</code>
<code>inph_mqtt_error_type_t::MQTT_ERROR_TYPE_NONE</code>	<code>inph_mqtt_event_t::total_data_len</code> (<i>C++</i>
<code>(C++ enumerator), 85</code>	<code>member), 92</code>
<code>inph_mqtt_error_type_t::MQTT_ERROR_TYPE_SUBSCRIBE_FAILED</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ enum</i>), 85
<code>(C++ enumerator), 85</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>inph_mqtt_error_type_t::MQTT_ERROR_TYPE_TRANSPORT</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>(C++ enumerator), 85</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>inph_mqtt_event_handle_t</code> (<i>C++ type</i>), 82	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>inph_mqtt_event_id_t</code> (<i>C++ enum</i>), 83	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>inph_mqtt_event_id_t</code> (<i>C++ type</i>), 81	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>inph_mqtt_event_id_t::MQTT_EVENT_ANY</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>(C++ enumerator), 83</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>inph_mqtt_event_id_t::MQTT_EVENT_BEFORE_CONNECT</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>(C++ enumerator), 84</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>inph_mqtt_event_id_t::MQTT_EVENT_CONNECTED</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>(C++ enumerator), 83</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>inph_mqtt_event_id_t::MQTT_EVENT_DATA</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>(C++ enumerator), 83</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>inph_mqtt_event_id_t::MQTT_EVENT_DELETED</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>(C++ enumerator), 84</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>inph_mqtt_event_id_t::MQTT_EVENT_DISCONNECTED</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>(C++ enumerator), 83</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>inph_mqtt_event_id_t::MQTT_EVENT_ERROR</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>(C++ enumerator), 83</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>inph_mqtt_event_id_t::MQTT_EVENT_PUBLISHED</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>(C++ enumerator), 83</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>inph_mqtt_event_id_t::MQTT_EVENT_SUBSCRIBED</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>(C++ enumerator), 83</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>inph_mqtt_event_id_t::MQTT_EVENT_UNSUBSCRIBED</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>(C++ enumerator), 83</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>inph_mqtt_event_id_t::MQTT_USER_EVENT</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>(C++ enumerator), 84</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>inph_mqtt_event_t</code> (<i>C++ struct</i>), 91	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>inph_mqtt_event_t</code> (<i>C++ type</i>), 82	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>inph_mqtt_event_t::client</code> (<i>C++ member</i>), 91	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>inph_mqtt_event_t::current_data_offset</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>(C++ member), 92</code>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>inph_mqtt_event_t::data</code> (<i>C++ member</i>), 91	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>inph_mqtt_event_t::data_len</code> (<i>C++ member</i>),	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
91	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>inph_mqtt_event_t::dup</code> (<i>C++ member</i>), 92	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82
<code>inph_mqtt_event_t::error_handle</code> (<i>C++</i>	<code>inph_mqtt_event_t::topic_len</code> (<i>C++ type</i>), 82

inph_sntp_config::start (C++ member), 111
 inph_sntp_config::sync_cb (C++ member), 111
 inph_sntp_config::wait_for_sync (C++ member), 111
 inph_sntp_enabled (C++ function), 115
 inph_sntp_get_sync_interval (C macro), 112
 inph_sntp_get_sync_mode (C macro), 111
 inph_sntp_get_sync_status (C macro), 111
 inph_sntp_getoperatingmode (C++ function), 115
 inph_sntp_getreachability (C++ function), 115
 inph_sntp_getserver (C++ function), 115
 inph_sntp_getservername (C++ function), 114
 inph_sntp_init (C++ function), 114
 inph_sntp_operatingmode_t (C++ enum), 112
 inph_sntp_operatingmode_t::INPH_Sntp_OPMODE_LISTENONLY (C++ enumerator), 113
 inph_sntp_operatingmode_t::INPH_Sntp_OPMODE_POLL (C++ enumerator), 113
 inph_sntp_restart (C macro), 112
 inph_sntp_set_sync_interval (C macro), 112
 inph_sntp_set_sync_mode (C macro), 111
 inph_sntp_set_sync_status (C macro), 111
 inph_sntp_set_time_sync_notification_cb (C macro), 111
 inph_sntp_setoperatingmode (C++ function), 114
 inph_sntp_setserver (C++ function), 114
 inph_sntp_setservername (C++ function), 114
 inph_sntp_stop (C++ function), 114
 inph_sntp_sync_time (C macro), 111
 inph_wifi_clear_ap_list (C++ function), 77
 inph_wifi_clear_default_wifi_driver_and_handlers (C++ function), 78
 inph_wifi_connect (C++ function), 75
 inph_wifi_deinit (C++ function), 73
 inph_wifi_destroy_if_driver (C++ function), 79
 inph_wifi_disconnect (C++ function), 76
 inph_wifi_get_if_mac (C++ function), 79
 inph_wifi_get_ps (C++ function), 78
 inph_wifi_init (C++ function), 73
 INPH_WIFI_MAX_FILTER_LEN (C macro), 24
 INPH_WIFI_MAX_FUP_SSI_LEN (C macro), 24
 INPH_WIFI_MAX_NEIGHBOR_REP_LEN (C macro), 24
 INPH_WIFI_MAX_SVC_INFO_LEN (C macro), 24
 INPH_WIFI_MAX_SVC_NAME_LEN (C macro), 24
 INPH_WIFI_MAX_SVC_SSI_LEN (C macro), 24
 inph_wifi_mode_t (C++ enum), 26
 inph_wifi_mode_t::WIFI_MODE_AP (C++ enumerator), 26
 inph_wifi_mode_t::WIFI_MODE_APSTA (C++ enumerator), 26
 inph_wifi_mode_t::WIFI_MODE_MAX (C++ enumerator), 26
 inph_wifi_mode_t::WIFI_MODE_NAN (C++ enumerator), 26
 inph_wifi_mode_t::WIFI_MODE_NULL (C++ enumerator), 26
 inph_wifi_mode_t::WIFI_MODE_STA (C++ enumerator), 26
 INPH_WIFI_NAN_DATAPATH_MAX_PEERS (C macro), 24
 INPH_WIFI_NAN_MAX_SVC_SUPPORTED (C macro), 24
 INPH_WIFI_NDP_ROLE_INITIATOR (C macro), 24
 INPH_WIFI_NDP_ROLE_RESPONDER (C macro), 24
 inph_wifi_register_if_rxcb (C++ function), 79
 inph_wifi_scan_get_ap_num (C++ function), 77
 inph_wifi_scan_get_ap_records (C++ function), 77
 inph_wifi_scan_start (C++ function), 76
 inph_wifi_scan_stop (C++ function), 76
 inph_wifi_set_config (C++ function), 74
 inph_wifi_set_default_wifi_ap_handlers (C++ function), 78
 inph_wifi_set_default_wifi_nan_handlers (C++ function), 78
 inph_wifi_set_default_wifi_sta_handlers (C++ function), 78
 inph_wifi_set_mode (C++ function), 74
 inph_wifi_set_ps (C++ function), 78
 inph_wifi_start (C++ function), 73
 inph_wifi_stop (C++ function), 74

M

MAX_PASSPHRASE_LEN (C macro), 25
 MAX_SSID_LEN (C macro), 24
 MAX_WPS_AP_CRED (C macro), 25
 mqtt5_error_reason_code_t (C++ enum), 99
 mqtt5_error_reason_code_t::__attribute__ (C++ enumerator), 100, 101
 mqtt5_error_reason_code_t::MQTT5_ADMINISTRATIVE_ACTION (C++ enumerator), 101
 mqtt5_error_reason_code_t::MQTT5_BAD_AUTH_METHOD (C++ enumerator), 100
 mqtt5_error_reason_code_t::MQTT5_BAD_USERNAME_OR_PASSWORD (C++ enumerator), 100
 mqtt5_error_reason_code_t::MQTT5_BANNED (C++ enumerator), 100
 mqtt5_error_reason_code_t::MQTT5_CONNECTION_RATE_EXCEEDED (C++ enumerator), 101
 mqtt5_error_reason_code_t::MQTT5_IMPLEMENT_SPECIFIC (C++ enumerator), 99

mqtt5_error_reason_code_t::MQTT5_INVALID_IDENTIFIER_reason_code_t::MQTT5_USE_ANOTHER_SERVER
 (C++ enumerator), 100 (C++ enumerator), 101
 mqtt5_error_reason_code_t::MQTT5_KEEP_ALIVE_TIMEOUT_reason_code_t::MQTT5_WILDCARD_SUBSCRIBE
 (C++ enumerator), 100 (C++ enumerator), 101
 mqtt5_error_reason_code_t::MQTT5_MALFORMED_PACKET_property_handle_t (C++ type), 99
 (C++ enumerator), 99 MQTT_ERROR_TYPE_INPH_TLS (C macro), 81
 mqtt5_error_reason_code_t::MQTT5_MAXIMUM_QOS_CONVERTING_SCHEME (C macro), 81
 (C++ enumerator), 101 MQTT_OVER_TCP_SCHEME (C macro), 81
 mqtt5_error_reason_code_t::MQTT5_MESSAGE_RATE_OVER_WS_SCHEME (C macro), 81
 (C++ enumerator), 101 MQTT_OVER_WSS_SCHEME (C macro), 81
 mqtt5_error_reason_code_t::MQTT5_NOT_AUTHORIZED
 (C++ enumerator), 100

S

 mqtt5_error_reason_code_t::MQTT5_PACKET_IDENTIFIER_IN_USE_SNTF_IDENTIFIER_LEN (C macro), 22
 (C++ enumerator), 100 sntp_get_sync_interval (C++ function), 114
 mqtt5_error_reason_code_t::MQTT5_PACKET_IDENTIFIER_NOT_FOUND (C++ function), 113
 (C++ enumerator), 100 sntp_get_sync_mode (C++ function), 113
 mqtt5_error_reason_code_t::MQTT5_PACKET_TOO_LARGE_Sntp_OP_MODE_POLL (C macro), 112
 (C++ enumerator), 100 sntp_restart (C++ function), 114
 mqtt5_error_reason_code_t::MQTT5_PAYLOAD_FORMAT_INVALID sntp_set_sync_interval (C++ function), 114
 (C++ enumerator), 101 sntp_set_sync_mode (C++ function), 113
 mqtt5_error_reason_code_t::MQTT5_PROTOCOL_ERROR sntp_set_sync_status (C++ function), 113
 (C++ enumerator), 99 sntp_set_time_sync_notification_cb (C++
 mqtt5_error_reason_code_t::MQTT5_QOS_NOT_SUPPORTED function), 113
 (C++ enumerator), 101 sntp_sync_mode_t (C++ enum), 112
 mqtt5_error_reason_code_t::MQTT5_QUOTA_EXCEEDED sntp_sync_mode_t::SNTP_SYNC_MODE_IMMED
 (C++ enumerator), 101 (C++ enumerator), 112
 mqtt5_error_reason_code_t::MQTT5_RECEIVE_MAXIMUM_EXCEEDED sntp_sync_mode_t::SNTP_SYNC_MODE_SMOOTH
 (C++ enumerator), 100 (C++ enumerator), 112
 mqtt5_error_reason_code_t::MQTT5_RETAIN_NOT_SUPPORTED sntp_sync_status_t (C++ enum), 112
 (C++ enumerator), 101 sntp_sync_status_t::SNTP_SYNC_STATUS_COMPLETED
 mqtt5_error_reason_code_t::MQTT5_SERVER_BUSY (C++ enumerator), 112
 (C++ enumerator), 100 sntp_sync_status_t::SNTP_SYNC_STATUS_IN_PROGRESS
 mqtt5_error_reason_code_t::MQTT5_SERVER_MOVED (C++ enumerator), 112
 (C++ enumerator), 101 sntp_sync_status_t::SNTP_SYNC_STATUS_RESET
 mqtt5_error_reason_code_t::MQTT5_SERVER_SHUTTING_DOWN (C++ enumerator), 112
 (C++ enumerator), 100 sntp_sync_time (C++ function), 113
 mqtt5_error_reason_code_t::MQTT5_SERVER_UNAVAILABLE sntp_sync_time_cb_t (C++ type), 112
 (C++ enumerator), 100 system_notify_msg_code (C++ enum), 27
 mqtt5_error_reason_code_t::MQTT5_SESSION_TAKEN_OVER system_notify_msg_code::SYSTEM_NOTIFY_MSG_AP_NOT_F
 (C++ enumerator), 100 (C++ enumerator), 27
 mqtt5_error_reason_code_t::MQTT5_SHARED_SUBSCRIBE_NOT_SUPPORTED system_notify_msg_code::SYSTEM_NOTIFY_MSG_PASSPHRASE
 (C++ enumerator), 101 (C++ enumerator), 27
 mqtt5_error_reason_code_t::MQTT5_SUBSCRIBE_IDENTIFIER_NOT_SUPPORT
 (C++ enumerator), 101

T

 mqtt5_error_reason_code_t::MQTT5_TOPIC_ALIAS_INVALID topic_t (C++ struct), 99
 (C++ enumerator), 100 topic_t::filter (C++ member), 99
 mqtt5_error_reason_code_t::MQTT5_TOPIC_FILTER_INVALID topic_t::qos (C++ member), 99
 (C++ enumerator), 100

U

 mqtt5_error_reason_code_t::MQTT5_TOPIC_NAME_INVALID
 (C++ enumerator), 100
 mqtt5_error_reason_code_t::MQTT5_UNSPECIFIED_ERROR uart_bitrate_detect_config_t (C++ struct),
 (C++ enumerator), 99 147
 mqtt5_error_reason_code_t::MQTT5_UNSUPPORTED_PROTOCOL_VERSION uart_bitrate_detect_config_t::rx_io_num
 (C++ enumerator), 99 (C++ member), 147

uart_bitrate_detect_config_t::source_clk (C++ member), 147
 uart_bitrate_res_t (C++ struct), 147
 uart_bitrate_res_t::clk_freq_hz (C++ member), 147
 uart_bitrate_res_t::neg_period (C++ member), 147
 uart_bitrate_res_t::pos_period (C++ member), 147
 uart_clear_intr_status (C++ function), 145
 uart_config_t (C++ struct), 146
 uart_config_t::allow_pd (C++ member), 146
 uart_config_t::backup_before_sleep (C++ member), 146
 uart_config_t::baud_rate (C++ member), 146
 uart_config_t::data_bits (C++ member), 146
 uart_config_t::flags (C++ member), 146
 uart_config_t::flow_ctrl (C++ member), 146
 uart_config_t::parity (C++ member), 146
 uart_config_t::rx_flow_ctrl_thresh (C++ member), 146
 uart_config_t::source_clk (C++ member), 146
 uart_config_t::stop_bits (C++ member), 146
 uart_detect_bitrate_start (C++ function), 146
 uart_detect_bitrate_stop (C++ function), 146
 uart_disable_intr_mask (C++ function), 145
 uart_disable_pattern_det_intr (C++ function), 145
 uart_disable_rx_intr (C++ function), 145
 uart_disable_tx_intr (C++ function), 145
 uart_driver_delete (C++ function), 144
 uart_driver_install (C++ function), 144
 uart_enable_intr_mask (C++ function), 145
 uart_enable_pattern_det_baud_intr (C++ function), 145
 uart_enable_rx_intr (C++ function), 145
 uart_enable_tx_intr (C++ function), 145
 uart_event_t (C++ struct), 147
 uart_event_t::size (C++ member), 147
 uart_event_t::timeout_flag (C++ member), 147
 uart_event_t::type (C++ member), 147
 uart_event_type_t (C++ enum), 143
 uart_event_type_t::UART_BREAK (C++ enumerator), 143
 uart_event_type_t::UART_BUFFER_FULL (C++ enumerator), 143
 uart_event_type_t::UART_DATA (C++ enumerator), 143
 uart_event_type_t::UART_DATA_BREAK (C++ enumerator), 143
 uart_event_type_t::UART_EVENT_MAX (C++ enumerator), 144
 uart_event_type_t::UART_FIFO_OVF (C++ enumerator), 143
 uart_event_type_t::UART_FRAME_ERR (C++ enumerator), 143
 uart_event_type_t::UART_PARITY_ERR (C++ enumerator), 143
 uart_event_type_t::UART_PATTERN_DET (C++ enumerator), 144
 uart_event_type_t::UART_WAKEUP (C++ enumerator), 144
 uart_flush (C++ function), 145
 uart_flush_input (C++ function), 145
 uart_get_baudrate (C++ function), 144
 uart_get_buffered_data_len (C++ function), 145
 uart_get_collision_flag (C++ function), 146
 uart_get_hw_flow_ctrl (C++ function), 144
 uart_get_parity (C++ function), 144
 uart_get_sclk_freq (C++ function), 144
 uart_get_selectlock (C++ function), 146
 uart_get_stop_bits (C++ function), 144
 uart_get_tx_buffer_free_size (C++ function), 145
 uart_get_wakeup_threshold (C++ function), 146
 uart_get_word_length (C++ function), 144
 UART_HW_FIFO_LEN (C macro), 141
 uart_hw_flowcontrol_t (C++ enum), 142
 uart_hw_flowcontrol_t::UART_HW_FLOWCTRL_CTS_RTS (C++ enumerator), 142
 uart_hw_flowcontrol_t::UART_HW_FLOWCTRL_DISABLE (C++ enumerator), 142
 uart_intr_config (C++ function), 145
 uart_intr_config_t (C++ struct), 146
 uart_intr_config_t::intr_enable_mask (C++ member), 147
 uart_intr_config_t::rx_timeout_thresh (C++ member), 147
 uart_intr_config_t::rxfifo_full_thresh (C++ member), 147
 uart_intr_config_t::txfifo_empty_intr_thresh (C++ member), 147
 uart_is_driver_installed (C++ function), 144
 uart_isr_handle_t (C++ type), 142
 uart_mode_t (C++ enum), 143
 uart_mode_t::UART_MODE_IRDA (C++ enumerator), 143
 uart_mode_t::UART_MODE_RS485 (C++ enumerator), 143
 uart_mode_t::UART_MODE_UART (C++ enumerator), 143
 UART_NUM_0 (C macro), 141
 UART_NUM_1 (C macro), 141

UART_NUM_2 (C macro), 141
 UART_NUM_MAX (C macro), 141
 uart_param_config (C++ function), 145
 uart_parity_t (C++ enum), 142
 uart_parity_t::UART_PARITY_DISABLE (C++ enumerator), 142
 uart_parity_t::UART_PARITY_EVEN (C++ enumerator), 142
 uart_parity_t::UART_PARITY_ODD (C++ enumerator), 142
 uart_pattern_get_pos (C++ function), 145
 uart_pattern_pop_pos (C++ function), 145
 uart_pattern_queue_reset (C++ function), 145
 UART_PIN_NO_CHANGE (C macro), 141
 uart_port_t (C++ type), 142
 uart_read_bytes (C++ function), 145
 uart_sclk_t (C++ enum), 143
 uart_sclk_t::UART_SCLK_DEFAULT (C++ enumerator), 143
 uart_select_notif_callback_t (C++ type), 142
 uart_select_notif_t (C++ enum), 144
 uart_select_notif_t::UART_SELECT_ERROR_NOTIFY (C++ enumerator), 144
 uart_select_notif_t::UART_SELECT_READ_NOTIFY (C++ enumerator), 144
 uart_select_notif_t::UART_SELECT_WRITE_NOTIFY (C++ enumerator), 144
 uart_set_always_rx_timeout (C++ function), 146
 uart_set_baudrate (C++ function), 144
 uart_set_dtr (C++ function), 145
 uart_set_hw_flow_ctrl (C++ function), 144
 uart_set_line_inverse (C++ function), 144
 uart_set_loop_back (C++ function), 146
 uart_set_mode (C++ function), 145
 uart_set_parity (C++ function), 144
 uart_set_pin (C++ function), 145
 uart_set_rts (C++ function), 145
 uart_set_rx_full_threshold (C++ function), 145
 uart_set_rx_timeout (C++ function), 146
 uart_set_select_notif_callback (C++ function), 146
 uart_set_stop_bits (C++ function), 144
 uart_set_sw_flow_ctrl (C++ function), 144
 uart_set_tx_empty_threshold (C++ function), 145
 uart_set_tx_idle_num (C++ function), 145
 uart_set_wakeup_threshold (C++ function), 146
 uart_set_word_length (C++ function), 144
 uart_signal_inv_t (C++ enum), 143
 uart_signal_inv_t::UART_SIGNAL_INV_CTS (C++ enumerator), 143
 uart_signal_inv_t::UART_SIGNAL_INV_DISABLE (C++ enumerator), 143
 uart_signal_inv_t::UART_SIGNAL_INV_RTS (C++ enumerator), 143
 uart_signal_inv_t::UART_SIGNAL_INV_RXD (C++ enumerator), 143
 uart_signal_inv_t::UART_SIGNAL_INV_TXD (C++ enumerator), 143
 uart_stop_bits_t (C++ enum), 142
 uart_stop_bits_t::UART_STOP_BITS_1 (C++ enumerator), 142
 uart_stop_bits_t::UART_STOP_BITS_2 (C++ enumerator), 142
 uart_tx_chars (C++ function), 145
 uart_wait_tx_done (C++ function), 145
 uart_wait_tx_idle_polling (C++ function), 146
 uart_word_length_t (C++ enum), 142
 uart_word_length_t::UART_DATA_5_BITS (C++ enumerator), 142
 uart_word_length_t::UART_DATA_6_BITS (C++ enumerator), 142
 uart_word_length_t::UART_DATA_7_BITS (C++ enumerator), 142
 uart_word_length_t::UART_DATA_8_BITS (C++ enumerator), 142
 uart_write_bytes (C++ function), 145
 uart_write_bytes_with_break (C++ function), 145

V

vendor_ie_data_t (C++ struct), 51
 vendor_ie_data_t::element_id (C++ member), 51
 vendor_ie_data_t::length (C++ member), 51
 vendor_ie_data_t::payload (C++ member), 51
 vendor_ie_data_t::vendor_oui (C++ member), 51
 vendor_ie_data_t::vendor_oui_type (C++ member), 51

W

wifi_2g_channel_bit_t (C++ enum), 34
 wifi_2g_channel_bit_t::WIFI_CHANNEL_1 (C++ enumerator), 34
 wifi_2g_channel_bit_t::WIFI_CHANNEL_2 (C++ enumerator), 34
 wifi_2g_channel_bit_t::WIFI_CHANNEL_3 (C++ enumerator), 34
 wifi_2g_channel_bit_t::WIFI_CHANNEL_4 (C++ enumerator), 34

wifi_2g_channel_bit_t::WIFI_CHANNEL_5 (C++ enumerator), 34	(C++ enumerator), 36
wifi_2g_channel_bit_t::WIFI_CHANNEL_6 (C++ enumerator), 34	wifi_5g_channel_bit_t::WIFI_CHANNEL_136 (C++ enumerator), 36
wifi_2g_channel_bit_t::WIFI_CHANNEL_7 (C++ enumerator), 34	wifi_5g_channel_bit_t::WIFI_CHANNEL_140 (C++ enumerator), 36
wifi_2g_channel_bit_t::WIFI_CHANNEL_8 (C++ enumerator), 34	wifi_5g_channel_bit_t::WIFI_CHANNEL_144 (C++ enumerator), 36
wifi_2g_channel_bit_t::WIFI_CHANNEL_9 (C++ enumerator), 34	wifi_5g_channel_bit_t::WIFI_CHANNEL_149 (C++ enumerator), 36
wifi_2g_channel_bit_t::WIFI_CHANNEL_10 (C++ enumerator), 34	wifi_5g_channel_bit_t::WIFI_CHANNEL_153 (C++ enumerator), 36
wifi_2g_channel_bit_t::WIFI_CHANNEL_11 (C++ enumerator), 34	wifi_5g_channel_bit_t::WIFI_CHANNEL_157 (C++ enumerator), 36
wifi_2g_channel_bit_t::WIFI_CHANNEL_12 (C++ enumerator), 34	wifi_5g_channel_bit_t::WIFI_CHANNEL_161 (C++ enumerator), 36
wifi_2g_channel_bit_t::WIFI_CHANNEL_13 (C++ enumerator), 35	wifi_5g_channel_bit_t::WIFI_CHANNEL_165 (C++ enumerator), 36
wifi_2g_channel_bit_t::WIFI_CHANNEL_14 (C++ enumerator), 35	wifi_5g_channel_bit_t::WIFI_CHANNEL_169 (C++ enumerator), 36
wifi_5g_channel_bit_t (C++ enum), 35	wifi_5g_channel_bit_t::WIFI_CHANNEL_173 (C++ enumerator), 36
wifi_5g_channel_bit_t::WIFI_CHANNEL_36 (C++ enumerator), 35	wifi_5g_channel_bit_t::WIFI_CHANNEL_177 (C++ enumerator), 37
wifi_5g_channel_bit_t::WIFI_CHANNEL_40 (C++ enumerator), 35	wifi_action_roc_done_cb_t (C++ type), 26
wifi_5g_channel_bit_t::WIFI_CHANNEL_44 (C++ enumerator), 35	wifi_action_rx_cb_t (C++ type), 25
wifi_5g_channel_bit_t::WIFI_CHANNEL_48 (C++ enumerator), 35	wifi_action_tx_req_t (C++ struct), 53
wifi_5g_channel_bit_t::WIFI_CHANNEL_52 (C++ enumerator), 35	wifi_action_tx_req_t::channel (C++ member), 53
wifi_5g_channel_bit_t::WIFI_CHANNEL_56 (C++ enumerator), 35	wifi_action_tx_req_t::data (C++ member), 53
wifi_5g_channel_bit_t::WIFI_CHANNEL_60 (C++ enumerator), 35	wifi_action_tx_req_t::data_len (C++ member), 53
wifi_5g_channel_bit_t::WIFI_CHANNEL_64 (C++ enumerator), 35	wifi_action_tx_req_t::dest_mac (C++ member), 53
wifi_5g_channel_bit_t::WIFI_CHANNEL_100 (C++ enumerator), 35	wifi_action_tx_req_t::ifx (C++ member), 53
wifi_5g_channel_bit_t::WIFI_CHANNEL_104 (C++ enumerator), 35	wifi_action_tx_req_t::no_ack (C++ member), 53
wifi_5g_channel_bit_t::WIFI_CHANNEL_108 (C++ enumerator), 35	wifi_action_tx_req_t::op_id (C++ member), 53
wifi_5g_channel_bit_t::WIFI_CHANNEL_112 (C++ enumerator), 35	wifi_action_tx_req_t::rx_cb (C++ member), 53
wifi_5g_channel_bit_t::WIFI_CHANNEL_116 (C++ enumerator), 36	wifi_action_tx_req_t::type (C++ member), 53
wifi_5g_channel_bit_t::WIFI_CHANNEL_120 (C++ enumerator), 36	wifi_action_tx_req_t::wait_time_ms (C++ member), 53
wifi_5g_channel_bit_t::WIFI_CHANNEL_124 (C++ enumerator), 36	wifi_action_tx_status_type_t (C++ enum), 45
wifi_5g_channel_bit_t::WIFI_CHANNEL_128 (C++ enumerator), 36	wifi_action_tx_status_type_t::WIFI_ACTION_TX_DONE (C++ enumerator), 45
wifi_5g_channel_bit_t::WIFI_CHANNEL_132 (C++ enumerator), 36	wifi_action_tx_status_type_t::WIFI_ACTION_TX_DURATION (C++ enumerator), 45
	wifi_action_tx_status_type_t::WIFI_ACTION_TX_FAILED (C++ enumerator), 45
	wifi_action_tx_status_type_t::WIFI_ACTION_TX_OP_CANCEL (C++ enumerator), 45

(C++ enumerator), 45
 wifi_action_tx_t (C++ enum), 26
 wifi_action_tx_t::WIFI_OFFCHAN_TX_CANCEL (C++ enumerator), 26
 wifi_action_tx_t::WIFI_OFFCHAN_TX_REQ (C++ enumerator), 26
 WIFI_ACTIVE_SCAN_MAX_DEFAULT_TIME (C macro), 21
 WIFI_ACTIVE_SCAN_MIN_DEFAULT_TIME (C macro), 21
 wifi_ant_config_t (C++ struct), 52
 wifi_ant_config_t::enabled_ant0 (C++ member), 52
 wifi_ant_config_t::enabled_ant1 (C++ member), 52
 wifi_ant_config_t::rx_ant_default (C++ member), 52
 wifi_ant_config_t::rx_ant_mode (C++ member), 52
 wifi_ant_config_t::tx_ant_mode (C++ member), 52
 wifi_ant_gpio_config_t (C++ struct), 52
 wifi_ant_gpio_config_t::gpio_cfg (C++ member), 52
 wifi_ant_gpio_t (C++ struct), 52
 wifi_ant_gpio_t::gpio_num (C++ member), 52
 wifi_ant_gpio_t::gpio_select (C++ member), 52
 wifi_ant_mode_t (C++ enum), 39
 wifi_ant_mode_t::WIFI_ANT_MODE_ANT0 (C++ enumerator), 39
 wifi_ant_mode_t::WIFI_ANT_MODE_ANT1 (C++ enumerator), 39
 wifi_ant_mode_t::WIFI_ANT_MODE_AUTO (C++ enumerator), 39
 wifi_ant_mode_t::WIFI_ANT_MODE_MAX (C++ enumerator), 39
 wifi_ap_config_t (C++ struct), 48
 wifi_ap_config_t::authmode (C++ member), 48
 wifi_ap_config_t::beacon_interval (C++ member), 48
 wifi_ap_config_t::bss_max_idle_cfg (C++ member), 49
 wifi_ap_config_t::channel (C++ member), 48
 wifi_ap_config_t::csa_count (C++ member), 48
 wifi_ap_config_t::dtim_period (C++ member), 48
 wifi_ap_config_t::ftm_responder (C++ member), 49
 wifi_ap_config_t::gtk_rekey_interval (C++ member), 49
 wifi_ap_config_t::max_connection (C++ member), 48
 wifi_ap_config_t::pairwise_cipher (C++ member), 49
 wifi_ap_config_t::password (C++ member), 48
 wifi_ap_config_t::pmf_cfg (C++ member), 49
 wifi_ap_config_t::sae_ext (C++ member), 49
 wifi_ap_config_t::sae_pwe_h2e (C++ member), 49
 wifi_ap_config_t::ssid (C++ member), 48
 wifi_ap_config_t::ssid_hidden (C++ member), 48
 wifi_ap_config_t::ssid_len (C++ member), 48
 wifi_ap_config_t::transition_disable (C++ member), 49
 WIFI_AP_DEFAULT_MAX_IDLE_PERIOD (C macro), 21
 wifi_band_mode_t (C++ enum), 33
 wifi_band_mode_t::WIFI_BAND_MODE_2G_ONLY (C++ enumerator), 33
 wifi_band_mode_t::WIFI_BAND_MODE_5G_ONLY (C++ enumerator), 34
 wifi_band_mode_t::WIFI_BAND_MODE_AUTO (C++ enumerator), 34
 wifi_band_t (C++ enum), 33
 wifi_band_t::WIFI_BAND_2G (C++ enumerator), 33
 wifi_band_t::WIFI_BAND_5G (C++ enumerator), 33
 wifi_bandwidth_t (C++ enum), 32
 wifi_bandwidth_t::WIFI_BW20 (C++ enumerator), 32
 wifi_bandwidth_t::WIFI_BW40 (C++ enumerator), 32
 wifi_bandwidth_t::WIFI_BW80 (C++ enumerator), 32
 wifi_bandwidth_t::WIFI_BW80_BW80 (C++ enumerator), 32
 wifi_bandwidth_t::WIFI_BW160 (C++ enumerator), 32
 wifi_bandwidth_t::WIFI_BW_HT20 (C++ enumerator), 32
 wifi_bandwidth_t::WIFI_BW_HT40 (C++ enumerator), 32
 wifi_bandwidths_t (C++ struct), 47
 wifi_bandwidths_t::ghz_2g (C++ member), 47
 wifi_bandwidths_t::ghz_5g (C++ member), 47
 wifi_bss_max_idle_config_t (C++ struct), 47
 wifi_bss_max_idle_config_t::period (C++ member), 48
 wifi_bss_max_idle_config_t::protected_keep_alive (C++ member), 48
 wifi_config_t (C++ union), 50

wifi_config_t::ap (C++ member), 50	(C++ enumerator), 28
wifi_config_t::nan (C++ member), 50	wifi_err_reason_t::WIFI_REASON_END_BA (C++ enumerator), 30
wifi_config_t::sta (C++ member), 50	wifi_err_reason_t::WIFI_REASON_EXCEEDED_TXOP (C++ enumerator), 30
wifi_csi_info_t (C++ type), 25	wifi_err_reason_t::WIFI_REASON_GROUP_CIPHER_INVALID (C++ enumerator), 29
wifi_err_reason_t (C++ enum), 27	wifi_err_reason_t::WIFI_REASON_GROUP_KEY_UPDATE_TIMEOUT (C++ enumerator), 29
wifi_err_reason_t::WIFI_REASON_4WAY_HANDSHAKE_TIMEOUT (C++ enumerator), 29	wifi_err_reason_t::WIFI_REASON_HANDSHAKE_TIMEOUT (C++ enumerator), 31
wifi_err_reason_t::WIFI_REASON_802_1X_AUTH_FAILED (C++ enumerator), 29	wifi_err_reason_t::WIFI_REASON_IE_IN_4WAY_DIFFERS (C++ enumerator), 29
wifi_err_reason_t::WIFI_REASON_AKMP_INVALID (C++ enumerator), 29	wifi_err_reason_t::WIFI_REASON_IE_INVALID (C++ enumerator), 29
wifi_err_reason_t::WIFI_REASON_ALTERNATIVE_CHANNEL_OCCUPIED (C++ enumerator), 31	wifi_err_reason_t::WIFI_REASON_INVALID_FT_ACTION_FRAME (C++ enumerator), 31
wifi_err_reason_t::WIFI_REASON_AP_INITIATED (C++ enumerator), 30	wifi_err_reason_t::WIFI_REASON_INVALID_FTE (C++ enumerator), 31
wifi_err_reason_t::WIFI_REASON_AP_TSF_RESET (C++ enumerator), 31	wifi_err_reason_t::WIFI_REASON_INVALID_MDE (C++ enumerator), 31
wifi_err_reason_t::WIFI_REASON_ASSOC_COMEBACK_TIMEOUT (C++ enumerator), 31	wifi_err_reason_t::WIFI_REASON_INVALID_PMKID (C++ enumerator), 31
wifi_err_reason_t::WIFI_REASON_ASSOC_EXPIRED (C++ enumerator), 28	wifi_err_reason_t::WIFI_REASON_INVALID_RSN_IE_CAP (C++ enumerator), 29
wifi_err_reason_t::WIFI_REASON_ASSOC_FAILED (C++ enumerator), 31	wifi_err_reason_t::WIFI_REASON_MIC_FAILURE (C++ enumerator), 29
wifi_err_reason_t::WIFI_REASON_ASSOC_LEAVE (C++ enumerator), 28	wifi_err_reason_t::WIFI_REASON_MISSING_ACKS (C++ enumerator), 30
wifi_err_reason_t::WIFI_REASON_ASSOC_NOT_AUTHED (C++ enumerator), 28	wifi_err_reason_t::WIFI_REASON_NO_AP_FOUND (C++ enumerator), 31
wifi_err_reason_t::WIFI_REASON_ASSOC_TOO_MANY (C++ enumerator), 28	wifi_err_reason_t::WIFI_REASON_NO_AP_FOUND_IN_AUTHM (C++ enumerator), 32
wifi_err_reason_t::WIFI_REASON_AUTH_EXPIRED (C++ enumerator), 28	wifi_err_reason_t::WIFI_REASON_NO_AP_FOUND_IN_RSSI (C++ enumerator), 32
wifi_err_reason_t::WIFI_REASON_AUTH_FAILED (C++ enumerator), 31	wifi_err_reason_t::WIFI_REASON_NO_AP_FOUND_W_COMPAT (C++ enumerator), 32
wifi_err_reason_t::WIFI_REASON_AUTH_LEAVE (C++ enumerator), 28	wifi_err_reason_t::WIFI_REASON_NO_SSP_ROAMING_AGREED (C++ enumerator), 30
wifi_err_reason_t::WIFI_REASON_BAD_CIPHER_OR_AKM (C++ enumerator), 30	wifi_err_reason_t::WIFI_REASON_NOT_ASSOCED (C++ enumerator), 28
wifi_err_reason_t::WIFI_REASON_BEACON_TIMEOUT (C++ enumerator), 31	wifi_err_reason_t::WIFI_REASON_NOT_AUTHED (C++ enumerator), 28
wifi_err_reason_t::WIFI_REASON_BSS_TRANSITION_DISASSOC (C++ enumerator), 29	wifi_err_reason_t::WIFI_REASON_NOT_AUTHORIZED_THIS_STA (C++ enumerator), 30
wifi_err_reason_t::WIFI_REASON_CIPHER_SUITE_REJECTED (C++ enumerator), 29	wifi_err_reason_t::WIFI_REASON_NOT_ENOUGH_BANDWIDTH (C++ enumerator), 30
wifi_err_reason_t::WIFI_REASON_CLASS2_FRAME_FROM_NONAUTH_STA (C++ enumerator), 28	wifi_err_reason_t::WIFI_REASON_PAIRWISE_CIPHER_INVALID (C++ enumerator), 29
wifi_err_reason_t::WIFI_REASON_CLASS3_FRAME_FROM_NONASSOC_STA (C++ enumerator), 28	wifi_err_reason_t::WIFI_REASON_PEER_INITIATED (C++ enumerator), 30
wifi_err_reason_t::WIFI_REASON_CONNECTION_FAILED (C++ enumerator), 31	wifi_err_reason_t::WIFI_REASON_ROAMING (C++ enumerator), 31
wifi_err_reason_t::WIFI_REASON_DISASSOC_DUE_TO_INACTIVITY (C++ enumerator), 28	wifi_err_reason_t::WIFI_REASON_SA_QUERY_TIMEOUT (C++ enumerator), 31
wifi_err_reason_t::WIFI_REASON_DISASSOC_DUE_TO_CAPABAD (C++ enumerator), 28	
wifi_err_reason_t::WIFI_REASON_DISASSOC_SUPCHAN_BAD (C++ enumerator), 28	

(C++ enumerator), 32	(C++ member), 61
wifi_err_reason_t::WIFI_REASON_SERVICE_CHANGE	wifi_event_sta_disconnected_t::mac
(C++ enumerator), 30	(C++ member), 61
wifi_err_reason_t::WIFI_REASON_SSP_REQUESTED_BY_STA	wifi_event_sta_disconnected_t::reason
(C++ enumerator), 30	(C++ member), 61
wifi_err_reason_t::WIFI_REASON_STA_LEAVING	wifi_event_ap_wps_rg_fail_reason_t (C++ struct), 65
(C++ enumerator), 30	
wifi_err_reason_t::WIFI_REASON_TDLS_PEER_WUNREACHABLE	wifi_event_ap_wps_rg_fail_reason_t::peer_macaddr
(C++ enumerator), 29	(C++ member), 65
wifi_err_reason_t::WIFI_REASON_TDLS_UNSPECIFIED	wifi_event_ap_wps_rg_fail_reason_t::reason
(C++ enumerator), 29	(C++ member), 65
wifi_err_reason_t::WIFI_REASON_TIMEOUT	wifi_event_ap_wps_rg_pin_t (C++ struct), 64
(C++ enumerator), 30	wifi_event_ap_wps_rg_pin_t::pin_code
wifi_err_reason_t::WIFI_REASON_TRANSMISSION_LINK_FAILURE	(C++ member), 65
(C++ enumerator), 31	wifi_event_ap_wps_rg_success_t (C++ struct), 65
wifi_err_reason_t::WIFI_REASON_UNKNOWN_BA	wifi_event_ap_wps_rg_success_t::peer_macaddr
(C++ enumerator), 30	(C++ member), 65
wifi_err_reason_t::WIFI_REASON_UNSPECIFIED	wifi_event_ap_wrong_password_t (C++ struct), 70
(C++ enumerator), 28	wifi_event_ap_wrong_password_t::mac
wifi_err_reason_t::WIFI_REASON_UNSPECIFIED_QOS	(C++ member), 70
(C++ enumerator), 30	wifi_event_bss_rssi_low_t (C++ struct), 61
wifi_err_reason_t::WIFI_REASON_UNSUPP_RSN_IE_VERSION	wifi_event_bss_rssi_low_t::rssi (C++ member), 62
(C++ enumerator), 29	
wifi_event_action_tx_status_t (C++ struct), 63	wifi_event_dpp_config_received_t (C++ struct), 72
wifi_event_action_tx_status_t::channel	wifi_event_dpp_config_received_t::wifi_cfg
(C++ member), 64	(C++ member), 73
wifi_event_action_tx_status_t::context	wifi_event_dpp_failed_t (C++ struct), 73
(C++ member), 64	wifi_event_dpp_failed_t::failure_reason
wifi_event_action_tx_status_t::ifx (C++ member), 64	(C++ member), 73
wifi_event_action_tx_status_t::op_id	wifi_event_dpp_uri_ready_t (C++ struct), 72
(C++ member), 64	wifi_event_dpp_uri_ready_t::uri (C++ member), 72
wifi_event_action_tx_status_t::status	
(C++ member), 64	wifi_event_dpp_uri_ready_t::uri_data_len
wifi_event_ap_probe_req_rx_t (C++ struct), 61	(C++ member), 72
wifi_event_ap_probe_req_rx_t::mac (C++ member), 61	wifi_event_ftm_report_t (C++ struct), 63
wifi_event_ap_probe_req_rx_t::rssi (C++ member), 61	wifi_event_ftm_report_t::dist_est (C++ member), 63
wifi_event_ap_staconnected_t (C++ struct), 60	wifi_event_ftm_report_t::ftm_report_data
wifi_event_ap_staconnected_t::aid (C++ member), 61	(C++ member), 63
wifi_event_ap_staconnected_t::is_mesh_child	wifi_event_ftm_report_t::ftm_report_num_entries
(C++ member), 61	(C++ member), 63
wifi_event_ap_staconnected_t::mac (C++ member), 61	wifi_event_ftm_report_t::peer_mac (C++ member), 63
wifi_event_ap_stadisconnected_t (C++ struct), 61	wifi_event_ftm_report_t::rtt_est (C++ member), 63
wifi_event_ap_stadisconnected_t::aid	wifi_event_ftm_report_t::rtt_raw (C++ member), 63
(C++ member), 61	wifi_event_ftm_report_t::status (C++ member), 63
wifi_event_ap_stadisconnected_t::is_mesh_child	wifi_event_home_channel_change_t (C++ struct), 62

wifi_event_home_channel_change_t::new_chan	(C++ member), 65	wifi_event_nan_svc_match_t::reserved	
(C++ member), 62		wifi_event_nan_svc_match_t::reserved_1	
wifi_event_home_channel_change_t::new_snd	(C++ member), 66	wifi_event_nan_svc_match_t::reserved_2	
(C++ member), 62		wifi_event_nan_svc_match_t::ssi	(C++ member), 66
wifi_event_home_channel_change_t::old_chan	(C++ member), 66	wifi_event_nan_svc_match_t::ssi_len	(C++ member), 66
(C++ member), 62		wifi_event_nan_svc_match_t::ssi_version	(C++ member), 66
wifi_event_home_channel_change_t::old_snd	(C++ member), 66	wifi_event_nan_svc_match_t::subscribe_id	(C++ member), 65
(C++ member), 62		wifi_event_nan_svc_match_t::update_pub_id	(C++ member), 65
WIFI_EVENT_MASK_ALL (C macro), 24		wifi_event_ndp_confirm_t (C++ struct), 68	
WIFI_EVENT_MASK_AP_PROBEREQRECVD (C macro), 24		wifi_event_ndp_confirm_t::ndp_id	(C++ member), 68
WIFI_EVENT_MASK_NONE (C macro), 24		wifi_event_ndp_confirm_t::own_ndi	(C++ member), 68
wifi_event_nan_receive_t (C++ struct), 67		wifi_event_ndp_confirm_t::peer_ndi	(C++ member), 68
wifi_event_nan_receive_t::inst_id	(C++ member), 67	wifi_event_ndp_confirm_t::peer_nmi	(C++ member), 68
wifi_event_nan_receive_t::peer_if_mac	(C++ member), 67	wifi_event_ndp_confirm_t::reserved_1	(C++ member), 69
wifi_event_nan_receive_t::peer_inst_id	(C++ member), 67	wifi_event_ndp_confirm_t::reserved_2	(C++ member), 69
wifi_event_nan_receive_t::peer_svc_info	(C++ member), 67	wifi_event_ndp_confirm_t::ssi	(C++ member), 69
wifi_event_nan_receive_t::reserved_1	(C++ member), 67	wifi_event_ndp_confirm_t::ssi_len	(C++ member), 69
wifi_event_nan_receive_t::reserved_2	(C++ member), 67	wifi_event_ndp_confirm_t::status	(C++ member), 68
wifi_event_nan_receive_t::ssi	(C++ member), 67	wifi_event_ndp_confirm_t::svc_info	(C++ member), 69
wifi_event_nan_receive_t::ssi_len	(C++ member), 67	wifi_event_ndp_indication_t (C++ struct), 67	
wifi_event_nan_replied_t (C++ struct), 66		wifi_event_ndp_indication_t::ndp_id	(C++ member), 68
wifi_event_nan_replied_t::publish_id	(C++ member), 66	wifi_event_ndp_indication_t::peer_ndi	(C++ member), 68
wifi_event_nan_replied_t::reserved_1	(C++ member), 66	wifi_event_ndp_indication_t::peer_nmi	(C++ member), 68
wifi_event_nan_replied_t::reserved_2	(C++ member), 66	wifi_event_ndp_indication_t::publish_id	(C++ member), 68
wifi_event_nan_replied_t::ssi	(C++ member), 67	wifi_event_ndp_indication_t::reserved_1	(C++ member), 68
wifi_event_nan_replied_t::ssi_len	(C++ member), 66	wifi_event_ndp_indication_t::reserved_2	(C++ member), 68
wifi_event_nan_replied_t::sub_if_mac	(C++ member), 66	wifi_event_ndp_indication_t::ssi	(C++ member), 68
wifi_event_nan_replied_t::subscribe_id	(C++ member), 66	wifi_event_ndp_indication_t::ssi_len	
wifi_event_nan_svc_match_t (C++ struct), 65			
wifi_event_nan_svc_match_t::datapath_reqd	(C++ member), 65		
wifi_event_nan_svc_match_t::fsd_gas	(C++ member), 66		
wifi_event_nan_svc_match_t::fsd_reqd	(C++ member), 65		
wifi_event_nan_svc_match_t::pub_if_mac	(C++ member), 65		
wifi_event_nan_svc_match_t::publish_id			

(C++ member), 68
 wifi_event_ndp_indication_t::svc_info
 (C++ member), 68
 wifi_event_ndp_terminated_t (C++ struct), 69
 wifi_event_ndp_terminated_t::init_ndi
 (C++ member), 69
 wifi_event_ndp_terminated_t::ndp_id
 (C++ member), 69
 wifi_event_ndp_terminated_t::reason
 (C++ member), 69
 wifi_event_neighbor_report_t (C++ struct),
 69
 wifi_event_neighbor_report_t::n_report
 (C++ member), 69
 wifi_event_neighbor_report_t::report
 (C++ member), 69
 wifi_event_neighbor_report_t::report_len
 (C++ member), 69
 wifi_event_roc_done_t (C++ struct), 64
 wifi_event_roc_done_t::channel (C++ mem-
 ber), 64
 wifi_event_roc_done_t::context (C++ mem-
 ber), 64
 wifi_event_roc_done_t::op_id (C++ mem-
 ber), 64
 wifi_event_roc_done_t::status (C++ mem-
 ber), 64
 wifi_event_sta_authmode_change_t (C++
 struct), 59
 wifi_event_sta_authmode_change_t::new_mode
 (C++ member), 60
 wifi_event_sta_authmode_change_t::old_mode
 (C++ member), 60
 wifi_event_sta_beacon_offset_unstable_t
 (C++ struct), 72
 wifi_event_sta_beacon_offset_unstable_t::beacon
 (C++ member), 72
 wifi_event_sta_connected_t (C++ struct), 58
 wifi_event_sta_connected_t::aid (C++
 member), 59
 wifi_event_sta_connected_t::authmode
 (C++ member), 59
 wifi_event_sta_connected_t::bssid (C++
 member), 59
 wifi_event_sta_connected_t::channel
 (C++ member), 59
 wifi_event_sta_connected_t::ssid (C++
 member), 59
 wifi_event_sta_connected_t::ssid_len
 (C++ member), 59
 wifi_event_sta_disconnected_t (C++ struct),
 59
 wifi_event_sta_disconnected_t::bssid
 (C++ member), 59
 wifi_event_sta_disconnected_t::reason
 (C++ member), 59
 wifi_event_sta_disconnected_t::rssi
 (C++ member), 59
 wifi_event_sta_disconnected_t::ssid
 (C++ member), 59
 wifi_event_sta_disconnected_t::ssid_len
 (C++ member), 59
 wifi_event_sta_scan_done_t (C++ struct), 58
 wifi_event_sta_scan_done_t::number (C++
 member), 58
 wifi_event_sta_scan_done_t::scan_id
 (C++ member), 58
 wifi_event_sta_scan_done_t::status (C++
 member), 58
 wifi_event_sta_wps_er_pin_t (C++ struct), 60
 wifi_event_sta_wps_er_pin_t::pin_code
 (C++ member), 60
 wifi_event_sta_wps_er_success_t (C++
 struct), 60
 wifi_event_sta_wps_er_success_t::ap_cred
 (C++ member), 60
 wifi_event_sta_wps_er_success_t::ap_cred_cnt
 (C++ member), 60
 wifi_event_sta_wps_er_success_t::passphrase
 (C++ member), 60
 wifi_event_sta_wps_er_success_t::ssid
 (C++ member), 60
 wifi_event_sta_wps_fail_reason_t (C++
 enum), 44
 wifi_event_sta_wps_fail_reason_t::WPS_FAIL_REASON_M
 (C++ enumerator), 44
 wifi_event_sta_wps_fail_reason_t::WPS_FAIL_REASON_M
 (C++ enumerator), 44
 wifi_event_sta_wps_fail_reason_t::WPS_FAIL_REASON_I
 (C++ enumerator), 44
 wifi_event_sta_wps_fail_reason_t::WPS_FAIL_REASON_I
 (C++ enumerator), 44
 wifi_event_sta_wps_fail_reason_t::WPS_FAIL_REASON_I
 (C++ enumerator), 44
 wifi_ftm_initiator_cfg_t (C++ struct), 54
 wifi_ftm_initiator_cfg_t::burst_period
 (C++ member), 54
 wifi_ftm_initiator_cfg_t::channel (C++
 member), 54
 wifi_ftm_initiator_cfg_t::frm_count
 (C++ member), 54
 wifi_ftm_initiator_cfg_t::rinph_mac
 (C++ member), 54
 wifi_ftm_initiator_cfg_t::use_get_report_api
 (C++ member), 54
 wifi_ftm_report_entry_t (C++ struct), 62
 wifi_ftm_report_entry_t::dlog_token
 (C++ member), 62
 wifi_ftm_report_entry_t::rssi (C++ mem-
 ber), 62

wifi_ftm_report_entry_t::rtt (C++ member), 62	(C++ member), 58
wifi_ftm_report_entry_t::t1 (C++ member), 62	wifi_nan_followup_params_t (C++ struct), 57
wifi_ftm_report_entry_t::t2 (C++ member), 62	wifi_nan_followup_params_t::inst_id (C++ member), 57
wifi_ftm_report_entry_t::t3 (C++ member), 63	wifi_nan_followup_params_t::peer_inst_id (C++ member), 57
wifi_ftm_report_entry_t::t4 (C++ member), 63	wifi_nan_followup_params_t::peer_mac (C++ member), 57
wifi_ftm_status_t (C++ enum), 44	wifi_nan_followup_params_t::ssi (C++ member), 57
wifi_ftm_status_t::FTM_STATUS_CONF_REJECTED (C++ enumerator), 45	wifi_nan_followup_params_t::ssi_len (C++ member), 57
wifi_ftm_status_t::FTM_STATUS_FAIL (C++ enumerator), 45	wifi_nan_followup_params_t::svc_info (C++ member), 57
wifi_ftm_status_t::FTM_STATUS_NO_RESPONSE (C++ enumerator), 45	wifi_nan_publish_cfg_t (C++ struct), 55
wifi_ftm_status_t::FTM_STATUS_NO_VALID_MSM (C++ enumerator), 45	wifi_nan_publish_cfg_t::datapath_reqd (C++ member), 55
wifi_ftm_status_t::FTM_STATUS_SUCCESS (C++ enumerator), 44	wifi_nan_publish_cfg_t::fsd_gas (C++ member), 55
wifi_ftm_status_t::FTM_STATUS_UNSUPPORTED (C++ enumerator), 44	wifi_nan_publish_cfg_t::fsd_reqd (C++ member), 55
wifi_ftm_status_t::FTM_STATUS_USER_TERM (C++ enumerator), 45	wifi_nan_publish_cfg_t::matching_filter (C++ member), 55
WIFI_MAX_REGULATORY_RULE_NUM (C macro), 25	wifi_nan_publish_cfg_t::reserved (C++ member), 56
WIFI_MAX_SUPPORT_COUNTRY_NUM (C macro), 25	wifi_nan_publish_cfg_t::service_name (C++ member), 55
wifi_nan_config_t (C++ struct), 49	wifi_nan_publish_cfg_t::single_replied_event (C++ member), 55
wifi_nan_config_t::master_pref (C++ member), 49	wifi_nan_publish_cfg_t::ssi (C++ member), 56
wifi_nan_config_t::op_channel (C++ member), 49	wifi_nan_publish_cfg_t::ssi_len (C++ member), 56
wifi_nan_config_t::scan_time (C++ member), 49	wifi_nan_publish_cfg_t::svc_info (C++ member), 55
wifi_nan_config_t::warm_up_sec (C++ member), 49	wifi_nan_publish_cfg_t::type (C++ member), 55
wifi_nan_datapath_end_req_t (C++ struct), 58	wifi_nan_service_type_t (C++ enum), 40
wifi_nan_datapath_end_req_t::ndp_id (C++ member), 58	wifi_nan_service_type_t::NAN_PUBLISH_SOLICITED (C++ enumerator), 40
wifi_nan_datapath_end_req_t::peer_mac (C++ member), 58	wifi_nan_service_type_t::NAN_PUBLISH_UNSOLICITED (C++ enumerator), 40
wifi_nan_datapath_req_t (C++ struct), 57	wifi_nan_service_type_t::NAN_SUBSCRIBE_ACTIVE (C++ enumerator), 40
wifi_nan_datapath_req_t::confirm_required (C++ member), 57	wifi_nan_service_type_t::NAN_SUBSCRIBE_PASSIVE (C++ enumerator), 40
wifi_nan_datapath_req_t::peer_mac (C++ member), 57	wifi_nan_subscribe_cfg_t (C++ struct), 56
wifi_nan_datapath_req_t::pub_id (C++ member), 57	wifi_nan_subscribe_cfg_t::datapath_reqd (C++ member), 56
wifi_nan_datapath_rinph_t (C++ struct), 57	wifi_nan_subscribe_cfg_t::fsd_gas (C++ member), 56
wifi_nan_datapath_rinph_t::accept (C++ member), 58	wifi_nan_subscribe_cfg_t::fsd_reqd (C++ member), 56
wifi_nan_datapath_rinph_t::ndp_id (C++ member), 58	wifi_nan_subscribe_cfg_t::matching_filter
wifi_nan_datapath_rinph_t::peer_mac	

<code>wifi_nan_subscribe_cfg_t::reserved</code> (C++ member), 56	<code>wifi_phy_rate_t::WIFI_PHY_RATE_2M_L</code> (C++ enumerator), 42
<code>wifi_nan_subscribe_cfg_t::service_name</code> (C++ member), 56	<code>wifi_phy_rate_t::WIFI_PHY_RATE_2M_S</code> (C++ enumerator), 42
<code>wifi_nan_subscribe_cfg_t::single_match_enable</code> (C++ member), 56	<code>wifi_phy_rate_t::WIFI_PHY_RATE_5M_L</code> (C++ enumerator), 42
<code>wifi_nan_subscribe_cfg_t::ssi</code> (C++ member), 57	<code>wifi_phy_rate_t::WIFI_PHY_RATE_5M_S</code> (C++ enumerator), 42
<code>wifi_nan_subscribe_cfg_t::ssi_len</code> (C++ member), 56	<code>wifi_phy_rate_t::WIFI_PHY_RATE_6M</code> (C++ enumerator), 42
<code>wifi_nan_subscribe_cfg_t::svc_info</code> (C++ member), 56	<code>wifi_phy_rate_t::WIFI_PHY_RATE_9M</code> (C++ enumerator), 42
<code>wifi_nan_subscribe_cfg_t::type</code> (C++ member), 56	<code>wifi_phy_rate_t::WIFI_PHY_RATE_11M_L</code> (C++ enumerator), 42
<code>wifi_nan_svc_proto_t</code> (C++ enum), 40	<code>wifi_phy_rate_t::WIFI_PHY_RATE_11M_S</code> (C++ enumerator), 42
<code>wifi_nan_svc_proto_t::WIFI_SVC_PROTO_BONJOUR</code> (C++ enumerator), 40	<code>wifi_phy_rate_t::WIFI_PHY_RATE_12M</code> (C++ enumerator), 42
<code>wifi_nan_svc_proto_t::WIFI_SVC_PROTO_CSA_MATTER</code> (C++ enumerator), 40	<code>wifi_phy_rate_t::WIFI_PHY_RATE_18M</code> (C++ enumerator), 42
<code>wifi_nan_svc_proto_t::WIFI_SVC_PROTO_GENERIC</code> (C++ enumerator), 40	<code>wifi_phy_rate_t::WIFI_PHY_RATE_24M</code> (C++ enumerator), 42
<code>wifi_nan_svc_proto_t::WIFI_SVC_PROTO_MAX</code> (C++ enumerator), 40	<code>wifi_phy_rate_t::WIFI_PHY_RATE_36M</code> (C++ enumerator), 42
<code>wifi_nan_svc_proto_t::WIFI_SVC_PROTO_RESERVED</code> (C++ enumerator), 40	<code>wifi_phy_rate_t::WIFI_PHY_RATE_48M</code> (C++ enumerator), 42
<code>wifi_nan_wfa_ssi_t</code> (C++ struct), 55	<code>wifi_phy_rate_t::WIFI_PHY_RATE_54M</code> (C++ enumerator), 42
<code>wifi_nan_wfa_ssi_t::payload</code> (C++ member), 55	<code>wifi_phy_rate_t::WIFI_PHY_RATE_LORA_250K</code> (C++ enumerator), 44
<code>wifi_nan_wfa_ssi_t::proto</code> (C++ member), 55	<code>wifi_phy_rate_t::WIFI_PHY_RATE_LORA_500K</code> (C++ enumerator), 44
<code>wifi_nan_wfa_ssi_t::wfa_oui</code> (C++ member), 55	<code>wifi_phy_rate_t::WIFI_PHY_RATE_MAX</code> (C++ enumerator), 44
<code>WIFI_OUI_LEN</code> (C macro), 24	<code>wifi_phy_rate_t::WIFI_PHY_RATE_MCS0_LGI</code> (C++ enumerator), 42
<code>WIFI_PASSIVE_SCAN_DEFAULT_TIME</code> (C macro), 21	<code>wifi_phy_rate_t::WIFI_PHY_RATE_MCS0_SGI</code> (C++ enumerator), 43
<code>wifi_phy_mode_t</code> (C++ enum), 38	<code>wifi_phy_rate_t::WIFI_PHY_RATE_MCS1_LGI</code> (C++ enumerator), 43
<code>wifi_phy_mode_t::WIFI_PHY_MODE_11A</code> (C++ enumerator), 38	<code>wifi_phy_rate_t::WIFI_PHY_RATE_MCS1_SGI</code> (C++ enumerator), 43
<code>wifi_phy_mode_t::WIFI_PHY_MODE_11B</code> (C++ enumerator), 38	<code>wifi_phy_rate_t::WIFI_PHY_RATE_MCS2_LGI</code> (C++ enumerator), 43
<code>wifi_phy_mode_t::WIFI_PHY_MODE_11G</code> (C++ enumerator), 38	<code>wifi_phy_rate_t::WIFI_PHY_RATE_MCS2_SGI</code> (C++ enumerator), 43
<code>wifi_phy_mode_t::WIFI_PHY_MODE_HE20</code> (C++ enumerator), 39	<code>wifi_phy_rate_t::WIFI_PHY_RATE_MCS3_LGI</code> (C++ enumerator), 43
<code>wifi_phy_mode_t::WIFI_PHY_MODE_HT20</code> (C++ enumerator), 38	<code>wifi_phy_rate_t::WIFI_PHY_RATE_MCS3_SGI</code> (C++ enumerator), 43
<code>wifi_phy_mode_t::WIFI_PHY_MODE_HT40</code> (C++ enumerator), 38	<code>wifi_phy_rate_t::WIFI_PHY_RATE_MCS4_LGI</code> (C++ enumerator), 43
<code>wifi_phy_mode_t::WIFI_PHY_MODE_LR</code> (C++ enumerator), 38	<code>wifi_phy_rate_t::WIFI_PHY_RATE_MCS4_SGI</code>
<code>wifi_phy_mode_t::WIFI_PHY_MODE_VHT20</code> (C++ enumerator), 39	
<code>wifi_phy_rate_t</code> (C++ enum), 40	
<code>wifi_phy_rate_t::WIFI_PHY_RATE_1M_L</code>	

<code>wifi_phy_rate_t::WIFI_PHY_RATE_MCS5_LGI</code> (C++ enumerator), 43	<code>WIFI_PROMIS_CTRL_FILTER_MASK_RTS</code> (C macro), 23
<code>wifi_phy_rate_t::WIFI_PHY_RATE_MCS5_SGI</code> (C++ enumerator), 43	<code>WIFI_PROMIS_CTRL_FILTER_MASK_WRAPPER</code> (C macro), 23
<code>wifi_phy_rate_t::WIFI_PHY_RATE_MCS6_LGI</code> (C++ enumerator), 43	<code>WIFI_PROMIS_FILTER_MASK_ALL</code> (C macro), 22
<code>wifi_phy_rate_t::WIFI_PHY_RATE_MCS6_SGI</code> (C++ enumerator), 44	<code>WIFI_PROMIS_FILTER_MASK_CTRL</code> (C macro), 22
<code>wifi_phy_rate_t::WIFI_PHY_RATE_MCS7_LGI</code> (C++ enumerator), 43	<code>WIFI_PROMIS_FILTER_MASK_DATA</code> (C macro), 23
<code>wifi_phy_rate_t::WIFI_PHY_RATE_MCS7_SGI</code> (C++ enumerator), 44	<code>WIFI_PROMIS_FILTER_MASK_DATA_AMPDU</code> (C macro), 23
<code>wifi_phy_rate_t::WIFI_PHY_RATE_MCS8_LGI</code> (C++ enumerator), 43	<code>WIFI_PROMIS_FILTER_MASK_DATA_MPDU</code> (C macro), 23
<code>wifi_phy_rate_t::WIFI_PHY_RATE_MCS8_SGI</code> (C++ enumerator), 44	<code>WIFI_PROMIS_FILTER_MASK_FCSFAIL</code> (C macro), 23
<code>wifi_phy_rate_t::WIFI_PHY_RATE_MCS9_LGI</code> (C++ enumerator), 43	<code>WIFI_PROMIS_FILTER_MASK_MGMT</code> (C macro), 22
<code>wifi_phy_rate_t::WIFI_PHY_RATE_MCS9_SGI</code> (C++ enumerator), 44	<code>WIFI_PROMIS_FILTER_MASK_MISC</code> (C macro), 23
<code>wifi_pm_flags</code> (C++ enum), 27	<code>wifi_promiscuous_filter_t</code> (C++ struct), 51
<code>wifi_pm_flags::WIFI_PM_DYN_LISTEN_INT</code> (C++ enumerator), 27	<code>wifi_promiscuous_filter_t::filter_mask</code> (C++ member), 52
<code>wifi_pm_flags::WIFI_PM_MCAST_DONT_CARE</code> (C++ enumerator), 27	<code>wifi_promiscuous_pkt_type_t</code> (C++ enum), 39
<code>wifi_pm_flags::WIFI_PM_PS_POLL</code> (C++ enumerator), 27	<code>wifi_promiscuous_pkt_type_t::WIFI_PKT_CTRL</code> (C++ enumerator), 39
<code>wifi_pm_flags::WIFI_PM_STA_ONLY_BROADCAST</code> (C++ enumerator), 27	<code>wifi_promiscuous_pkt_type_t::WIFI_PKT_DATA</code> (C++ enumerator), 39
<code>wifi_pm_flags::WIFI_PM_STA_RX_NAP</code> (C++ enumerator), 27	<code>wifi_promiscuous_pkt_type_t::WIFI_PKT_MGMT</code> (C++ enumerator), 39
<code>wifi_pm_flags::WIFI_PM_TX_PS</code> (C++ enumerator), 27	<code>wifi_promiscuous_pkt_type_t::WIFI_PKT_MISC</code> (C++ enumerator), 39
<code>wifi_pmf_config_t</code> (C++ struct), 47	<code>WIFI_PROTOCOL_11A</code> (C macro), 22
<code>wifi_pmf_config_t::capable</code> (C++ member), 47	<code>WIFI_PROTOCOL_11AC</code> (C macro), 22
<code>wifi_pmf_config_t::required</code> (C++ member), 47	<code>WIFI_PROTOCOL_11AX</code> (C macro), 22
<code>WIFI_PROMIS_CTRL_FILTER_MASK_ACK</code> (C macro), 23	<code>WIFI_PROTOCOL_11B</code> (C macro), 22
<code>WIFI_PROMIS_CTRL_FILTER_MASK_ALL</code> (C macro), 23	<code>WIFI_PROTOCOL_11G</code> (C macro), 22
<code>WIFI_PROMIS_CTRL_FILTER_MASK_BA</code> (C macro), 23	<code>WIFI_PROTOCOL_11N</code> (C macro), 22
<code>WIFI_PROMIS_CTRL_FILTER_MASK_BAR</code> (C macro), 23	<code>WIFI_PROTOCOL_LR</code> (C macro), 22
<code>WIFI_PROMIS_CTRL_FILTER_MASK_CFEND</code> (C macro), 23	<code>wifi_protocols_t</code> (C++ struct), 46
<code>WIFI_PROMIS_CTRL_FILTER_MASK_CFENDACK</code> (C macro), 23	<code>wifi_protocols_t::ghz_2g</code> (C++ member), 47
<code>WIFI_PROMIS_CTRL_FILTER_MASK_CTS</code> (C macro), 23	<code>wifi_protocols_t::ghz_5g</code> (C++ member), 47
<code>WIFI_PROMIS_CTRL_FILTER_MASK_PSPOLL</code> (C macro), 23	<code>wifi_ps_type_t</code> (C++ enum), 33
	<code>wifi_ps_type_t::WIFI_PS_MAX_MODEM</code> (C++ enumerator), 33
	<code>wifi_ps_type_t::WIFI_PS_MIN_MODEM</code> (C++ enumerator), 33
	<code>wifi_ps_type_t::WIFI_PS_NONE</code> (C++ enumerator), 33
	<code>wifi_reg_rule_t</code> (C++ struct), 70
	<code>wifi_reg_rule_t::end_channel</code> (C++ member), 70
	<code>wifi_reg_rule_t::is_dfs</code> (C++ member), 71
	<code>wifi_reg_rule_t::max_bandwidth</code> (C++ member), 70
	<code>wifi_reg_rule_t::max_eirp</code> (C++ member), 70
	<code>wifi_reg_rule_t::reserved</code> (C++ member), 71

`wifi_reg_rule_t::start_channel` (C++ member), 70
`wifi_regdomain_t` (C++ struct), 71
`wifi_regdomain_t::cn` (C++ member), 71
`wifi_regdomain_t::regulatory_type` (C++ member), 71
`wifi_regulatory_t` (C++ struct), 71
`wifi_regulatory_t::n_reg_rules` (C++ member), 71
`wifi_regulatory_t::reg_rules` (C++ member), 71
`wifi_roc_done_status_t` (C++ enum), 39
`wifi_roc_done_status_t::WIFI_ROC_DONE` (C++ enumerator), 39
`wifi_roc_done_status_t::WIFI_ROC_FAIL` (C++ enumerator), 40
`wifi_roc_req_t` (C++ struct), 53
`wifi_roc_req_t::channel` (C++ member), 54
`wifi_roc_req_t::done_cb` (C++ member), 54
`wifi_roc_req_t::ifx` (C++ member), 54
`wifi_roc_req_t::op_id` (C++ member), 54
`wifi_roc_req_t::rx_cb` (C++ member), 54
`wifi_roc_req_t::sec_channel` (C++ member), 54
`wifi_roc_req_t::type` (C++ member), 54
`wifi_roc_req_t::wait_time_ms` (C++ member), 54
`wifi_roc_t` (C++ enum), 27
`wifi_roc_t::WIFI_ROC_CANCEL` (C++ enumerator), 27
`wifi_roc_t::WIFI_ROC_REQ` (C++ enumerator), 27
`wifi_sae_pk_mode_t` (C++ enum), 37
`wifi_sae_pk_mode_t::WPA3_SAE_PK_MODE_AUTOMATIC` (C++ enumerator), 37
`wifi_sae_pk_mode_t::WPA3_SAE_PK_MODE_DISABLED` (C++ enumerator), 37
`wifi_sae_pk_mode_t::WPA3_SAE_PK_MODE_ONLY` (C++ enumerator), 37
`wifi_sae_pwe_method_t` (C++ enum), 37
`wifi_sae_pwe_method_t::WPA3_SAE_PWE_BOTH` (C++ enumerator), 37
`wifi_sae_pwe_method_t::WPA3_SAE_PWE_HASH_TO_ELEMENT` (C++ enumerator), 37
`wifi_sae_pwe_method_t::WPA3_SAE_PWE_HUNT_AND_PEEK` (C++ enumerator), 37
`wifi_sae_pwe_method_t::WPA3_SAE_PWE_UNSPECIFIED` (C++ enumerator), 37
`wifi_scan_default_params_t` (C++ struct), 46
`wifi_scan_default_params_t::home_channel` (C++ member), 46
`wifi_scan_default_params_t::scan_time` (C++ member), 46
`WIFI_SCAN_HOME_CHANNEL_DWELL_DEFAULT_TIME` (C macro), 22
`wifi_scan_method_t` (C++ enum), 32
`wifi_scan_method_t::WIFI_ALL_CHANNEL_SCAN` (C++ enumerator), 33
`wifi_scan_method_t::WIFI_FAST_SCAN` (C++ enumerator), 32
`WIFI_SCAN_PARAMS_DEFAULT_CONFIG` (C macro), 22
`wifi_scan_threshold_t` (C++ struct), 46
`wifi_scan_threshold_t::authmode` (C++ member), 46
`wifi_scan_threshold_t::rssi` (C++ member), 46
`wifi_scan_threshold_t::rssi_5g_adjustment` (C++ member), 46
`wifi_sort_method_t` (C++ enum), 33
`wifi_sort_method_t::WIFI_CONNECT_AP_BY_SECURITY` (C++ enumerator), 33
`wifi_sort_method_t::WIFI_CONNECT_AP_BY_SIGNAL` (C++ enumerator), 33
`wifi_sta_info_t` (C++ struct), 50
`wifi_sta_info_t::is_mesh_child` (C++ member), 51
`wifi_sta_info_t::mac` (C++ member), 50
`wifi_sta_info_t::phy_11a` (C++ member), 50
`wifi_sta_info_t::phy_11ac` (C++ member), 51
`wifi_sta_info_t::phy_11ax` (C++ member), 51
`wifi_sta_info_t::phy_11b` (C++ member), 50
`wifi_sta_info_t::phy_11g` (C++ member), 50
`wifi_sta_info_t::phy_11n` (C++ member), 50
`wifi_sta_info_t::phy_lr` (C++ member), 50
`wifi_sta_info_t::reserved` (C++ member), 51
`wifi_sta_info_t::rssi` (C++ member), 50
`WIFI_STATIS_ALL` (C macro), 25
`WIFI_STATIS_BUFFER` (C macro), 25
`WIFI_STATIS_DIAG` (C macro), 25
`WIFI_STATIS_HW` (C macro), 25
`WIFI_STATIS_PS` (C macro), 25
`WIFI_STATIS_RXTX` (C macro), 25
`wifi_storage_t` (C++ enum), 37
`wifi_storage_t::WIFI_STORAGE_FLASH` (C++ enumerator), 37
`wifi_storage_t::WIFI_STORAGE_RAM` (C++ enumerator), 37
`wifi_tx_info_t` (C++ struct), 71
`wifi_tx_info_t::data` (C++ member), 72
`wifi_tx_info_t::data_len` (C++ member), 72
`wifi_tx_info_t::des_addr` (C++ member), 71
`wifi_tx_info_t::ifidx` (C++ member), 71
`wifi_tx_info_t::rate` (C++ member), 72
`wifi_tx_info_t::src_addr` (C++ member), 71
`wifi_tx_info_t::tx_status` (C++ member), 72
`wifi_tx_rate_config_t` (C++ struct), 70

```

wifi_tx_rate_config_t::dcm (C++ member),
    70
wifi_tx_rate_config_t::ersu (C++ member),
    70
wifi_tx_rate_config_t::phymode (C++ mem-
    ber), 70
wifi_tx_rate_config_t::rate (C++ member),
    70
wifi_tx_status_t (C++ enum), 46
wifi_tx_status_t::WIFI_SEND_FAIL    (C++
    enumerator), 46
wifi_tx_status_t::WIFI_SEND_SUCCESS
    (C++ enumerator), 46
WIFI_VENDOR_IE_ELEMENT_ID (C macro), 22
wifi_vendor_ie_id_t (C++ enum), 38
wifi_vendor_ie_id_t::WIFI_VND_IE_ID_0
    (C++ enumerator), 38
wifi_vendor_ie_id_t::WIFI_VND_IE_ID_1
    (C++ enumerator), 38
wifi_vendor_ie_type_t (C++ enum), 37
wifi_vendor_ie_type_t::WIFI_VND_IE_TYPE_ASSOC_REQ
    (C++ enumerator), 38
wifi_vendor_ie_type_t::WIFI_VND_IE_TYPE_ASSOC_RESP
    (C++ enumerator), 38
wifi_vendor_ie_type_t::WIFI_VND_IE_TYPE_BEACON
    (C++ enumerator), 37
wifi_vendor_ie_type_t::WIFI_VND_IE_TYPE_PROBE_REQ
    (C++ enumerator), 38
wifi_vendor_ie_type_t::WIFI_VND_IE_TYPE_PROBE_RESP
    (C++ enumerator), 38
wps_fail_reason_t (C++ enum), 45
wps_fail_reason_t::WPS_AP_FAIL_REASON_AUTH
    (C++ enumerator), 45
wps_fail_reason_t::WPS_AP_FAIL_REASON_CONFIG
    (C++ enumerator), 45
wps_fail_reason_t::WPS_AP_FAIL_REASON_MAX
    (C++ enumerator), 46
wps_fail_reason_t::WPS_AP_FAIL_REASON_NORMAL
    (C++ enumerator), 45

```